Knowledgebase Context and Docs

Consolidated printable PDF generated on June 18, 2026

Sources: 25 markdown files, starting with CONTEXT.md and followed by docs files in product, ADR, agent, and handoff order.

Source Files

CONTEXT.md

docs/product-core.md

docs/mvp-frontend-core-loop.md

docs/adr/0001-separate-referents-tags-and-knowledge-entries.md

docs/adr/0002-use-type-detail-tables-for-knowledge-entries.md

docs/adr/0003-model-knowledge-slots-as-singular-workflow-requests.md

docs/adr/0004-knowledge-entries-uniquely-represent-same-typed-referents.md

docs/adr/0005-link-every-user-to-a-person-entry.md

docs/adr/0006-store-smart-storage-proposals-as-durable-silver-records.md

docs/adr/0007-store-smart-storage-proposals-as-persistence-agnostic-domain-contracts.md

docs/adr/0008-use-contribution-submissions-as-smart-storage-parents.md

docs/agents/domain.md

docs/agents/issue-tracker.md

docs/agents/triage-labels.md

docs/handoffs/durable-bookmarked-knowledge-pages.md

docs/handoffs/durable-direct-contributions-gold-entries.md

docs/handoffs/durable-notification-inbox.md

docs/handoffs/durable-pinned-knowledge-pages.md

docs/handoffs/durable-smart-storage-spine.md

docs/handoffs/durable-subscriptions-notification-readiness.md

docs/handoffs/header-sidebar-navigation.md

docs/handoffs/mvp-answer-feed.md

docs/handoffs/mvp-entry-and-slot-cards.md

docs/handoffs/mvp-frontend-contracts-and-routing.md

docs/handoffs/mvp-knowledge-navigator.md

Knowledgebase

This context describes the core domain language for a knowledgebase that helps Christian organizations store, retrieve, and work from human knowledge. The application treats named things in the real world as first-class references for organizing answers and work.

Language

Knowledge Context: The set of Tags that locate a Knowledge Entry and shape which Knowledge Requests it can help answer. _Avoid_: Context, folder, AI context window

Active Knowledge Context: The Knowledge Context in effect for a User's current Knowledge Page. Active Knowledge Context describes where the User currently is in the application; it is distinct from the User's historical Recognized Context. _Avoid_: Recognized Context, role, personal saved set

Global Knowledge Context: The Knowledge Context available to every user and organization by default, containing the Referents all users and organizations must acknowledge as infallible Recognized Context. In this application, the Global Knowledge Context contains Scripture. _Avoid_: Global Scripture Context, public folder, Bible folder

Global Search: Search across everything the current User can access, independent of the current Active Knowledge Context. Global Search is not limited to public or Global Knowledge Context material. _Avoid_: Context search, public search, Global Knowledge Context search

Tag: A named, typed pointer to a Referent and the intended set of all knowledge about it, including content already stored in the application and relevant knowledge that has not yet been stored or tagged. A Tag is canonical to its Referent rather than scoped separately per user or organization. _Avoid_: Label, keyword, local tag

Knowledge Type: A typed shape of knowledge the application understands, such as a Bible Passage, Question, Quote, Sermon, Lesson, Event, Person, Organization, Group, Place, or Words. Knowledge Types may be added over time as the application learns to recognize more specific kinds of Referents. _Avoid_: Schema, entity type, file type

Knowledge Entry: A typed, contextualized unit of knowledge in the application, whether created directly by a user or produced through Smart Storage. A Knowledge Entry represents one Referent of the same Knowledge Type and its Knowledge Context is constituted by its Tags. _Avoid_: File, document, source, post

Entry Representation: A content form or media form through which a Knowledge Entry is expressed, such as editable rich text, plain text, audio, video, an external URL, or a stored file. An Entry Representation does not change the Knowledge Type or Referent represented by the Knowledge Entry. _Avoid_: Source, Knowledge Type, file type

Primary Representation: The default Entry Representation the application uses when it needs to show, open, preview, or play a Knowledge Entry and cannot present every representation at once. Primary Representation is a display and interaction default, not a claim that other representations are less true or less preserved. _Avoid_: Source, canonical version, original file

Representation Role: The role an Entry Representation plays for a Knowledge Entry, such as manuscript, slides, transcript, recording, thumbnail, primary content, or supporting material. Representation Role is distinct from representation kind because kind describes the medium while role describes how the representation functions for the entry. _Avoid_: Entry Representation, file type, Knowledge Type

Visibility Scope: The audience allowed to access a Knowledge Entry, such as one user, an organization, a group, a network of organizations, or everyone. _Avoid_: Public/private, global context, sharing folder

Review Scope: The audience allowed to view or manage Bronze Sources, Smart Storage Runs, Smart Storage Proposals, or other pending review material before it becomes Gold Layer knowledge. Review Scope may differ from the intended Visibility Scope of the resulting Knowledge Entry. _Avoid_: Visibility Scope, Delivery Target, approver list

Delivery Target: The User, Person, Group, Organization, or other recipient target a contribution or action is sent to or notifies. Delivery Target is distinct from Visibility Scope and Knowledge Context; receiving a notification does not define who may access the Knowledge Entry or which Tags it references. _Avoid_: Visibility Scope, audience, permission, send to page

Type Behavior: The versioned domain behavior the application applies to a Knowledge Type, including how entries of that type are recognized, enriched, related, displayed, stored, reviewed, scored, or scoped. _Avoid_: Implementation, schema behavior

Answer: A Knowledge Entry considered as something that can help satisfy future Knowledge Requests in whole or in part. _Avoid_: AI response, chat reply

Answer Feed: The mixed Knowledge Page surface that presents relevant Knowledge Entries and Knowledge Slots for the current Knowledge Context. _Avoid_: Entries list, search results, slot list

Human Weight: A rating of how strongly a Knowledge Entry reflects human substance that artificial intelligence cannot cheaply counterfeit: agency, excellence, judgment, lived use, or divine inspiration, scored from Slop at 0 to Soul at 100. Whether and how Human Weight applies depends on the Knowledge Entry's Knowledge Type. _Avoid_: AI score, quality score, robot score

Human Weight Expectation: The Type Behavior or workflow-specific standard that determines whether low Human Weight is acceptable, merely informative, or a concern for a Knowledge Entry. Human Weight Expectation levels are none, informative, expected, and required. _Avoid_: AI detector, quality requirement, grade

Human Weight Evidence: The supporting signals used to assign or revise Human Weight for a Knowledge Entry, such as credited attribution, provenance, human review, lived use, ratings, or recognition by users and organizations. _Avoid_: Like count, popularity score, raw engagement

Human Weight Feedback: User-provided feedback on a Knowledge Entry that becomes Human Weight Evidence without directly determining Human Weight. _Avoid_: Like, vote, popularity rating

Evidence Maturity: A measure of how settled the Human Weight for a Knowledge Entry is based on the amount and reliability of its Human Weight Evidence. _Avoid_: AI confidence, popularity, certainty

Feed Priority: The derived ordering value used by the Answer Feed to balance Human Weight, Evidence Maturity, freshness, feedback needs, Knowledge Context fit, and contribution opportunities. _Avoid_: Human Weight, popularity rank, recency sort

Context Expertise: A derived estimate of a User's or Person's reliable ability to author, recognize, place, or contribute useful future Answers within a Knowledge Context. _Avoid_: Human Weight, popularity, Role, permission

Context Expertise Evidence: The bounded contribution signals used to assign or revise Context Expertise, such as authoring, sharing, confirming, curating, fulfilling, or giving feedback on an Answer within a Knowledge Context. _Avoid_: Human Weight Evidence, view count, repeated reference, raw engagement

Context Expertise Maturity: A measure of how settled a User's or Person's Context Expertise is based on the amount, reliability, and outcomes of their Context Expertise Evidence. _Avoid_: Raw contribution count, certainty, popularity

Context Expertise Inheritance: The way Context Expertise earned in one Knowledge Context can inform rankings in related broader or narrower Knowledge Contexts without treating every Tag overlap as equal. _Avoid_: Global reputation, universal authority, tag popularity

Context Expert: A User or Person surfaced because their Context Expertise is high within a Knowledge Context. _Avoid_: Top contributor, admin, official authority, title

Weight-Bearing Knowledge Type: A Knowledge Type whose entries can meaningfully carry Human Weight because they express human substance, ingenuity, craft, judgment, lived use, or divine inspiration. _Avoid_: Human-weight-enabled type, scoreable type

Non-Weight-Bearing Knowledge Type: A Knowledge Type whose entries do not meaningfully express human ingenuity and therefore do not carry Human Weight. _Avoid_: Zero-weight type, low-quality type

Knowledge Request: A usually transient user request for knowledge, help, or work that is answered from the application's Knowledge Entries within an applicable Knowledge Context. A Knowledge Request is not itself a Knowledge Entry unless intentionally Contributed as a Question or another Knowledge Type. _Avoid_: Prompt, query, chat message

Knowledge Composer: A user-facing input surface for asking, searching, contributing, or shaping the Active Knowledge Context from one place, with the user's submission determining whether it creates a Knowledge Request, a Contribution, or context changes. _Avoid_: Chat box, search box, Add Tags control

Question: A Knowledge Type for a durable question mapped to a Knowledge Context, preserving useful information about what parts of that Knowledge Context need to be connected or answered. A Question may be selected as a Tag to define a narrower Knowledge Context for future Knowledge Requests and Answers. _Avoid_: Prompt, query

Question Space: The set of Knowledge Requests that could meaningfully be asked within a Knowledge Context. _Avoid_: Search space, prompt space, chat history

Knowledge Page: A user-facing location in the application where a User works within or navigates to a Knowledge Context, Referent, Knowledge Entry, Organization, user relationship, or knowledge-oriented view. A Knowledge Page is distinct from the User's active Role; the same User may visit the same Knowledge Page while acting in different Roles. _Avoid_: Place, route, screen

Knowledge Page Shell: The consistent user-facing frame shared by Knowledge Pages, providing page identity, Knowledge Context controls, Explore, Contribute, and work or feed regions while allowing each page type to supply page-specific content. _Avoid_: Knowledge Context, bespoke page layout, route template

Page-Specific Module: A bounded part of a Knowledge Page that presents content or workflow unique to the current page type without replacing the shared Knowledge Page Shell. _Avoid_: Bespoke page layout, custom page shell, hero

Page-Specific Subroute: A subordinate route reached from a Knowledge Page for page-specific workflow or settings that should not occupy the shared Knowledge Page Shell. _Avoid_: Nested Knowledge Page, custom page, feature page

User View: A user-scoped view whose contents are assembled around the current User's activity, responsibilities, preferences, or account state rather than around a shared Knowledge Context or Referent. _Avoid_: Knowledge Page, personal Knowledge Context, private page

Bookmark: A User relationship to a Knowledge Page that saves it in the User's personal saved set for later reference and may inform user-specific tagging or Knowledge Context recommendations. Bookmarking does not place the Knowledge Page in sidebar navigation or subscribe the User to notifications about it. _Avoid_: Pin, subscription, notification, favorite

Pinned Knowledge Page: A User relationship to a Knowledge Page that keeps the Knowledge Page easy to return to, especially from sidebar navigation. Pinning a Knowledge Page does not itself subscribe the User to notifications about that Knowledge Page. _Avoid_: Subscription, notification, role, folder

Default Pinned Knowledge Page: A Pinned Knowledge Page automatically seeded from a User's affiliation with a School, Church, Family, or Community. A User may unpin a Default Pinned Knowledge Page, and that suppression should persist unless the User manually pins it again. _Avoid_: Required page, role, permanent navigation

Dashboard: The user-facing place for Explore and Contribute when the Knowledge Navigator has no active Tags and the user is located in the Global Knowledge Context. _Avoid_: Home page, global context page

Topic: A Knowledge Type for a named subject of discussion that Knowledge Entries can address, argue about, explain, illustrate, or apply. _Avoid_: Context page, forum, category

Series: A Knowledge Type for a named collection or sequence of related Knowledge Entries, such as sermons, lessons, books, poems, stories, songs, or Events. _Avoid_: Topic, folder, collection

Context Page: A user-facing place for Explore and Contribute within a Knowledge Context determined by two or more active Tags in the Knowledge Navigator. _Avoid_: Topic, channel, folder, Dashboard, Referent Page

Referent Page: The user-facing place for Explore and Contribute focused on one active Tag and the Referent it points to. A Referent Page is reached by going to a Tag rather than by opening a Knowledge Entry. _Avoid_: Knowledge Entry page, Tag page, Folder View

Knowledge Navigator: The user-facing control for selecting active Tags and thereby determining the current Knowledge Context. When no Tags are active in the Knowledge Navigator, the current location is the Global Knowledge Context. _Avoid_: Sidebar, filter, breadcrumb

Words: The base Knowledge Type used for a Referent when no more specific Knowledge Type is recognized by the application. _Avoid_: Text, document, generic

Bible Passage: A Knowledge Type for Scripture references, independent of translation wording and citation-string formatting, including a single verse, one verse range, a chapter, a larger passage, or a set of passages across multiple books of the Bible. A Bible Passage Referent may be a subset of another Bible Passage Referent. _Avoid_: Bible verse, Scripture tag, translation-specific passage, raw citation string

Bible Translation: A textual rendering, edition, or source text of Scripture that may represent the words of a Bible Passage without changing the identity of the Bible Passage Referent. A Bible Translation may be known to the application before its full text is available. _Avoid_: Translation Knowledge Type, separate passage

Quote: A Knowledge Type for a cited excerpt from a larger Source or Knowledge Entry that can stand as its own Referent within a Knowledge Context. _Avoid_: Clip, snippet, excerpt

Sermon: A Knowledge Type for a preached teaching that can be represented by audio, video, transcript, notes, or another Source derived from the act of preaching. _Avoid_: Talk, sermon clip

Essay: A Knowledge Type for a written composition or assigned written work, initially understood as a named wrapper over Words. _Avoid_: Paper, document

Poem: A Knowledge Type for a named poetic work, initially understood as a named wrapper over Words. _Avoid_: Poetry text, document

Song: A Knowledge Type for a named musical work, initially understood as a named wrapper over Words. _Avoid_: Lyrics, track

Book: A Knowledge Type for a named written work published or treated as a book, initially understood as a named wrapper over Words. _Avoid_: Text, volume

Short Story: A Knowledge Type for a named short fictional narrative, initially understood as a named wrapper over Words. _Avoid_: Story, text

Comment: A Knowledge Type for a response to another Knowledge Entry, used when Words-like content needs threaded or relational response behavior. _Avoid_: Reply, note

Prayer Request: A Knowledge Type for a request for prayer, especially within a church, family, group, or community Knowledge Context. _Avoid_: Announcement, comment, note

Event: A Knowledge Type for a scheduled occurrence that may request RSVP entries or connect other Knowledge Entries to a real-world meeting, class, service, or gathering. _Avoid_: Calendar item, appointment

RSVP: A Knowledge Type for a person's response to an Event invitation. _Avoid_: Attendance, signup

Lesson: A Knowledge Type for a reusable plan for teaching or learning, which may be connected to one or more scheduled Events when taught or used. _Avoid_: Lesson plan, class notes

Person: A Knowledge Type for a human being who may be referenced as an author, teacher, student, speaker, invitee, commenter, or other participant, whether or not that person is also a User. _Avoid_: User, account, profile

Role: The relation of a Person to a Knowledge Type or Knowledge Entry, such as author, teacher, student, speaker, parent, or invitee. _Avoid_: Knowledge Type, user type

Active Role: The Role a User is currently acting in while using the application, such as teacher, student, parent, church member, or administrator. Active Role may be unset in global contexts; when set, there is only one Active Role at a time, and it remains distinct from the User's current Knowledge Page or Active Knowledge Context. _Avoid_: Knowledge Page, Active Knowledge Context, account

User: A person with access to the application through an account. Every User is linked to one Person Knowledge Entry so the User can be tagged through that Person, but not every Person is a User. _Avoid_: Person, author, participant

Contact Identity: A contact value, such as an email address, that can support matching or claiming a Person when ownership is proven without defining the Person's identity by itself. _Avoid_: Person, User, account identity

Organization: A Knowledge Type for a collective body recognized by the application, including a School, Church, Family, or Community, that can recognize Tags, receive Knowledge Slots, and participate in Visibility Scopes. _Avoid_: Account, workspace, tenant

Group: A Knowledge Type for a collection of People, whether or not each Person is linked to a User account. _Avoid_: User group, organization, audience

Membership: The relationship between a Person and an Organization or Group, whether or not that Person is linked to a User account. _Avoid_: Invitation, account membership, user group

Pending Membership: A Membership for a Person who has not yet been linked to a User account with proven identity. _Avoid_: Invitation, invite, placeholder user

Membership Claim: The act of connecting an existing Membership for a Person to the User who has proven they are that Person, including through a verified Contact Identity. _Avoid_: Sign-up, invitation acceptance, account creation

School: An Organization associated with formal teaching and learning. A User may sign up through association with a School. _Avoid_: Class, campus

Church: An Organization associated with Christian worship, teaching, fellowship, and ministry. A User may sign up through association with a Church. _Avoid_: Ministry, congregation

Family: An Organization formed by grouping people into a household or family unit. _Avoid_: Household, group

Community: An Organization inferred from a person's hometown or place-based association. _Avoid_: Hometown, locality

Place: A Knowledge Type for a location that can anchor a Community, Event, Organization, or other Knowledge Entry. _Avoid_: Address, geography model

Type Reclassification: The refinement of a Tag from Words to a more specific Knowledge Type when the Referent's identity remains the same. _Avoid_: Retagging, duplicate tag

Referent: The thing a Tag points to, identified by both name and Knowledge Type so similarly named things remain distinct. A Referent may exist before any Knowledge Entry represents it. _Avoid_: Entity, item, entry

Represented Referent: The same-typed Referent a Knowledge Entry uniquely expresses or records. A Referent may have at most one Knowledge Entry that represents it. _Avoid_: Primary Referent, subject, entity

Referent Identity Scope: The scope within which a Referent's identity should be matched as the same thing, such as globally public, organization-scoped, group-scoped, or user-scoped. Referent Identity Scope is distinct from Visibility Scope because identity matching and access control answer different questions. _Avoid_: Visibility Scope, public/private, audience

Represents: The relationship between a Knowledge Entry and its Represented Referent. _Avoid_: Is, contains, owns

References: The relationship created when a Knowledge Entry includes a Tag in its Knowledge Context, pointing from that entry to another Referent. _Avoid_: Contains, belongs to, filed under

Virtual File System: The user-facing model in which Tags behave like folders and Knowledge Entries behave like files that can appear in many folders at once. _Avoid_: Directory tree, physical storage

Explore: To make a Knowledge Request and browse the Answers surfaced within the mapped Knowledge Context. _Avoid_: Search, chat, browse

Contribute: To add a future Answer to the knowledgebase, either by submitting a Source, creating a Knowledge Entry directly, or responding to an existing Knowledge Entry. _Avoid_: Upload, post, store, work

Contribution Submission: A single user intent to Contribute, collecting the material and choices submitted from a composer before it is posted directly or processed through Smart Storage. A Contribution Submission may include multiple Sources and contribution guidance, but it is not itself a Knowledge Entry. _Avoid_: Source, Knowledge Entry, upload bundle

Primary Intended Entry: The one Knowledge Entry a Contribution Submission is primarily meant to create or update. Smart Storage may propose additional derived Knowledge Entries from the same Contribution Submission without changing the Primary Intended Entry. _Avoid_: Parent entry, main Source, bundle entry

Authored Text Source: User-authored or user-pasted text submitted as substantive raw material in a Contribution Submission. Authored Text Source is a kind of Source, distinct from the Words Knowledge Type. _Avoid_: Words, body text, editor text

Contribution Note: Non-substantive guidance a User provides to explain or steer a Contribution Submission without itself becoming represented knowledge by default. _Avoid_: Source, Words, entry body

Link Preview: Fetched metadata that helps a User recognize an external URL in a Contribution Submission or Entry Representation. A Link Preview is not a Knowledge Entry, Factual Provenance, or Human Weight Evidence by itself. _Avoid_: Knowledge Entry, evidence, Source

Knowledge Slot: A predefined request for one Knowledge Entry of a specified Knowledge Type within a specified Knowledge Context. A Knowledge Slot directs a user, group, organization, network, or open audience to Contribute a missing future Answer. _Avoid_: Todo, assignment, prompt, bounty, call to action

Fulfillment: The state of a Knowledge Slot after it has received the requested Knowledge Entry. _Avoid_: Completion, submission, done

Subscription: A user's standing interest in activity within a Knowledge Context, Organization, Knowledge Slot, or Event. _Avoid_: Alert, follow, notification setting

Notification: A user-visible notice that relevant activity occurred within a Subscription, assigned Knowledge Slot, or Event participation. _Avoid_: Message, feed item

Source: Raw user-provided material submitted to the application, such as an uploaded file, pasted text, note, URL, or other input. _Avoid_: File, upload, raw data

Bronze Layer: The preserved raw record of submitted Sources, stored as close as possible to their original form. In Smart Storage, Bronze Layer is to Source as Silver Layer is to Smart Storage Proposal and Gold Layer is to Knowledge Entry. _Avoid_: Bronze data, raw folder

Smart Storage: The AI-assisted process of preserving a Source, identifying relevant Tags, and refining or enriching the Source toward one or more pieces of structured knowledge the application understands. Smart Storage can propose Gold Layer knowledge, but user confirmation is required before that knowledge becomes Gold. _Avoid_: Upload, import, ingestion

Factual Enrichment: The Smart Storage act of using external factual information to refine a Source when the Source points to knowledge it does not itself contain. Factual Enrichment is not measured by Human Weight unless it becomes part of a user-confirmed Knowledge Entry. _Avoid_: AI authorship, generated answer, hallucination

Factual Provenance: The evidence trail for an enriched fact in a Smart Storage Proposal or Knowledge Entry, identifying the external URL, Knowledge Entry, or model-only basis for the proposed fact. Factual Provenance may support the whole proposed entry or a specific factual field without changing who confirmed or owns the whole Knowledge Entry. _Avoid_: Source, ownership, proof

Proposal Confidence: A coarse Smart Storage review signal for how well a Smart Storage Proposal or enriched factual field appears supported by its type match, candidates, and Factual Provenance. Proposal Confidence guides review but is not Human Weight, truth, or Gold Layer confirmation. _Avoid_: Human Weight, truth score, AI quality score

Smart Storage Proposal: A durable Silver Layer candidate for creating or updating one Knowledge Entry from Smart Storage before user confirmation, expressed in Smart Storage Contract terms rather than persistence-specific write operations. A Smart Storage Proposal may be accepted, rejected, or edited before it affects Gold Layer knowledge. _Avoid_: Draft entry, AI answer, unconfirmed Knowledge Entry

Smart Storage Run: A record of one Smart Storage attempt against a Source, preserving the contract versions, request-specific input snapshot, and raw model output that produced, failed to produce, or helped produce Smart Storage Proposals. _Avoid_: LLM call, job log, parser output

Smart Storage Contract: The versioned stable domain contract Smart Storage gives to an LLM so it can match Sources to Knowledge Types and propose structured knowledge. A Smart Storage Contract includes the reusable domain shape needed for recognition and proposal generation without exposing the raw persistence schema or request-specific input as the model's contract. _Avoid_: Database schema prompt, raw schema, implementation prompt

Reprocessing: The act of revisiting existing Sources or Knowledge Entries when the application gains new Knowledge Types or improved recognition, so previously untyped knowledge can be represented more specifically. _Avoid_: Migration, retagging, cleanup

Upgrade Candidate: A Source or Knowledge Entry that may be refined further because a new or improved Knowledge Type can represent knowledge it previously held only indirectly. _Avoid_: Demoted entry, failed gold, invalid entry

Silver Layer: A durable intermediate refinement layer where Source data is cleaned, structured, reviewed, or prepared beyond its raw form before it becomes typed and formatted knowledge the application understands. In Smart Storage, Silver Layer records are Smart Storage Proposals. _Avoid_: Final answer, typed knowledge, truth

Gold Layer: Knowledge Entries that have been represented according to the most specific Knowledge Types the application currently understands. In Smart Storage, Gold Layer records are confirmed Knowledge Entries. _Avoid_: Final truth, human-approved truth, business-ready data

Recognized Context: The union of Knowledge Contexts where a User or Organization has taken meaningful action across their activity history, represented by the typed Tags they have interacted with. Recognized Context indicates Referents the User or Organization recognizes as meaningful within their domain, but it is not the same as the User's Active Knowledge Context. _Avoid_: Truth model, personal truth, organization truth

Product Core

This product is a knowledgebase for Christian users and organizations that treats named things in the real world as first-class references for storing, finding, and doing work. It can be understood as a smart Google Drive or virtual file system: Tags behave like folders, Knowledge Entries behave like files, and an entry can appear in many folders because it can reference many Referents.

The application is not only a repository. It is intended to become the place where people ask for knowledge, contribute future answers, and do day-to-day work from the same context where prior answers are found.

Product Commitments

The application is built for Christians who affirm the inerrancy of Scripture. The Global Knowledge Context is available to every user and organization by default, and in this application it contains Scripture because Scripture is the infallible Recognized Context all users and organizations must acknowledge.

The application should promote human thought over automated output while still using AI for useful recognition, extraction, structuring, and retrieval. AI helps store and surface knowledge; it does not replace human judgment.

Weight-bearing Knowledge Entries are rated by Human Weight on a Slop to Soul scale from 0 to 100. Human Weight is interpreted through the entry's Knowledge Type, and each weight-bearing Type Behavior should define the credited human role: a Quote should credit the human substance of the person to whom the quote is attributed, while a Words entry should credit the user or person who authored those words. Some workflow types, such as RSVP, may have no Human Weight because they do not meaningfully express human ingenuity. Bible passages have full Soul because they are inspired by the Holy Spirit.

The MVP weight-bearing Knowledge Types are Words, Question, Quote, Sermon, Essay, Poem, Song, Book, Short Story, Lesson, Comment, Prayer Request, Series, and Event. The MVP non-weight-bearing Knowledge Types are RSVP, Person, Organization, Group, Place, and Topic. Bible Passage is a special case: it has full Soul as Scripture, but it is not an authorable Knowledge Entry type in the MVP.

Low Human Weight means low human substance, but whether that is bad depends on the Human Weight Expectation for the Knowledge Type and workflow. Type Behavior should provide the default Human Weight Expectation, while workflows such as Knowledge Slots may override that expectation when they request a specific kind of human work. For example, a low-Human Weight student Essay is a concern because the workflow expects the student's authored human substance and may indicate AI-written work, while a useful AI-assisted summary may be low Human Weight without being harmful.

Human Weight Expectation has four levels: none, informative, expected, and required. None means Human Weight does not apply. Informative means Human Weight is useful context, but low weight is not a problem by itself. Expected means human substance is normally expected and low weight should be surfaced as a possible concern. Required means human substance is required by the workflow and low weight should trigger review or warning. For example, RSVP is none, a generic AI-assisted summary may be informative, an ordinary Essay may be expected, and a student-submitted Essay for a Knowledge Slot may be required.

The initial Human Weight bands are Slop at 0-19, Assisted at 20-39, Shaped at 40-59, Substantial at 60-79, Weighty at 80-94, and Soul at 95-100. These bands are interpretive anchors for the product and implementation; they may be refined as the product learns how users recognize human substance.

Human Weight is a recalculable current estimate, not immutable entry metadata. The product should preserve Human Weight Evidence and enough evaluation context to revise scores when the Human Weight definition improves. User feedback, ratings, recognition, or other gamified signals may contribute evidence, but they should support the rating rather than directly determine it or replace the Type Behavior's credited human role.

Evidence Maturity should be tracked separately from Human Weight. Human Weight answers how much human substance the entry carries; Evidence Maturity answers how settled that rating is. A promising new entry may have high estimated

Human Weight with low Evidence Maturity, while a long-used reviewed entry may have the same Human Weight with high Evidence Maturity.

Human Weight Feedback should begin with a small set of evidence-oriented responses, such as recognize, used, not human, and wrong context. Recognize and used may also be derived from other product activity when the application has enough data to infer them responsibly, rather than requiring explicit feedback every time.

Answer Feed ranking should use Feed Priority: a derived ordering value that prioritizes Human Weight while also giving low-Evidence Maturity weight-bearing entries enough exposure to gather Human Weight Evidence from users. Human Weight calculation may happen asynchronously, such as through scheduled recalculation, when evidence changes or the scoring definition is refined.

For multi-Source Contribution Submissions, Human Weight belongs to each accepted Gold Layer Knowledge Entry rather than to the Contribution Submission or Bronze Sources themselves. Sources, Entry Representations, provenance, authorship, review, and usage may supply Human Weight Evidence for the resulting entries.

Core Model

A Knowledge Entry is a typed, contextualized unit of knowledge. It represents one Referent of the same Knowledge Type and references other Referents through its Tags. Those Tags constitute the entry's Knowledge Context.

The canonical Tag for a Knowledge Entry's Represented Referent should be included among the entry's Tags. This lets one Tag relationship model both the entry's own navigable identity and the other Referents it references in its Knowledge Context.

A Tag is a named, typed pointer to a Referent and to the intended set of knowledge about that Referent. A Referent is identified by name plus Knowledge Type, so similarly named things remain distinct, such as `Charlotte's Web, book` and `Charlotte's Web, essay`.

Referents, Tags, and Knowledge Entries should remain distinct. A Referent is the stable identity of the thing being pointed at, a Tag is the navigable handle that points to that Referent, and a Knowledge Entry is content or work that represents one same-typed Referent and references other Referents through Tags. A Referent may exist without any Knowledge Entry representing it, and a Referent may have at most one Knowledge Entry that represents it.

Creating a Knowledge Entry should create or reuse the Represented Referent and its canonical Tag. If the user attempts to create a Knowledge Entry for a Referent that already has a representing Knowledge Entry, such as `Moby Dick, Book`, the app should not create a duplicate; it should show that the Referent already exists, offer a link to that Referent's Knowledge Page, and guide the user toward tagging, editing, or proposing an authorized update as appropriate.

Most edits offered from a Referent's Knowledge Page should edit the represented Knowledge Entry, its Entry Representations, Tags, aliases, provenance, or type-specific fields rather than the bare Referent identity. Editing Referent identity should be a special permissioned operation for identity correction, aliasing, merge/split, or Type Reclassification because it affects every Tag and entry that points to that Referent.

The identity fields that determine whether a Referent already exists are Knowledge Type-specific. Many types may use type plus title or name, while others may require additional identity fields such as author, preacher, date, location, or source. Composer and Smart Storage duplicate checks should use the relevant Type Behavior identity rules and tell the User when the intended Referent already exists.

Referent Identity Scope should also be governed by Type Behavior. Some Knowledge Types or specific entries represent globally public things that should participate in global duplicate matching, while others represent organization-, group-, or user-scoped things whose identity should remain local unless intentionally published or shared more broadly. Context-dependent types such as Essay or Lesson should use source provenance, author, publication status, current organization context, Visibility Scope, and explicit user choice to decide whether the intended Referent is public or scoped.

Widening a Knowledge Entry's Visibility Scope should not automatically widen its Referent Identity Scope. If a scoped Referent later appears to become a public Referent or match an existing public Referent, the app should route that through identity review, such as merge, alias, split, or Type Reclassification, rather than silently changing identity scope.

Composer and Smart Storage should infer Referent Identity Scope from Type Behavior and context when the answer is clear, such as public Books or scoped Prayer Requests. The User should be asked only for context-dependent Knowledge Types or ambiguous evidence where identity scope changes duplicate matching, visibility expectations, or future reuse.

Composer Smart Storage may flag possible duplicate Referents or identity ambiguity, but Referent merge and split should belong to a separate permissioned review workflow rather than being performed automatically from contribution acceptance.

Tags should be canonical per Referent, not duplicated per user or organization. User and organization relationships to a Tag should be represented through Recognized Context, subscriptions, aliases, visibility, or other local relationships rather than by creating separate Tags for the same Referent.

The first schema pass should include Tag Recognition so users and organizations can record that a canonical Tag is meaningful to them without creating local duplicate Tags.

Active Knowledge Context is the Knowledge Context in effect for the user's current Knowledge Page. Recognized Context is historical: the union of Knowledge Contexts where the user or organization has taken meaningful action, recorded through canonical Tags without making those Tags active for every request.

Plain Knowledge Page visits should not add to Recognized Context by default. Page visits are analytics and may serve as weak recommendation evidence, while Recognized Context should come from intentional actions such as bookmarking, pinning, subscribing, contributing, commenting, asking from a Knowledge Context, fulfilling a Knowledge Slot, editing Tags, sharing, or assigning work.

Bookmarked Knowledge Pages should not appear directly in the primary sidebar by default. They belong in user/account navigation, such as the user's Profile or a user dropdown from the profile control, while the sidebar remains focused on Pinned Knowledge Pages and primary navigation.

Bookmarks should be available as a section or tab on the User's profile, with the sidebar avatar menu offering a shortcut to that profile section. Bookmarks do not need a separate User View unless the saved set becomes large or workflow-heavy enough to require one.

Knowledge Pages and User Views should remain distinct in navigation. Knowledge Pages are grounded in a shared Knowledge Context, Referent, Knowledge Entry, Organization, or other world-facing knowledge object. User Views are assembled around the current User's activity, responsibilities, preferences, or account state, such as Calendar, Notifications, Settings, or editing the user's profile. A public profile is a Knowledge Page for a Person or User presentation; editing one's own profile is a User View.

Knowledge Pages should share a Knowledge Page Shell with compact page-specific identity. The top of each Knowledge Page should identify the page and expose essential page actions without turning Organization, Profile, Referent, Event, or other page-specific details into a large bespoke hero. The compact identity band may include a tiny Active Knowledge Context summary such as `Global Knowledge Context`, a single Tag label, or a Tag count, but the full interactive Active Tag set should remain in the left rail. Page-Specific Module links or controls may appear in or directly below the compact identity band as low-profile actions. Page-Specific Modules such as an Event guest list or Organization overview should appear as contained expandable sections, dialogs, or Page-Specific Subroutes such as Organization settings.

The shared Knowledge Page Shell should avoid redundant generic headings when the page structure already makes the region clear. The Knowledge Navigator does not need its own header in the standard shell, Dashboard does not need a repeated `School Day` or `Today at Arche Classical Academy` heading above the working layout, and the Answer Feed does not need a separate `Answers` heading when it is already the primary feed region. Suggested Entry or Requested Entry placeholder panels should not appear below the Knowledge Navigator by default when there are no real requested entries.

The left rail of the standard Knowledge Page Shell should stay focused on active Tags and a compact Knowledge Composer for the current context. Active Tags should remain visible, while suggested or available Tags should stay compact, secondary, and capped so they support the Knowledge Navigator without becoming a large browse panel. The rail should not have a separate `Add Tags` control; the Knowledge Composer should support wording such as `Ask a`

`Question or Context` and use typeahead suggestions for Tags and Questions that can be added to the Active Knowledge Context. Selecting a suggestion should add that Tag or Question to the context. Pressing Enter without selecting a suggestion should run a text search for matching Knowledge Entries within the current Knowledge Context rather than changing the Active Knowledge Context. Knowledge Slot or Requested Entry content belongs in the Answer Feed rather than in a separate rail panel because the Answer Feed is a mixed surface made of Knowledge Entries plus Knowledge Slots. Selecting a Knowledge Slot may put the Contribution Editor into a fulfillment state for that slot, but the slot itself should remain represented as a feed item.

Knowledge Slot cards in the Answer Feed should use the same overall card grammar as Knowledge Entry cards so they feel peer-level in the feed. Their distinctive feature should be a very clear missing-content area that names what must be filled in, who or what it is for, and the action to contribute the missing Knowledge Entry.

Knowledge Type filtering belongs with the Answer Feed controls rather than the Knowledge Navigator. Filtering the feed to Songs, Lessons, Sermons, Questions, Requests, or another Knowledge Type narrows which matching feed items are shown; it does not change the Active Knowledge Context. The Knowledge Navigator should remain responsible for active Tags and Questions that define context, while the Answer Feed may provide compact filters such as `All`, `Entries`, `Requests`, and Knowledge Type chips.

The Answer Feed should default to a masonry-style card grid on desktop and collapse to a single stacked column on narrow screens. The feed should avoid a large standalone heading when it is already the primary work region, but it may show a compact control row for counts, Knowledge Type filters, feed kind filters, and sorting.

The Contribution Editor should remain above the Answer Feed in the standard Knowledge Page Shell. It should be compact by default and expand on focus or when the User chooses a Knowledge Slot to fulfill. The collapsed editor should keep the current streamlined input-first design and should not show a bulky metadata strip such as Direct Post, Knowledge Type, or Knowledge Context before the User expands or reviews the contribution. When expanded, the main contribution input should remain first, with contribution metadata such as Knowledge Type, Visibility Scope, Knowledge Context, Direct Post versus Smart Storage, and slot fulfillment state shown afterward as compact editable chips, details, or a preview area.

Bible Passage Referent Pages are the exception to the compact-only identity rule because Scripture text is the substance of the page. They should keep the Scripture Text section prominent, but should not show a generic Bible Passage Overview module by default.

The primary sidebar should carry destination navigation: Dashboard as the first global Knowledge Page, Pinned Knowledge Pages in the middle, and User Views or account navigation at the bottom. The sidebar avatar should be the account-menu entry point for the User's profile, Bookmarks, Settings, and Sign out. The header should stay focused on the user's Active Role and Global Search. The current Knowledge Page or Active Knowledge Context should be presented below the header in the page content area rather than inside the header, so users do not mistake Global Search for context-scoped search. Account controls should live in one place, not duplicated in both sidebar and header.

Global Search should search everything the current User can access, independent of the current Active Knowledge Context. It should not mean public-only search or search limited to the Global Knowledge Context. Search results should make scope legible with labels such as Global, an Organization name, or Personal when needed.

Knowledge Pages may also provide context-scoped Search, Ask, or Explore controls, but those controls should live below the header near the page title or Active Knowledge Context. Context-scoped controls should be visually distinct from header Global Search so users can tell which scope they are using.

When a User selects a Global Search result, the app should navigate to that result's Knowledge Page and synchronize the Knowledge Navigator's active Tags to that page's Active Knowledge Context. For example, opening `Arche Classical Academy` from Global Search lands on Arche's Knowledge Page with Arche's Tag as the single active Tag.

The Knowledge Navigator should be visible on every Knowledge Page because it is the canonical control for the Active Knowledge Context. Its presentation may vary: compact on simple Knowledge Pages and expanded on Dashboard or exploration-heavy pages.

The Knowledge Navigator should not be shown as the main navigator on User Views such as Calendar or Notifications. User Views may show context chips or links inside their items, but those should navigate to the relevant Knowledge Pages rather than making the User View itself context-scoped.

Dashboard should be a fixed first sidebar route for the Global Knowledge Context, not a user-removable Pinned Knowledge Page.

Explore should not be a separate primary sidebar icon for now. Explore is an action available from Dashboard and other Knowledge Pages, while Dashboard is the fixed global Knowledge Page.

Pinned Knowledge Pages should not rely on icon-only recognition. The sidebar may have a compact or collapsed state, but Pinned Knowledge Pages need a visible label affordance beyond hover tooltips because user-pinned pages and default Organization pages are not universally recognizable from icons alone.

On desktop, the sidebar should prefer visible labels for Pinned Knowledge Pages. On smaller screens or constrained layouts, the sidebar may collapse into a compact rail or drawer. When there are more Pinned Knowledge Pages than the available sidebar space can show, overflow should collapse into a concise control such as `+3 more` rather than crowding or shrinking every item.

Default Pinned Knowledge Pages for Organizations should use the specific Organization name as the primary label, such as `Arche Classical Academy`, rather than only generic labels like `My School`. The Organization kind, such as School, Church, Family, or Community, may appear through an icon, secondary label, or grouping.

Default Pinned Knowledge Page seeding should be capped so initial navigation stays calm. When a User has multiple Organizations of the same kind, the app should seed at most the most relevant one per kind and make the others available through pin management or recommendations rather than pinning every affiliation automatically.

The Active Role display in the header should also be the Role switcher. The default global state should have no Active Role selected; switching Active Role should change acting capacity, permissions, prompts, and defaults without navigating the User away from the current Knowledge Page or User View.

Navigating to an Organization Knowledge Page or unambiguous organization-scoped context should default the Active Role to the Role the User assumes within that Organization. If a User has more than one Role in that Organization, the app should require or preserve an explicit choice rather than guessing.

Organization-scoped visibility may default from the current Organization Page even when Active Role is unset or ambiguous. Role-specific authority, prompts, and permissions should require an explicit Active Role when multiple Roles match the same Organization.

Notifications should remain a visible bottom User View icon rather than being hidden inside the avatar menu, because notification count and unread status need a persistent badge.

Calendar should be a bottom User View icon because it is assembled around the current User's roles, assignments, Events, subscriptions, and memberships. Specific Events remain Knowledge Pages; the Calendar itself is a User View.

Settings should live in the sidebar avatar menu rather than as a persistent bottom icon unless future usage proves it needs more prominence. The User's own profile should be reached through the sidebar avatar and edited inline through editable profile fields rather than through a separate Edit Profile view. Organization settings should be reached from the relevant Organization Knowledge Page by Users whose Active Role permits that action.

The base Knowledge Type is Words. When the application does not yet understand a more specific type, a Referent may be represented as Words until that type is added. Later, Type Reclassification can refine the Tag from Words to a more specific Knowledge Type when the Referent's identity is the same.

Knowledge Types are how the application learns new domain behavior. Adding a type means teaching the app how to recognize, relate, display, scope, and work with entries of that type.

Core Loop

The product loop has two main user actions:

1. **Explore**: the user makes a Knowledge Request, and the app maps it to a Knowledge Context where relevant Answers can be browsed.
2. **Contribute**: the user adds a future Answer by submitting a Source, creating a Knowledge Entry directly, or responding to an existing Knowledge Entry.

A Knowledge Request is transient by default. It becomes durable knowledge only when the user intentionally Contributes it as a Question or another Knowledge Type.

A contributed Question should create a Knowledge Entry and represented Question Tag, but it should not automatically move the user into that Question's Knowledge Context. The user may then navigate to or select the Question Tag to Explore and Contribute within the narrower context defined by the original context plus that Question.

Contribute should support both direct Knowledge Entry creation and Smart Storage. Smart Storage should not run automatically for every Contribution; some Knowledge Types and workflows, such as straightforward Comments, should be posted directly as the Knowledge Entry currently shown to the user. Direct posting should create only the Knowledge Entry by default; a Bronze Layer Source is needed when the user chooses Smart Storage, import, upload, or another workflow that preserves raw material for later refinement.

Explore and Contribute should happen in the same place. Wherever users see Answers, they should also be able to add the missing future Answer that belongs in that Knowledge Context.

The product should treat search boxes, ask boxes, and contribution editors as related Knowledge Composer surfaces. A composer may support a transient Knowledge Request, a durable Contribution, or both, but the user action should remain clear at the moment of submission.

A Contribution Submission should become durable only when the user intent needs preserved raw material, multiple Sources, deferred review, retry, Smart Storage, import, upload, or Reprocessing. A simple direct post should not create durable Contribution Submission workflow state by default; it should create the Gold Layer Knowledge Entry and any needed Entry Representations directly.

Each Contribution Submission should have one Primary Intended Entry: the Knowledge Entry the user is principally trying to create or update. Smart Storage may propose additional derived Knowledge Entries from the same submitted Sources, but those derived proposals should remain reviewable separately from the Primary Intended Entry.

When Smart Storage identifies multiple entries from one Contribution Submission, the user should accept one Smart Storage Proposal at a time. Even when the Primary Intended Entry is a future Course or MVP Series and the Sources contain many Lessons or other child entries, Gold Layer creation should remain explicit per proposed Knowledge Entry rather than bulk-accepted by default.

Sources belong first to the durable Contribution Submission that preserved the user's raw material. Smart Storage Proposals should identify which submitted Sources, excerpts, file ranges, or external URLs support each proposed create or update, and accepted Gold Layer Knowledge Entries should be linked back to the Sources that produced or informed them.

A Contribution Submission title should default into the Primary Intended Entry's identity or display title, subject to Smart Storage proposing a more canonical title for user confirmation. A description should not have one universal meaning: the composer should distinguish description as entry display content, entry summary, Authored Text Source, or Contribution Note depending on the user's intent and the Knowledge Type behavior.

A durable Contribution Submission should carry both an intended Visibility Scope for resulting Gold Layer knowledge and a Review Scope for Bronze Sources, Smart Storage Runs, Smart Storage Proposals, and pending review material. These scopes often match, but may differ when a smaller set of Users or Roles should review material before it becomes visible to the intended Gold audience.

Review Scope should default to the submitting User plus authorized reviewers for the active Organization or relevant scope when such reviewers exist. It should not automatically include every User in the intended Visibility Scope when pending raw or proposed material needs review before becoming Gold Layer knowledge. In the first implementation, Smart Storage should default Review Scope to the submitting User unless the composer is explicitly acting in an Organization or Group review context.

When the Sources for a Primary Intended Entry cause Smart Storage to propose creating or updating another Knowledge Entry, the app should not create a vague generic relationship such as `relatedTo` by default. Shared Source provenance preserves the evidence that the entries have overlapping information, while Gold Layer relationships should be expressed through Tags, existing typed relationships, Knowledge Type attributes, or later Type Behavior that can name the relationship precisely.

Sources do not automatically become Entry Representations when a Smart Storage Proposal is accepted. Acceptance should explicitly determine which submitted Sources become confirmed representations of the Gold Layer Knowledge Entry, which remain only Bronze raw material, which serve as Factual Provenance, and which were only Contribution Notes or processing guidance.

The user-facing place for this loop depends on how many Tags are active in the Knowledge Navigator. The Dashboard is used when no Tags are active and the user is located in the Global Knowledge Context. A Referent Page is used when exactly one Tag is active and the user is focused on the Referent that Tag points to. A Context Page is used when two or more Tags are active and the user is exploring their combined Knowledge Context.

Referent Pages should be reached through Tags rather than Knowledge Entry IDs. In the MVP, non-Scripture Referent Pages may use a route such as `/goto/:tagId`; Bible Passage Referent Pages should use Scripture's familiar citation language with a route such as `/scripture/:passageString`, while still behaving like a one-Tag Referent Page.

Bible Passage Tags and Referents may be created lazily. Visiting `/scripture/:passageString` should not by itself require a persisted Tag or Referent, but analytics should still record the visit against the parsed, normalized passage target so the app can report commonly visited Bible passages before those passages have been tagged or contributed around.

Analytics should distinguish Referent Page visits from Knowledge Navigator usage. A visit records that a user opened a page for a target such as `John 3:16`; Navigator usage records that the user selected a Tag as part of the Knowledge Context for Explore or Contribute.

Analytics should keep raw page visit events separately from aggregate visit stats. Raw events preserve useful history for debugging and future analysis, while aggregate stats support product queries such as commonly visited Bible passages without scanning event history.

Topic should be reserved for a named subject of discussion, such as `atonement`, `friendship`, or `Christian education`. Topic is an MVP Knowledge Type, but it should not mean the Context Page or Referent Page itself. A Topic Tag can be the active Tag for a Referent Page or one of multiple active Tags for a Context Page.

Doctrines should be represented as Topics in the MVP. Doctrine may become a more specific Knowledge Type later if the product needs confession-specific behavior, doctrinal positions, church statements, or theological taxonomies.

Themes should be represented as Topics in the MVP. Literary or theological themes can become more specific later only if they need behavior that Topic cannot express.

Subjects should be represented as Topics in the MVP. School-subject behavior such as grade-level standards, course catalogs, or credits can be added later if needed.

Genre is not a Knowledge Type. Genre can be an attribute of works such as Books, Songs, Poems, or Short Stories, or a Topic when users discuss the genre itself.

Mood and Tone are not Knowledge Types. They are attributes or analysis labels on works, quotes, comments, or related entries.

Claim should be deferred as a Knowledge Type. Claims can live inside Quotes, Essays, Comments, Questions, Sermons, or Words until argument or reasoning behavior becomes first-class.

Argument should be deferred as a Knowledge Type. Arguments can live inside Essays, Sermons, Comments, Questions, or Words until claim/evidence/reasoning structure becomes first-class.

Evidence is not a Knowledge Type. Evidence is a role a Knowledge Entry can play in relation to a Claim, Argument, or Question.

Annotation should be deferred as a Knowledge Type. Annotation overlaps with Comment and Quote until anchored marginalia or highlighting behavior becomes first-class.

Highlight is not a Knowledge Type. Highlight is selection or interaction state over a Source, Quote, Bible Passage, or other entry.

Summary should be deferred as a Knowledge Type. Summaries can live inside Words, Comments, Essays, Lessons, or Sermons until summary-specific provenance, target, length, or review behavior matters.

Transcript should be deferred until at least Phase 2 as an academic record summarizing courses taken across an academic career. Do not use Transcript as the domain term for a text representation of audio or video; that should remain a Source or representation of another Knowledge Entry such as a Sermon.

Record is not an MVP Knowledge Type. It is too broad and overlaps with Knowledge Entry, Source, future academic Transcript, and future administrative records.

Series is an MVP Knowledge Type for named collections or sequences, such as `Narnia`, `Romans Sermon Series`, or `Grade 7 Literature Unit`. Series should not be collapsed into Topic because a Series is a curated or ordered grouping, not merely a subject of discussion.

Series should cover curriculum-like groupings in the MVP. Course, Canon of Literature, and Curriculum should be added in Phase 2 when the product needs richer education-specific behavior.

Collection is not an MVP Knowledge Type. Series covers named curated or ordered groupings, while Tags and Context Pages cover everything related to a Knowledge Context.

Smart Storage

Smart Storage is the AI-assisted process of preserving a Source, identifying relevant Tags, and refining that Source toward one or more pieces of structured knowledge the application understands.

Smart Storage is an optional contribution path rather than the only way to Contribute. When direct posting and Smart Storage are both available, the user should be able to choose between posting the entry as currently displayed and storing smartly for AI-assisted proposal generation.

Smart Storage may use Factual Enrichment when a Source points to factual knowledge it does not itself contain, such as a fuzzy description of a known quotation. Factual Enrichment is encouraged for factual information, but it must produce a user-confirmable proposal rather than writing directly to the Gold Layer.

Every enriched factual field in a Smart Storage Proposal should carry Factual Provenance whenever feasible. Factual Provenance may point to an external URL, to another Knowledge Entry, or to a model-only basis when no external evidence was checked.

Factual Provenance may attach to the whole Smart Storage Proposal or Knowledge Entry when one evidence trail supports the proposal as a whole. When enriched factual fields come from different evidence or have different confidence, Factual Provenance should attach to the specific field or claim it supports.

Required Factual Provenance should be determined by the Smart Storage Contract and the Type Behavior for each Knowledge Type and field. Type Behavior may mark fields as enrichable and may require provenance for enriched values, such as a Book proposal enriching a fuzzy Source into the title `Pride and Prejudice` and author `Jane Austen`.

Smart Storage Proposals and enriched factual fields may include coarse Proposal Confidence such as high, medium, or low. Proposal Confidence should guide user review, especially for model-only provenance or ambiguous candidates, but it must not be presented as Human Weight or as a substitute for user confirmation.

Low Proposal Confidence should warn the user but should not universally block acceptance. Acceptance should be blocked by invalid Smart Storage Contract shape, unresolved required fields, unresolved identity ambiguity, or missing required Factual Provenance rather than by confidence alone.

Smart Storage should send a curated Smart Storage Contract to the LLM rather than the raw database schema. The contract should include whatever domain information the LLM needs to match Sources to Knowledge Types and propose Gold Layer structure, such as allowed Knowledge Types, type-specific fields, proposal requirements, current Knowledge Context, relevant existing Tags or Referents, examples, and provenance expectations.

Smart Storage Contracts and Type Behaviors must be versioned and tracked in the database as immutable content snapshots, not only as version labels pointing to code or configuration. Each Smart Storage Proposal should record the Smart Storage Contract version and Type Behavior version that generated it, so later rule changes can mark proposals stale or route them through Reprocessing intentionally.

Type Behavior should be versioned as a whole per Knowledge Type, with field-level rules inside the immutable snapshot. For example, a Book Type Behavior snapshot may define whether `title` and `author` are enrichable and whether enriched values require Factual Provenance, without creating separate version records for each field.

Knowledge Type-specific navigation, identity, edit, and exception behavior should live behind a TypeScript Type Behavior class or interface now, rather than remaining scattered across composer, Smart Storage, routing, and entry-editing code. The first implementation should stay small, such as a Type Behavior interface plus registry, and should avoid a large inheritance hierarchy until repeated behavior proves it is needed. The first-slice Type Behavior interface should expose `knowledgeType`, `version`, `identity`, `referentIdentityScope`, `composerDefaults`, `smartStorageChallenge`, `representationRoles`, `primaryRepresentation`, `humanWeight`, and `provenance`, using plain config objects or small functions. Shared resolver services should perform the actual lookup against Tags, Referents, aliases, and represented Knowledge Entries so database and permission logic is not duplicated per Knowledge Type. For example, Comment may not use the default represented-Referent share-link or edit flow because it is born as a response to another Knowledge Entry.

Smart Storage Contract versions should contain stable reusable rules and templates, not request-specific data. Request-specific input should be snapshotted separately with the enrichment run or Smart Storage Proposal, including the Source reference, the specific Source text or excerpts sent to the LLM, active Knowledge Context, candidate existing Tags or Referents, retrieved evidence, and other facts used for that specific proposal generation.

Request-specific input snapshots should record what the LLM actually saw while linking back to the full Bronze Layer Source as the durable raw record. The full raw Source should not be duplicated into the input snapshot unless the full Source was actually sent to the LLM.

Smart Storage Runs should preserve the raw model output separately from parsed Smart Storage Proposals. The raw output supports audit, debugging, parser failure recovery, and future contract improvement, while the Smart Storage Proposal remains the cleaned, validated, contract-shaped Silver Layer record users review.

Failed LLM calls, parse failures, and validation failures should create or update Smart Storage Runs, not Smart Storage Proposals. A Smart Storage Proposal should exist only after the app has parsed and validated a contract-shaped candidate that the user can review.

Smart Storage Run status should describe operational processing rather than user review. A queued run is waiting to call the model. A running run is actively enriching. A succeeded run produced at least one parsed Smart Storage Proposal. A no-proposal run completed without producing a reviewable proposal. A failed run stopped because the LLM call, parse, or validation failed before any proposal existed. A superseded run was replaced by a newer run for the same Source and request context.

No-proposal outcomes should be surfaced quietly in the contribution or review area, such as "Saved as Source; no structured proposal found." They should not create Answer Feed items, Knowledge Slots, or failed Smart Storage Proposal cards.

Bronze Sources and Silver Smart Storage Proposals should not appear in the normal Answer Feed, which should remain focused on accepted Gold Layer Knowledge Entries and Knowledge Slots. If Bronze or Silver material has been matched to a Referent without producing Gold Layer knowledge, the Referent Page may surface it as pending or reviewable material and let authorized Users continue the review process from there. Visibility for that pending material should follow the Contribution Submission or review scope, not the Referent Page alone.

Smart Storage Proposal acceptance should validate against the Smart Storage Contract and Type Behavior versions that generated the proposal unless those versions have been marked incompatible or retired. Incompatible or retired versions should make affected proposals stale and require Reprocessing before acceptance.

Accepting a Smart Storage Proposal requires all relevant permissions, not only access to view the proposal. The User must be authorized under the Review Scope, authorized to create or update the resulting Gold Layer Knowledge Entry under its intended Visibility Scope, authorized to edit any existing target entry, and authorized to create or reuse identity within the relevant Referent Identity Scope.

Smart Storage may challenge a user-selected Knowledge Type when the Source appears to match a more specific or more appropriate Knowledge Type. A Knowledge Slot's requested Knowledge Type remains fixed during Slot fulfillment, so Smart Storage should not challenge it.

In the composer and proposal review UI, Knowledge Type changes from Smart Storage should be presented as recommendations or blocking mismatches for the User to resolve, not silent changes to the submitted intent. Slot fulfillment keeps the Slot's requested Knowledge Type fixed.

When a user creates or refines a Knowledge Entry, the creation flow should first search for existing Tags and Referents before creating new ones. The accepted behavior is to reuse the canonical Tag when the intended Referent already exists, and to create a new Tag only when the app cannot confidently match an existing Referent or the user confirms the proposed new Referent is distinct.

The bronze, silver, and gold progression describes the degree to which useful information has been extracted, cleaned, structured, and shaped from the original Source:

- The Bronze Layer preserves submitted Sources as close as possible to their original form.
- The Silver Layer is an intermediate refinement layer for cleaned and structured data that has not yet become fully typed knowledge.
- The Gold Layer contains Knowledge Entries represented according to the most specific Knowledge Types the application currently understands.

For Smart Storage, Bronze Layer is to Source as Silver Layer is to Smart Storage Proposal and Gold Layer is to Knowledge Entry. Bronze preserves raw data, Silver holds reviewable proposed knowledge, and Gold stores confirmed Knowledge Entries.

For Smart Storage, the Bronze Layer Source should be preserved immediately when the user submits, before any LLM call or Smart Storage proposal generation. If enrichment fails, times out, or produces no acceptable proposal, the preserved Source should remain available for retry or Reprocessing.

Uploaded files should become preserved Bronze Layer Sources as soon as storage succeeds, even before extraction, parsing, transcription, preview generation, or Smart Storage analysis succeeds. Unsupported extraction or analysis failures should be represented in Smart Storage Run or processing state without deleting or invalidating the preserved Source.

When a user later opts a directly created Knowledge Entry into Smart Storage or Reprocessing, the app should create a Bronze Layer Source snapshot of the entry's current representation at that moment and link it back to the existing Knowledge Entry. That Source preserves the reprocessing input; it should not be treated as the original raw direct-post submission.

Before direct posting or Smart Storage, the application should provide a deterministic Contribution Preview that shows the best current guess for the Knowledge Type, Knowledge Context, and visible entry attributes that would be contributed. This preview should be computed by application logic rather than an LLM and should update as user input, uploaded material,

or selected context changes.

Gold Layer Knowledge Entries produced through Smart Storage require user confirmation. LLM-assisted enrichment can improve a proposal, but confirmation is the boundary where proposed structured knowledge becomes stored Knowledge Entry data.

User confirmation can make the confirming user responsible for the whole Knowledge Entry while individual factual fields remain attributed to their Factual Provenance. External factual material that should remain navigable in the knowledgebase may be represented by a Knowledge Entry; otherwise an external URL can serve as the provenance target.

Factual Provenance should default to an external URL when the source is only needed as evidence for a proposed fact. Smart Storage should create or propose a Knowledge Entry for the provenance target only when that target is itself a meaningful Referent users may navigate, tag, search, reuse, or cite repeatedly.

One Source can produce many Knowledge Entries. For example, an uploaded essay or transcript can produce one Primary Intended Entry and many Quote entries, each with its own Knowledge Context. The Primary Intended Entry's Knowledge Context may be a superset of the union of its quotes' Knowledge Contexts.

The user review unit for Smart Storage should be a durable Smart Storage Proposal linked to the saved Source, and each Smart Storage Proposal should correspond to one proposed Knowledge Entry. A single Source may therefore produce many Smart Storage Proposals, letting the user accept, reject, or edit each proposed Knowledge Entry independently.

Smart Storage Proposals belong to the Silver Layer and should survive refresh, navigation, failed enrichment, and deferred review. They are review records tied to Bronze Sources, not temporary UI previews and not Gold Layer Knowledge Entries.

Smart Storage Proposals should store contract-shaped domain proposals rather than raw Convex write payloads. On acceptance, the backend should validate the proposal and translate it into the current persistence shape, keeping Silver Layer records portable if the application later migrates away from Convex or changes its internal schema.

Smart Storage Proposal records should preserve the original generated proposal separately from the current reviewed proposal. The original generated proposal supports audit, debugging, and Reprocessing, while the current reviewed proposal is the editable version the user may accept into Gold Layer knowledge.

Guided Smart Storage follow-up questions should update the current reviewed Smart Storage Proposal before acceptance. Follow-up answers may resolve required fields, identity ambiguity, context Tags, referenced Referents, or proposal details, but they should not create partial Gold Layer Knowledge Entries while the proposal remains under review.

Guided Smart Storage follow-up questions should resolve required fields and blocking ambiguities first, one question at a time. Once the proposal is valid and accept-ready, optional enrichment questions may be offered as skippable prompts or suggestions, but they should not block acceptance.

Smart Storage Proposal status should stay small. A drafted proposal is generated and awaiting review. A needs-resolution proposal requires the user to choose between candidates or resolve ambiguity. An accepted proposal has produced a Gold Layer Knowledge Entry. A rejected proposal was declined by the user. A stale proposal was superseded by a newer Smart Storage Contract, Type Behavior, or Reprocessing run.

Accepting a Smart Storage Proposal should be atomic for one complete proposed Knowledge Entry. Users should edit the proposal, split it into multiple proposals, or reject unwanted proposals before acceptance rather than partially accepting individual fields into Gold Layer knowledge.

Proposal generation may suggest existing Tags, Referents, and Knowledge Entries, but acceptance is the authoritative identity check. At acceptance time, the application should re-check current canonical Tags and Referents, reuse existing matches, and refuse, redirect, or propose an authorized update when another same-typed Knowledge Entry already represents the proposed Referent.

When Smart Storage identifies an existing Knowledge Entry that should receive new information from the submitted Sources, the review flow should make that update explicit. If the reviewing User has permission to edit the existing entry, acceptance may add the confirmed information to that entry; otherwise the proposal should not silently modify Gold Layer knowledge.

The first implementation of Smart Storage Proposal acceptance should create new Knowledge Entries only. If acceptance finds that the proposed represented Referent already has a Knowledge Entry, it should stop and return a reviewable target-exists state rather than patching the existing entry. Updating existing entries should be a later permissioned slice with explicit conflict and audit behavior.

Smart Storage should create one primary Smart Storage Proposal for the user's intended Knowledge Entry and include referenced Tags or Referents inside that proposal. If a referenced Referent already exists, the proposal should explicitly show that the resulting Knowledge Entry will reference that existing Referent through its Tag. If the referenced Referent does not yet exist, the proposal should explicitly show that acceptance will create the new Referent and Tag and then include that Tag in the Knowledge Entry's Knowledge Context. Smart Storage should create a separate Knowledge Entry proposal for a referenced Referent only when the Source contains separate entry content for that Referent.

When Factual Enrichment finds multiple plausible matches for a user's intent, ambiguity should remain inside Smart Storage Proposal review. The user must choose the exact candidate before the proposal becomes Gold Layer knowledge, and Smart Storage should create multiple Gold Layer Knowledge Entries from a fuzzy Source only when the user explicitly accepts multiple proposals.

The first implementation slice, meaning the first independently buildable and reviewable increment of the upgraded Knowledge Composer, should establish the durable multi-Source Smart Storage spine before delivery channels or advanced extraction intelligence. That slice should create and persist a durable Contribution Submission with intended Visibility Scope, Review Scope, Primary Intended Entry metadata, and Contribution Note; persist multiple Bronze Sources per submission for Authored Text Source, uploaded file storage IDs, and external URLs; store Link Preview metadata for external URLs; queue a Smart Storage Run linked to the Contribution Submission and source IDs; create a conservative scaffold Smart Storage Proposal; show and review that proposal; and accept it into one new Gold Layer Knowledge Entry with selected Entry Representations and Source/output links. It should also include a small TypeScript Type Behavior interface or registry for MVP identity and composer defaults. Delivery channels, SMS, email, DM, full extractor pipelines, real LLM contract generation, update-existing-entry acceptance, complex Course child-entry generation, save drafts, and Referent merge/split review should come later.

Uploaded files in the first slice should use direct browser-to-Convex storage before Contribution Submission persistence. The composer should obtain an upload URL, upload file bytes to Convex storage, then submit the resulting storage ID and file metadata as a Bronze Source; Contribution Submission mutations should not receive raw file bytes.

Pre-submit uploaded files should be treated as temporary until attached to a durable Contribution Submission. The first implementation should track temporary uploads with uploader, storage ID, metadata, creation time, and expiration or cleanup status so abandoned composer sessions do not leave unmanaged storage objects.

The first implementation should include a small `temporaryUploads` table for browser-to-Convex uploads that have not yet been attached to a durable Contribution Submission. The table should include `storageId`, `uploadedByUserId`, `fileName`, `contentType`, `fileSizeBytes`, `uploadStatus`, `expiresAt`, `attachedContributionSubmissionId`, `createdAt`, and `updatedAt`. The initial upload status enum should be `uploaded`, `attached`, `expired`, and `deleted`. Keeping the temporary upload row after attach lets cleanup and audit code distinguish attached uploads from abandoned uploads.

When the User submits through the Smart Storage path, preserving the durable Contribution Submission should automatically queue the first Smart Storage Run. Direct post, upload-only, save-draft, or other non-Smart-Storage paths should not queue Smart Storage unless the User explicitly opts in later.

The first implementation slice should not include a full save-draft workflow for multi-Source Contribution Submissions. Durable preservation should begin at explicit submission; pre-submit autosave, abandoned upload cleanup, and editable draft sessions should be designed separately when needed.

In the first implementation slice, attachments and external URLs submitted through the composer should use the durable Contribution Submission and Bronze Source path rather than simple direct posting. Direct post may remain focused on text, type, and context until attachment-to-Entry-Representation and direct external-URL representation behavior is implemented.

Text-only Contributions may still use direct post when the User chooses the direct path. User-entered text becomes an Authored Text Source only when submitted through Smart Storage, import, Reprocessing, or a multi-Source Contribution Submission that uses the Bronze path.

The first-slice composer may keep separate direct post and Smart Storage actions for text-only Contributions. Once the User adds uploaded files or external URLs, the composer should route the submission through the durable Bronze path and hide, disable, or replace direct post with a clear Store Smartly or review-oriented action until direct attachment and direct external-URL representation behavior exists.

Before advanced extraction exists, first-slice Smart Storage Proposals should be conservative scaffolds rather than pretending to understand files or media that have not been extracted. A scaffold proposal may preserve the selected Knowledge Type, title, selected Tags, intended Visibility Scope, Review Scope, Contribution Note, and source inventory with extraction or preview status, while deferring derived proposals until the app has actual extracted content or user-confirmed structure.

Scaffold Smart Storage Proposals may be accepted into Gold Layer knowledge when Type Behavior says the proposed minimum entry is valid and the User explicitly confirms which Sources become Entry Representations. Acceptance should create the Gold Knowledge Entry, confirmed Entry Representations, and Source/output links together. When required fields, identity ambiguity, provenance, permissions, or representation decisions are unresolved, the proposal should remain needs-resolution rather than becoming Gold.

One Source may eventually produce multiple Entry Representations, such as an uploaded sermon manuscript producing both a stored file representation and extracted plain text, or a video URL later producing an external URL representation and transcript text. In the first implementation slice, confirmed Sources should usually map one-to-one into Entry Representations unless a User action or extractor explicitly splits the Source into multiple representations.

Type Behavior should provide the default Primary Representation rule for each Knowledge Type, and the User should be able to override that default during proposal review when more than one Entry Representation is valid. Primary Representation determines the default display, open, preview, or playback behavior; it does not make other representations less valid or less preserved.

Entry Representations should carry Representation Roles in addition to representation kinds when the role is known or inferred. Users should not be required to manually label every representation up front; Type Behavior, deterministic metadata, and Smart Storage may infer roles from source kind, file name, content type, link metadata, extracted content, selected Knowledge Type, and Contribution Notes. Proposal review should show inferred roles and allow the User to correct them before acceptance.

Representation Role does not replace explicit Primary Representation selection. Type Behavior may use roles to choose a default Primary Representation, but the accepted primary selection should remain explicit entry data so role and default display behavior can evolve independently.

When Representation Role inference is low-confidence, the app should fall back to a generic role such as supporting material or unspecified and should not block acceptance unless the Knowledge Type's Type Behavior requires a specific role. If a required role is missing or ambiguous, proposal review should ask the User to identify which Source or representation fulfills that role.

Gold Layer Entry Representations should store the accepted Representation Role as normal entry data. Inference confidence, rationale, and model or metadata basis should remain on the Smart Storage Proposal, run history, or audit trail rather than becoming hot Gold Layer display data after the role is accepted.

Representation Role values should start as a small global enum, such as primary content, supporting material, manuscript, slides, transcript, recording, thumbnail, external reference, and unspecified. Type Behavior may later introduce controlled type-specific role extensions, but Gold Layer roles should not be arbitrary free-text labels.

Accepting a scaffold proposal should not consume or retire the underlying Sources. Accepted entries should remain linked to the Sources and proposals that produced them so later extraction, enrichment, Type Behavior improvements, or Reprocessing can propose updates to the existing entry or additional derived entries.

Reprocessing revisits existing Sources or Knowledge Entries when the application gains new Knowledge Types or improved recognition. A previously complete entry can become an Upgrade Candidate when a new type reveals knowledge it held only indirectly.

Reprocessing may propose edits to an existing Gold Layer Knowledge Entry when the same knowledge can be represented more specifically, such as changing its Knowledge Type, fields, represented Referent, or Tags. It may also propose additional linked Knowledge Entries when the Source or existing entry contains separate knowledge units. The proposal review must make the outcome explicit: acceptance either updates an existing Gold entry, creates new Gold entries, or does both through clearly separated proposals.

When a Smart Storage Contract or Type Behavior version changes, the app may run Reprocessing across a selected dataset or the entire eligible dataset to produce suggested edits and new proposals. Dataset-wide Reprocessing should create reviewable Smart Storage Proposals or mark Upgrade Candidates; it should not silently rewrite Gold Layer Knowledge Entries.

Dataset-wide Reprocessing may include direct-post Knowledge Entries by default. For direct-post entries that do not already have Bronze Sources, the app should first create a Bronze Layer Source snapshot of each eligible entry's current representation before running Smart Storage. Existing Smart Storage, import, or upload entries can reuse their preserved Bronze Sources when those Sources remain the correct input for the new run.

Dataset-wide Reprocessing suggestions should be routed to users with authority over the affected Visibility Scope, Organization, or entry. Personal suggestions should go to the owning user; organization-context suggestions should go to users with the relevant organization role; public or global suggestions should use a trusted maintainer workflow. Bulk Reprocessing may generate suggestions at scale, but accepting suggested edits remains permissioned per affected entry or proposal.

Reprocessing notifications should present new type or contract improvements as opportunities to enrich the knowledgebase, not as errors in existing data. For example, when a new Canon of Literature type is added later, a school user might see: "Exciting news! We just added the Canon of Literature type. You have books in your school's context that look like they might make good additions to your canon. Click to see suggestions." The exact copy may vary, but the posture should be invitational and review-oriented.

Clicking a Reprocessing notification should open a scoped suggestions queue rather than jumping directly to the first suggestion. The queue should be filtered to the relevant Knowledge Context, Organization, new Knowledge Type, or contract improvement, show the size and shape of the opportunity, and let authorized users accept, reject, or open individual suggestions for review.

The MVP suggestions queue should not support bulk accept for Reprocessing suggestions. Accepting suggested edits or new Gold entries should remain one suggestion at a time. Bulk dismiss, bulk reject, or "not now" actions may be allowed because they avoid writing Gold Layer knowledge; bulk accept can be reconsidered later for low-risk suggestions with strong provenance, audit, and rollback.

Rejected or dismissed Reprocessing suggestions should be remembered so the app does not repeatedly present the same candidate to the same review scope. The dismissal should be tied to the proposal or candidate key, the Smart Storage Contract or Type Behavior version that produced it, and the relevant reviewer scope. A materially changed suggestion from a later contract or behavior version may be surfaced again.

Accepted Reprocessing edits should preserve explicit upgrade provenance, such as the prior Knowledge Type or shape, the accepted proposal, and the Smart Storage Run or contract versions that produced the suggestion. User-facing UI should present this subtly, such as in history or metadata rather than as a persistent warning. Upgrade provenance and old versions should not be stored in a way that bloats hot Knowledge Entry records or slows normal reads; after an appropriate retention window, older versions may be archived into colder history or audit storage while preserving enough summary provenance to explain the upgrade.

Archived old versions should remain governed by the affected entry's Visibility Scope and relevant role checks. A user who can view the current entry may see subtle summary provenance such as "upgraded from Words," but full old-version contents should be visible only to users who could view the relevant entry/version under its scope or to current authorized

maintainers for that scope.

Dataset-wide Reprocessing is descriptive product language, not a new domain glossary term in the MVP. If the implementation later needs a persisted batch object for scheduling, progress, retries, or audit, it may introduce a product or implementation concept such as a Reprocessing Pass without changing the core domain term Reprocessing.

Source is not an MVP Knowledge Type. A Source belongs to the Bronze Layer as raw submitted material and can produce Knowledge Entries, but it should not itself be treated as represented knowledge in the Gold Layer.

Media formats such as audio, video, image, and file are not MVP Knowledge Types. They belong to Sources, attachments, or representations of Knowledge Entries. For example, a Sermon may be represented by audio, video, transcript, or notes, but its Knowledge Type remains Sermon.

Long-form or rich editable content belongs to Entry Representations rather than to type-specific detail rows. Words has no separate type detail table; its full content is represented through an Entry Representation while Knowledge Entries retain the bounded text needed for cards and search.

Knowledge Slots

A Knowledge Slot is a predefined request for one Knowledge Entry of a specified Knowledge Type within a specified Knowledge Context. It is the app's way to request future Answers from users.

A saved Question should not automatically create a Knowledge Slot. The Question records a durable question and can define a narrower Knowledge Context; a Knowledge Slot is created only when a user intentionally requests a future Answer from a user, group, organization, network, or open audience.

Examples:

- A teacher assigns an essay on `Pride and Prejudice, book`; each student receives a Knowledge Slot for an Essay entry in that Knowledge Context.
- A user creates an Event entry and invites people to fulfill RSVP slots.
- A Knowledge Request maps to a Knowledge Context with no existing Answers; the user creates a Knowledge Slot directed to an expert, a group, an organization network, or an open audience until an Answer is contributed.

Fulfillment is the state of a Knowledge Slot after the requested Knowledge Entry exists.

The MVP should classify calls to action generically as Knowledge Slots rather than adding an Assignment Knowledge Type. For example, a teacher assigning an Essay, a user requesting an expert Answer, or an Event asking for RSVP entries are all Knowledge Slots requesting future Knowledge Entries within specified Knowledge Contexts.

Task and Todo are not MVP Knowledge Types. Calls to action that request future Knowledge Entries should be represented as Knowledge Slots; tasks that do not request knowledge are outside the MVP.

Question is an MVP Knowledge Type because questions provide valuable information about which parts of a Knowledge Context need to be connected. A user may ask a transient Knowledge Request, but a Question can also be represented as a Knowledge Entry within the Knowledge Context it maps to, helping reveal the shape of the Question Space.

Question Template should be deferred as a Knowledge Type. Reusable request or slot templates introduce authoring and reuse behavior beyond the MVP.

Template is not an MVP Knowledge Type. Templates are reusable authoring structures for creating other entries, questions, slots, lessons, or related workflows.

Visibility

Visibility Scope belongs to Knowledge Entries. A Knowledge Entry may be visible to one user, an organization, a group, a network of organizations, or everyone.

Contribution defaults should favor organization visibility rather than public visibility. When a Contribution happens from an Organization Page or another unambiguous organization-scoped context, the default Visibility Scope should be that organization. When the User is on the Dashboard or outside any organization-scoped context, the default should be all Organizations the User belongs to if no Active Role is selected, or the Organization corresponding to the selected Active Role when one is selected. The composer should always allow the User to explicitly choose the Visibility Scope for the entry.

Visibility defaults and role authority are separate. The current Organization Page may supply an organization Visibility Scope even when the User has not chosen among multiple Roles in that Organization, but any role-specific edit, send, review, or administrative action should require an explicit Active Role.

Visibility Scope and Delivery Target are separate. Visibility Scope controls who may access a Knowledge Entry after it exists; Delivery Target controls who should be notified, assigned, messaged, or otherwise sent a contribution or action. Sending an entry to a group does not by itself define every user who may access the entry, and making an entry visible to an organization does not require notifying every member of that organization.

The composer may capture intended Delivery Targets at submission time, but normal delivery should occur after Gold Layer acceptance. Sending pending raw or Silver material before acceptance should be an explicit review workflow action, not the default Delivery Target behavior. After Gold acceptance, authorized Users should also be able to send or re-send the Knowledge Entry to Users, Groups, Organizations, or other Delivery Targets.

Delivery must validate access against the Knowledge Entry's Visibility Scope. Sending to a Delivery Target should not silently expand visibility; if recipients cannot access the entry, the app should require an explicit authorized Visibility Scope change or block delivery to those recipients.

Review Scope and Visibility Scope are separate. Review Scope controls who may see or manage pending Bronze and Silver material before Gold Layer knowledge exists, while Visibility Scope controls who may access the accepted Knowledge Entry afterward.

When a workflow has explicit reviewer Roles, Review Scope should default narrower than intended Visibility Scope. When no reviewer Role exists, Review Scope should still include the submitting User and any authorized organization or scope maintainers rather than broad-delivering pending material to the eventual audience by default.

"Send to page" should not be used as product or domain language. If a User intends an entry to appear in relation to a Knowledge Page, the entry should reference the relevant Tag through its Knowledge Context. Delivery is reserved for notifying, assigning, or messaging actual recipient targets.

Tags and Referents become visible indirectly through visible Knowledge Entries that represent or reference them. The Global Knowledge Context is not the same thing as global visibility: an entry can be visible to everyone without belonging to the Global Knowledge Context.

Tags do not grant access. A Knowledge Entry may reference an Organization, Group, Person, Place, Topic, Bible Passage, or other Referent through its Knowledge Context without becoming visible to users associated with that Referent. Visibility Scope remains the access boundary.

Canonical Referent matching may reveal that a same-typed Referent already exists, but it must not leak hidden Knowledge Entry details beyond safe identity and discoverability fields. A User may be told that a matching Referent exists and may be allowed to reference its canonical Tag, while the represented Knowledge Entry's protected content, representations, provenance, and edit actions remain governed by Visibility Scope and permissions.

Composer tag entry may feel freeform, but stored Tags must resolve to canonical Referents. Before submission or acceptance, a typed tag should either match an existing Tag/Referent or be confirmed as a proposed new Referent with a Knowledge Type; unresolved local labels should not be stored as Tags.

Composer tag suggestions should distinguish current-context Tags, deterministic recommendations, and Smart Storage recommendations. Current-context Tags come from the active Knowledge Navigator or current Knowledge Page; deterministic recommendations come from user or organization Recognized Context, recent use, pinned pages, memberships, selected Knowledge Type, and visible submission metadata; Smart Storage recommendations come from

AI-assisted analysis of Sources or enrichment and should remain reviewable before they affect Gold Layer knowledge.

Current-context Tags should be selected by default for ordinary Contributions, but the User may remove them before submission when the entry does not actually belong in that Knowledge Context. Knowledge Slot fulfillment is different: Slot Tags are the frozen Knowledge Context for the requested entry and should remain locked unless a future workflow explicitly allows changing the Slot's context.

Entry-adjacent actions should remain workflow, user, or delivery state rather than core Knowledge Entry fields. Copy link should copy a link to the Knowledge Page for the Knowledge Entry's Represented Referent by default, not to an implementation-specific entry record URL; Knowledge Type behavior may define exceptions such as Comment. Mark as read should mark a Notification as no longer new. Done should mark a Knowledge Slot as fulfilled. Reply should create a Comment Knowledge Entry or fulfill a response Slot. Send as email, SMS, or DM should create delivery records against Delivery Targets and channel-specific delivery behavior.

MVP Direction

The MVP Knowledge Type set is locked as: Words, Bible Passage, Topic, Series, Question, Quote, Sermon, Essay, Poem, Song, Book, Short Story, Lesson, Comment, Prayer Request, Event, RSVP, Person, Organization, Group, and Place. New Knowledge Types should be deferred unless they prove required for one of the MVP loops.

Bible Passage is an MVP Knowledge Type for Referents and Tags, but it is not an authorable Knowledge Entry type in the MVP. Scripture text belongs to the Bible structure and Bible verse text tables, while user-created entries such as notes, sermons, lessons, comments, or questions reference Bible Passage Tags in their Knowledge Context.

Sermon Clip should be deferred unless a later workflow needs quote-like Type Behavior specifically for sermon media.

Essay, Poem, Song, Book, and Short Story are MVP Knowledge Types because churches and schools need to refer to named works precisely. Their initial Type Behavior can be mostly the same as Words: they are named wrappers that let the application distinguish similarly named Referents and present them with the right human meaning before richer type-specific behavior exists.

Hymn is not an MVP Knowledge Type. Hymns should be represented as Songs unless hymn-specific behavior becomes necessary later.

Liturgy should be deferred as a Knowledge Type. Liturgical content can begin as Words, Song, Bible Passage, Event context, or Series depending on its shape until worship-service behavior becomes first-class.

Sacrament and Ordinance should be deferred as Knowledge Types. In the MVP, baptism or communion services can be Events, while theology of sacraments or ordinances can be Topics.

Offering and Donation should be deferred as Knowledge Types. They imply payments, finance, receipts, stewardship records, and sensitive permissions beyond the MVP.

Reading Plan should be deferred as a separate Knowledge Type. In the MVP, a reading plan can be represented as a Series of Bible Passages, Books, Lessons, and Knowledge Slots until scheduling or progress behavior becomes distinct.

Progress is not a Knowledge Type. Progress is state on a User's relationship to a Knowledge Slot, Lesson, Series, future Reading Plan, or future Assessment.

Plan is not an MVP Knowledge Type. Lesson, Series, Event, and future Reading Plan cover the concrete planning concepts; a generic Plan type would be too broad.

Article should be deferred from the MVP. It may become another named wrapper over Words later, but Essay, Book, Quote, and Words are enough until a day-one workflow needs Article identity.

Artifact should be deferred from the MVP. It is too broad for day one and risks becoming another catch-all unless a concrete workflow needs object or resource behavior that the required Knowledge Types cannot express.

Comment is an MVP Knowledge Type because the core loop needs relational response behavior. A Comment may contain Words-like content, but it is born as a response to another Knowledge Entry and can support threaded discussion,

correction, or contribution underneath an existing Answer.

Prayer Request is an MVP Knowledge Type because prayer is a first-class church and family workflow. The MVP should support Prayer Requests with appropriate visibility and response behavior while reserving advanced pastoral-care workflows for later.

Testimony should be deferred as a Knowledge Type. Testimonies can begin as Words or Essay-like entries until testimony-specific visibility, attribution, liturgical, or pastoral behavior is needed.

Devotional should be deferred as a Knowledge Type. Devotional content can begin as Words, Essay, Lesson, or Sermon depending on its shape until devotional-specific behavior is needed.

Confession and Catechism should be deferred as Knowledge Types. In the MVP, they can be represented as Book or Series entries, with doctrines represented as Topics, until structured doctrinal-standard behavior is needed.

Assessment, Quiz, and Test should be Phase 2 Knowledge Types. They require education-specific behavior such as grading, attempts, scoring, due dates, and feedback that is beyond the MVP loop.

Grade and Score are not Knowledge Types. They are attributes or results attached to assessment behavior.

Standard and Learning Objective should be Phase 2 Knowledge Types. In the MVP, Topic, Lesson, Series, and Knowledge Slots should cover the basic school workflow until richer curriculum and assessment behavior is added.

Rubric should be a Phase 2 Knowledge Type. Rubrics belong with assessment and grading behavior; MVP feedback can use Comments and Knowledge Slots.

Term, Semester, and School Year are not Knowledge Types. They are time or calendar grouping attributes for Events, Lessons, Groups, Series, or future Courses.

Grade Level is not a Knowledge Type. It is an attribute of a Group, Lesson, Series, future Course, or student context.

Event and RSVP are MVP Knowledge Types because some Knowledge Entries are connected to scheduled real-world activity, such as lessons, classes, services, or gatherings. The MVP should support basic scheduling and invitation responses while reserving advanced scheduling behavior for later.

Service and Worship Service are not MVP Knowledge Types. They should be represented as Events until worship-specific behavior such as liturgy, sermon linkage, music setlists, attendance, sacraments, or recurring-service behavior becomes necessary.

Meeting is not an MVP Knowledge Type. Meetings should be represented as Events until meeting-specific behavior such as agendas, minutes, decisions, action items, or quorum becomes necessary.

Decision should be deferred as a Knowledge Type. Decisions can begin as Words or Comments attached to an Event or meeting context until governance behavior becomes first-class.

Policy should be deferred as a Knowledge Type. Policies can begin as Words or Book-like entries until authority, effective dates, approval, versioning, or compliance behavior becomes first-class.

Procedure and Checklist should be deferred as Knowledge Types. They can begin as Words or Lesson-like entries until step, order, or completion behavior becomes first-class.

Form should be deferred as a Knowledge Type. Forms imply fields, submissions, validation, permissions, and workflow behavior beyond the MVP.

Report should be deferred as a Knowledge Type. Reports can begin as Words or Essay-like entries, or as views over other entries, until reporting behavior becomes first-class.

Reflection and Journal Entry should be deferred as Knowledge Types. Personal reflections can begin as Words or Essay-like entries with appropriate visibility until journaling behavior becomes first-class.

Biography should be deferred as a Knowledge Type. Biographical content can be represented as a Book, Essay, or Words entry tagged to a Person until life-history behavior becomes distinct.

Invitation is not an MVP Knowledge Type. An Event can create RSVP Knowledge Slots directed to People, Groups, or Organizations; the RSVP is the contributed response, while the invitation itself is workflow or message state.

Attendance is not an MVP Knowledge Type. Actual attendance can be represented later as participation state tied to a Person or User and an Event, while RSVP remains the MVP Knowledge Type for invitation responses.

Calendar is not a Knowledge Type. Calendar is a view composed of Event-based Knowledge Entries and related scheduled entries such as Lessons or RSVP slots.

Notification is not a Knowledge Type. Notifications are reactions to existing Knowledge Entries or requests for users to create future Knowledge Entries through Knowledge Slots.

Reaction is not a Knowledge Type. Reactions are interaction state attached to Knowledge Entries or Comments.

Bookmark is not a Knowledge Type. Bookmark is a User relationship to a Knowledge Page that saves it for later reference without sidebar placement or notification behavior. Bookmarks may inform user-specific tagging recommendations and Knowledge Context recommendations because they record Knowledge Pages the user has intentionally brought into relationship with their activity.

Pinning is not a Knowledge Type. Pinning is a User relationship to a Knowledge Page that keeps it easy to return to through navigation, while Subscription separately controls notification behavior.

Subscription is not a Knowledge Type. A subscription is a User relationship to a Referent Page, Context Page, Tag, Knowledge Entry, Group, Event, or Organization that affects notification behavior.

Dashboard, Referent Page, and Context Page are not Knowledge Types. They are user-facing places or views determined by the number of active Tags in the Knowledge Navigator.

Knowledge Context is not a Knowledge Type. It is the set of Tags that locates Knowledge Entries and Knowledge Requests.

Visibility Scope is not a Knowledge Type. It is access or audience metadata on a Knowledge Entry.

Lesson is an MVP Knowledge Type because schools and church classes need planned teaching material that can be connected to Events. The user experience should feel like working with a lesson plan, but the canonical Knowledge Type is Lesson. A reusable Lesson may be connected to many scheduled uses over time, such as teaching the same lesson in different years.

Person is an MVP Knowledge Type because churches and schools need to reference authors, teachers, students, speakers, invitees, commenters, and other participants. Every User must be linked to exactly one Person Knowledge Entry so the User can be tagged through that Person, but Person and User are not the same thing. For example, C.S. Lewis can be a Person who authored a Book without ever being a User, while a student is both a User and the Person who authored an Essay.

Account is not a Knowledge Type. Account is authentication and access infrastructure for a User.

User is not a Knowledge Type. User is the account or access identity that must link to a Person Knowledge Entry; Person is the Knowledge Type.

Profile is not a Knowledge Type. A profile is a view or presentation of a Person, User, Organization, Group, or related referent.

Character is not an MVP Knowledge Type. Fictional characters should be represented as Person Referents in the MVP, with a later split only if fictional-person behavior becomes necessary.

Bible Character and Biblical Figure are not MVP Knowledge Types. Biblical people should be represented as Person Referents.

Role is not an MVP Knowledge Type. A Role is the relation of a Person to a Knowledge Type or Knowledge Entry, such as author of a Book, teacher of a Lesson, student in a Group, speaker of a Sermon, parent in a Family, or invitee to an Event.

Author is not an MVP Knowledge Type. Author is a Role of a Person in relation to a Book, Essay, Poem, Song, Short Story, Quote, or other Knowledge Entry.

Speaker, Preacher, Teacher, and Student are not MVP Knowledge Types. They are Roles of a Person in relation to a Sermon, Lesson, Event, Group, Organization, Knowledge Slot, or other Knowledge Entry.

The MVP should use direct type-detail fields for known single-person relationships, such as a quoted person on a Quote or a respondent on an RSVP. A separate cross-type Person-role table should be deferred until the first UI or query needs role-based search across Knowledge Types.

Denomination should be deferred as a Knowledge Type. It may begin as an Organization attribute or Topic and can become a Knowledge Type later if denominational affiliation needs first-class discovery, visibility, or trust behavior.

Ministry is not an MVP Knowledge Type. A ministry may be represented as a Group, Organization-related body, Topic, Series, or Event context depending on how it is used, until distinct ministry behavior is needed.

Organization is an MVP Knowledge Type, but the MVP should understand only four Organization kinds: School, Church, Family, and Community. To use the app, a User must be a member of at least one Organization, and initial signup must associate the User's Person with a School or Church. Users can also be grouped into Families and can specify a hometown to become a de facto member of a Community. Deeper organization networks, permissions, and membership workflows should be reserved for later.

Network should be a Phase 2 Knowledge Type or organization capability. In the MVP, Organization plus Visibility Scope is enough; named networks of organizations can be added when cross-organization behavior becomes first-class.

Group is an MVP Knowledge Type for a collection of People, not a collection of Users. Since every User links to a Person, user-based participation can still be represented through Person membership, while Groups can also include people who are not application Users. Group should cover informal or temporary collections such as classes, teams, committees, or volunteer cohorts without forcing them to become Organizations.

Groups can receive Knowledge Slots, but fulfillment is performed by Users. When a Knowledge Slot is directed to a Group, the expected fulfillers are Users linked to People in that Group. People who are not linked to Users can still belong to Groups as historical or referential members, but they cannot perform user actions until linked to an account.

Membership is not an MVP Knowledge Type. It is the relationship between a Person and a Group or Organization, with user actions performed through a linked User when one exists.

Place is an MVP Knowledge Type because Community depends on hometown or place-based association, and Events often need locations. The MVP should keep Place narrow: enough to represent hometowns, event locations, and organization locations, without becoming a full geography model.

Map is not a Knowledge Type. A map is a view or representation over Places, Organizations, Events, and Communities.

Address is not a Knowledge Type. Address is an attribute or locator for a Place, Organization, or Event.

Time and Date are not Knowledge Types. They are scheduling attributes of Events and other scheduled entries.

The MVP should present Scripture references through one Knowledge Type: Bible Passage. Bible Passage can represent one verse, many verses, a chapter, a larger passage, or a set of passages across multiple books of the Bible. The application must understand subset relationships between Bible Passage Referents, such as `Matthew 24:1` being part of both `Matthew 24:1-25:46` and `Matthew 24:1-25:46; Mark 13:1-37; Luke 21:5-36`.

Bible Passage Referents and Tags are not created by users as Knowledge Entries. A user-created entry can reference a Bible Passage through its Tags, but the entry's Represented Referent should be a same-typed Referent such as Words, Sermon, Lesson, Question, Comment, or another authorable Knowledge Type.

Bible Passage identity should be based on normalized canonical verse ranges, not raw citation strings. User-entered or URL passage strings such as `Romans 8:28` or `Matthew 24:1-25:46; Mark 13:1-37; Luke 21:5-36` should be parsed into canonical locations that can support display, lookup, and subset checks. The MVP should assume a single 66-book Protestant versification for these canonical locations, with alternate canons or versification systems deferred until

they become required.

Bible Passage range identity should sort ranges into canonical Bible order and merge overlapping or adjacent ranges. Different user-entered orderings or equivalent split ranges should resolve to the same Referent, while the original input may still be retained for display history or analytics.

The persisted canonical key for a Bible Passage should be based on normalized verse ordinal ranges, while human-readable labels should be generated from canonical structure. For example, an internal key may represent `23145-23145`, while the display label is `Romans 8:28`.

In the MVP, Bible Passage Referents should store their normalized range array inline, with a reasonable cap on passage-set size to avoid unbounded arrays. A separate range table should be deferred until the app needs very large passage sets or independent querying of range components.

Bible Passage subset and containment relationships should be computed dynamically from normalized verse ordinal ranges in the MVP. Persisted relationship rows should be deferred until the app needs curated relationship labels, manual cross-reference relationships, or proven performance improvements.

One Bible Passage Referent may contain multiple verse ranges across multiple books when the intended referent is the combined passage set. For example, `Matthew 24:1-25:46; Mark 13:1-37; Luke 21:5-36` can be one Referent for the Olivet Discourse across books. Adding another Scripture Tag through the Knowledge Navigator creates a multi-Tag Context Page for cross-reference or comparison, even if the first Tag already contains several Bible books internally.

The first Scripture seed should include Bible structure before translation text: books, chapters, verses, and canonical verse positions. Full translation text should be seeded only from a clearly licensed or public-domain source present in the repository. If a vetted KJV source is present, the first pass may seed KJV verse text; otherwise KJV should be represented as known translation metadata until a source is added.

The app should seed known Bible Translation metadata separately from verse text availability. The translation registry may include metadata-only entries for translations and source texts the app knows about, including English translations, Greek source texts, Hebrew source texts, and Latin texts, even when the application does not yet store their full verse text.

Bible Translation records should have internal database IDs for relationships and stable unique short codes for import, lookup, display, and migration. Examples include `KJV` for King James Version, `TR1894` for Scrivener's Textus Receptus, and `VULGATE` for Biblia Sacra Vulgata.

Bible verse structure and Bible verse text should be stored separately. Canonical verse records should identify the book, chapter, verse number, and verse position used for passage lookup. Translation-specific verse text records should point to a Bible Translation and a canonical verse, allowing structure to be seeded before any full translation text is available.

Verse is not an MVP Knowledge Type. A single verse is represented as a Bible Passage.

Bible Book is not an MVP Knowledge Type. A whole book of the Bible can be represented as a Bible Passage in the MVP.

Bible Story is not an MVP Knowledge Type. A Bible story can be represented by a Bible Passage, often with Topic, Series, or Lesson Tags.

Memory Verse is not an MVP Knowledge Type. The referent is a Bible Passage; memorization is a learning activity or call to action represented through a Knowledge Slot, Lesson, or later assessment behavior.

Scripture cross references are not a separate MVP Knowledge Type. They can be captured by tagging multiple Bible Passages in the same Knowledge Entry and by recognizing relationships between those Bible Passage Tags.

Translation is not an MVP Knowledge Type. A Bible Passage may have one or more translations or versions as attributes or representations, but the Knowledge Type remains Bible Passage.

Language is not an MVP Knowledge Type. It should be represented as an attribute of Sources, representations, Knowledge Entries, users, or other relevant records.

Tag is not a Knowledge Type. A Tag is the named, typed pointer to a Referent; it is part of the organizing mechanism rather than the thing being represented.

Knowledge Entry is not a Knowledge Type. A Knowledge Entry is the typed, contextualized unit that represents a Referent; the Knowledge Type describes what kind of Referent the entry represents.

Answer is not a Knowledge Type. Answer is a role a Knowledge Entry can play relative to a Knowledge Request or Question.

Prompt should not be used as a domain term or Knowledge Type. Use Knowledge Request for the user action, Question for a stored question, and Knowledge Slot for a call to contribute future knowledge.

Note is not an MVP Knowledge Type. Notes should enter as Words unless they are or become a more specific Knowledge Type such as Comment, Question, Essay, Quote, or Lesson.

Resource is not an MVP Knowledge Type. It is too broad and overlaps with Source, Artifact, Words, Book, Lesson, media formats, and attachments.

Link and URL are not MVP Knowledge Types. They are attributes or external representations of Knowledge Entries, since many Knowledge Entry contents may exist at external URLs.

Link Preview is not a Knowledge Type, Knowledge Entry, Factual Provenance, or Human Weight Evidence. It is fetched metadata that helps a user recognize an external URL in the composer, Source review, or Entry Representation UI. When Smart Storage uses information from a linked page or preview to propose facts, the external URL remains the provenance anchor.

External URL Sources should always preserve the submitted URL. Fetched Link Preview metadata may be stored to help the User recognize the URL, but fetched page text, transcripts, or excerpts should be snapshotted only when the app retrieves or uses them for Smart Storage, enrichment, or review. Full remote media copies should not be the default; for large media such as YouTube, preserve the URL, provider metadata, and any transcript or extracted text actually retrieved.

Link Preview fetching should be backend-owned, such as through a Convex action, rather than performed in the browser. The client should submit the URL, and backend processing should fetch, normalize, timestamp, and store preview metadata and fetch status. Link Preview failure should not block preserving the external URL Source.

In the first implementation, Link Preview metadata for an external URL Source should live on the `sources` row as latest snapshot fields: `linkPreviewStatus`, `linkPreviewTitle`, `linkPreviewDescription`, `linkPreviewImageUrl`, `linkPreviewSiteName`, `linkPreviewFetchedAt`, and `linkPreviewError`. A separate Link Preview history table should wait until multiple fetch attempts or preview history become product requirements.

Work is not an MVP Knowledge Type. The MVP should use concrete Knowledge Types such as Book, Song, Poem, Essay, Short Story, Sermon, Lesson, and Quote rather than introducing an abstract Work type.

Citation and Reference are not MVP Knowledge Types. Quote is the Knowledge Type for cited excerpts; citation and reference details should be metadata or relationships to the Source, parent entry, Book, Bible Passage, or other relevant Referent.

Page, Chapter, and Section should be deferred as Knowledge Types. They are structural locations within a work and can be represented through references, ranges, Quotes, or relationships until structural navigation becomes first-class.

Announcement should be deferred as a Knowledge Type. Informational announcements can begin as Words, scheduled announcements as Events, and responsive announcements as Comments until broadcast behavior becomes first-class.

The MVP should prove the core loop:

- A user can Explore through a Knowledge Request mapped to a Knowledge Context.
- A user can Contribute from that same place.
- Smart Storage can preserve Sources and produce one or more Knowledge Entries.
- Knowledge Slots can request missing future Answers.

Schema Invariants

These invariants are implied by the MVP domain model and `convex/schema.ts`. Convex validators and indexes document the shape, but most cross-document rules must be enforced in mutations, migrations, seed scripts, and tests.

Referents and Tags

- A Referent is uniquely identified by `knowledgeType` and `canonicalKey`.
- A Referent's `knowledgeType` must match the type of the thing it identifies.
- A Tag is canonical to exactly one Referent through `referentId`.
- A Referent should have at most one canonical Tag.
- A Tag's `knowledgeType` must match its Referent's `knowledgeType`.
- Tag `lookupKey` values should be normalized consistently before lookup or insert.
- Tag aliases must point to canonical Tags, not create local duplicate Tags.
- A tag alias's `knowledgeType` must match the referenced Tag's `knowledgeType`.
- User and organization recognition of a Tag belongs in `tagRecognitions`, not in duplicate Tags.
- A `tagRecognitions` row must identify exactly one recognizer: either a User or an Organization.

Knowledge Entries

- A Knowledge Entry represents exactly one Referent through `representedReferentId`.
- A Referent may have at most one Knowledge Entry representing it.
- A Knowledge Entry's `knowledgeType` must match its represented Referent's `knowledgeType`.
- Knowledge Entry creation should create or reuse the Represented Referent and canonical Tag, but should not create a duplicate Knowledge Entry when that same-typed Referent is already represented.
- When a same-typed represented entry already exists, the app should redirect the user toward the existing Referent's Knowledge Page, tagging that Referent, editing it, or proposing an authorized update.
- Most Knowledge Page edits should affect the represented Knowledge Entry or related entry data; Referent identity edits should be special permissioned operations for identity correction, aliasing, merge/split, or Type Reclassification.
- Duplicate Referent checks must use Knowledge Type-specific identity fields defined by Type Behavior, not a universal title-only match.
- Duplicate Referent checks must respect Referent Identity Scope, so public works can be matched globally while organization-, group-, or user-scoped works remain local unless intentionally published or shared more broadly.
- Smart Storage may flag possible duplicate Referents, but Referent merge or split must be handled by a separate permissioned review workflow.
- `knowledgeEntries.knowledgeType` must never be `biblePassage` in the MVP.
- `primaryTagId` must point to the canonical Tag for `representedReferentId`.
- Each Knowledge Entry must have exactly one `entryTags` row with `tagPurpose: "represented"`.
- The represented `entryTags` row must use the same Tag as `primaryTagId`.
- All other Knowledge Context Tags for an entry should use `tagPurpose: "context"`.
- `contextPreviewTagLabels` is denormalized card data and must stay bounded.
- `searchText`, `previewText`, and `publicPreviewText` must stay bounded enough for card/search use.

- Long or rich content must live in `entryRepresentations`, not directly on `knowledgeEntries`.
- `humanWeight`, when present, should stay within the product scale of 0 through 100.
- `createdAt` should be set once; `updatedAt` should move forward when entry-visible data changes.
- Discoverability and visibility are separate: a public preview may exist without read access to the full entry.
- When `publicPreviewText` is exposed through discovery, it must be safe to show to that discoverability audience.

Type Detail Tables

- Words has no one-to-one detail table; the common `knowledgeEntries` row is the Words-level shape.
- Bible Passage has no authorable detail table in the MVP.
- Every non-Words authorable Knowledge Type should have at most one matching type-detail row.
- A type-detail row's `entryId` must point to a Knowledge Entry of the matching `knowledgeType`.
- Empty-ish detail tables with only `entryId` are acceptable until a type has real MVP-specific fields.
- `commentEntries.parentEntryId` must point to the entry being answered or discussed.
- A Comment should not use itself as its parent.
- A Comment's represented Referent identity should be generated from stable relational facts, such as parent Knowledge Entry, commenting Person Referent, and generated uniqueness, not from user-authored body text.
- Comment UI should not ask for a title; any stored title or display label should be generated from the parent entry while the body or preview text carries the comment substance.
- `quoteEntries.quotedPersonReferentId`, when present, must point to a Person Referent.
- `quoteEntries.sourceEntryId`, when present, must point to the larger entry/source represented by the Quote.
- `eventEntries.locationPlaceReferentId`, when present, must point to a Place Referent.
- Event times should be coherent: `endsAt`, when present, should not be earlier than `startsAt`.
- `rsvpEntries.eventEntryId` must point to an Event Knowledge Entry.
- `rsvpEntries.personReferentId` must point to a Person Referent.
- An RSVP should be unique per Event and Person unless the product later supports response history.
- `organizationEntries.organizationKind` must be one of School, Church, Family, or Community.
- An Organization requires a name and Organization kind in the MVP. Place, address, website, members, and parent relationships are optional enrichment unless a later Type Behavior makes one required for a specific Organization kind.
- In general, Person, Group, Place, and similar world-modeling Knowledge Types should require only a name to establish their initial represented Referent identity. Additional details such as membership, address, relationships, roles, contact information, or biographical facts should be optional enrichment unless the Type Behavior requires a specific discriminator to create a valid entry.
- A Person can be validly created before role, organization membership, user-account linkage, email, or family relationships are known. Smart Storage should ask follow-up questions only when needed to resolve plausible identity ambiguity or optional enrichment.
- A Place can be validly created from a name alone. Address, locality, region, country, and geographic precision are optional enrichment unless needed to distinguish between plausible matching Places.

Entry Representations and Sources

- An Entry Representation belongs to exactly one Knowledge Entry.

- A Knowledge Entry should have at most one primary representation per representation need.
- Type Behavior should define default Primary Representation selection, with user override during proposal review when multiple representations are valid.
- Representation Role should not replace explicit Primary Representation selection.
- Type Behavior should define allowed or default Representation Roles for each Knowledge Type, while Smart Storage may infer roles and the User may correct them during proposal review.
- `entryRepresentations.representationRole` should be required, using `unspecified` when the role is not yet known. Type Behavior may require a more specific Representation Role for workflows where `unspecified` is not acceptable.
- Representation Role should use a small closed shared enum in persistence, while Type Behavior defines which roles are allowed or default for each Knowledge Type.
- The initial Representation Role enum should be `unspecified`, `primaryContent`, `manuscript`, `slides`, `transcript`, `recording`, `thumbnail`, and `supportingMaterial`.
- `prosemirrorDocumentId` remains an arbitrary string compatible with the collaborative editor.
- File, audio, video, URL, and plain text representations should use the fields matching their `representationKind`.
- A Source is Bronze Layer raw material, not a Knowledge Type and not a Knowledge Entry.
- A Contribution Submission may group multiple Sources under one user intent, such as typed substantive text, uploaded files, external URLs, audio, or video submitted together for one Lesson, Sermon, or other intended Knowledge Entry.
- A durable Contribution Submission is required for multi-Source, Smart Storage, import, upload, deferred review, retry, or Reprocessing workflows, but not for simple direct posts that create Gold Layer Knowledge Entries immediately.
- The first `contributionSubmissions` table should include `submittedByUserId`, `submissionStatus`, `primaryIntendedKnowledgeType`, `primaryIntendedTitle`, `primaryIntendedBodyPreview`, `contributionNote`, `intendedVisibilityKind`, `intendedVisibilityTargetKey`, `reviewScopeKind`, `reviewScopeTargetKey`, `createdAt`, and `updatedAt`. Draft sessions, delivery channels, and denormalized child lists should stay out of the first schema slice.
- The initial Contribution Submission status enum should be `submitted`, `processing`, `reviewReady`, `partiallyAccepted`, `accepted`, `rejected`, and `cancelled`. Submission status describes the parent lifecycle and should not replace Smart Storage Run status or Smart Storage Proposal status.
- Review Scope should use its own schema enum, initially `private`, `organization`, `group`, and `public`, rather than reusing the Visibility Scope validator. The matching initial values do not make Review Scope and Visibility Scope interchangeable.
- `reviewScopeTargetKey` should use the same string-key storage style as `visibilityTargetKey` in the first implementation, with backend validation that the key format matches `reviewScopeKind`.
- The first Smart Storage implementation should persist Contribution Submissions as parent rows and persist Sources as child rows linked to the Contribution Submission. Standalone Sources should not remain the main Smart Storage grouping model.
- `sources.contributionSubmissionId` may be optional in the first schema change for migration compatibility, but new durable Contribution Submission and Smart Storage mutation paths must provide it.
- Uploaded file Sources should reference Convex storage IDs and metadata after storage succeeds; contribution mutations should not carry file bytes.
- Uploaded files should remain preserved Bronze Sources after successful storage even if extraction, preview, transcription, or Smart Storage analysis fails.

- External URL Sources should always preserve the submitted URL, while fetched content or transcripts should be snapshotted when retrieved or used so Smart Storage review remains explainable after link drift.
- Each Contribution Submission should have one Primary Intended Entry, while Smart Storage may propose additional derived Knowledge Entries from the same Sources.
- Multiple Smart Storage Proposals from one Contribution Submission should be accepted one proposed Knowledge Entry at a time.
- Sources are submission-level raw material; Smart Storage Proposals should cite the specific Sources, excerpts, ranges, or URLs that support them.
- Source citations for Smart Storage Proposals should be stored as child rows, not as an unbounded array on the proposal document.
- The first `proposalSourceCitations` table should include `proposalId`, `sourceId`, `citationKind`, `excerptText`, `locator`, `externalUrl`, `rationale`, and `createdAt`. The initial citation kind enum should be `wholeSource`, `textExcerpt`, `fileLocator`, and `externalUrl`; `excerptText` should be bounded and optional.
- Shared Source provenance may indicate that entries are related, but the app should not create a generic `relatedTo` Gold Layer relationship; explicit relationships should be typed, Tag-based, or introduced through later Knowledge Type behavior.
- Accepted proposals should explicitly decide which submitted Sources become Entry Representations; Bronze Sources should not automatically become Gold Layer representations.
- A Source created for later Smart Storage or Reprocessing of a directly created Knowledge Entry is a snapshot of that entry's current representation at the time of reprocessing, not the original raw direct-post submission.
- A Source may produce many Knowledge Entries through `sourceOutputs`.
- A `sourceOutputs` row must point to an existing Source and an existing produced or derived Knowledge Entry.
- In Smart Storage, Bronze Layer maps to Sources, Silver Layer maps to Smart Storage Proposals, and Gold Layer maps to confirmed Knowledge Entries.
- Smart Storage should not run automatically for every Contribution; direct posting should create the Knowledge Entry currently displayed to the user when that path is selected, without creating a Bronze Layer Source by default.
- Contribution Preview should use deterministic application logic, not an LLM, to show the proposed Knowledge Type, Knowledge Context, and visible entry attributes before the user submits.
- Formal Silver Layer records are required for Smart Storage Proposals because review, retry, and partial acceptance workflows need durable state between Bronze Sources and Gold Layer Knowledge Entries.
- Smart Storage Proposals should store contract-shaped domain data rather than raw Convex write payloads, and acceptance should translate validated proposals into the current persistence schema.
- Smart Storage Contracts and Type Behaviors must be versioned in the database as immutable content snapshots, and each Smart Storage Proposal should record the versions used to generate it.
- Type Behavior should be versioned as a whole per Knowledge Type, with field-level enrichment and provenance rules inside the immutable snapshot.
- Smart Storage Contract versions should contain stable reusable rules and templates; request-specific Source, context, candidate, and evidence input should be snapshotted separately with the enrichment run or proposal.
- Request-specific input snapshots should record what the LLM actually saw and link to the full Bronze Layer Source rather than duplicating raw Source content by default.
- Smart Storage Runs should link to the durable Contribution Submission and may also link to a primary Source when a run is centered on one submitted Source.

- Smart Storage Runs should preserve raw model output separately from parsed Smart Storage Proposals.
- Failed LLM calls, parse failures, and validation failures should be represented on Smart Storage Runs rather than by creating Smart Storage Proposals.
- Smart Storage Run status should track operational processing with queued, running, succeeded, no-proposal, failed, and superseded states, separate from Smart Storage Proposal review status.
- No-proposal Smart Storage Runs should be visible only as quiet Source/review state, not as Answers, Knowledge Slots, or proposal cards.
- Bronze Sources and Silver Smart Storage Proposals should stay out of the normal Answer Feed, but matched pending material may be shown on the relevant Referent Page for authorized review under the Contribution Submission or review scope.
- Smart Storage Proposals should link directly to their Contribution Submission as well as to their Smart Storage Run so review queries can load pending proposals by submission without hopping through runs. The backend must enforce that the Proposal and Run point to the same Contribution Submission.
- Smart Storage Proposal acceptance should validate against the original recorded Smart Storage Contract and Type Behavior versions unless those versions are marked incompatible or retired.
- Smart Storage Proposal acceptance must check Review Scope permission plus create, update, and Referent Identity Scope permissions for the Gold Layer result.
- Smart Storage Proposal records should preserve the original generated proposal separately from the current reviewed proposal.
- Accepting a Smart Storage Proposal should atomically create or connect the complete proposed Knowledge Entry shape; partial acceptance should be represented by editing, splitting, or rejecting proposals before acceptance.
- Smart Storage Proposal acceptance must perform the authoritative current identity check for Tags, Referents, and same-typed represented Knowledge Entries, regardless of matches proposed earlier.
- When an accepted proposal targets an existing Knowledge Entry, the backend must verify the User has permission to edit that entry before adding confirmed information to it.
- Smart Storage Proposal review should explicitly distinguish existing referenced Referents and Tags from new Referents and Tags that acceptance will create.
- Reprocessing proposals should explicitly distinguish edits to an existing Gold Layer Knowledge Entry from creation of additional Gold Layer Knowledge Entries.
- Smart Storage Contract or Type Behavior version changes may trigger dataset-wide Reprocessing to create suggested edits, Smart Storage Proposals, or Upgrade Candidates, but should not silently rewrite existing Gold Layer Knowledge Entries.
- Accepted Reprocessing edits should preserve explicit upgrade provenance without storing old versions on hot Knowledge Entry records in a way that harms normal read performance.
- Older upgrade versions may be archived after a retention window as long as enough summary provenance remains to explain the upgrade and locate audit history when authorized.
- Factual Provenance may be entry-level when one evidence trail supports the whole proposal, but enriched factual fields with distinct evidence should carry field-level provenance.
- Required Factual Provenance should be specified by the Smart Storage Contract and Knowledge Type field behavior rather than applied globally to every field.
- Proposal Confidence should be coarse review guidance for proposals or enriched factual fields, not Human Weight, truth, or a replacement for user confirmation.

- Low Proposal Confidence should warn but not universally block acceptance; contract validity, required field completion, required provenance, and identity resolution are the blocking conditions.
- Smart Storage Proposal status should distinguish review state from enrichment-run failures; failed LLM calls should belong to the run or retry state rather than to a proposal that may never have been created.

Bible Passage and Scripture

- Bible Passage is a Referent and Tag Knowledge Type, but not an authorable Knowledge Entry type in the MVP.
- Bible Passage identity must be based on normalized verse ordinal ranges, not raw citation strings.
- Bible Passage ranges should be sorted in canonical Bible order.
- Overlapping or adjacent Bible Passage ranges should be merged before computing `canonicalKey`.
- Equivalent passage strings must resolve to the same Bible Passage Referent.
- Bible Passage range arrays must stay bounded; use a range table later if passage sets become large.
- Bible structure records identify canonical books, chapters, verses, and verse ordinals.
- Bible verse text is translation-specific and must stay separate from canonical verse structure.
- A `bibleVerseTexts` row must be unique for a translation and canonical verse.
- Translation text should only be seeded from a vetted source with acceptable licensing.
- Lazy Bible Passage navigation may record analytics for a normalized passage target before a persisted Tag or Referent exists.

Users, People, Organizations, and Memberships

- User is authentication/access infrastructure, not a Knowledge Type.
- Every signed-up User must link to exactly one Person Knowledge Entry through `userProfiles`.
- Not every Person Referent or Person Entry has a User.
- `userProfiles.personEntryId` must point to a Person Knowledge Entry.
- `userProfiles.personReferentId` must be the represented Referent for `personEntryId`.
- `userProfiles.personTagId` must be the canonical Tag for `personReferentId`.
- A User should have exactly one active `userProfiles` row.
- Profile-facing identity facts should belong to the linked Person Knowledge Entry or related world-model records, while account-only access settings should remain on User/auth/profile infrastructure.
- A Person may be a member of Organizations and Groups through `memberships`.
- `memberships.personReferentId` must point to a Person Referent.
- Organization member management should present active and Pending Memberships together as members, with status making unclaimed access clear rather than separating them into an invitation list.
- Adding a member by email may create the smallest valid Person placeholder and a Pending Membership; the email is Contact Identity evidence, not the Person's definitive identity.
- A membership target must identify exactly one target matching `targetKind`.
- For `targetKind: "organization"`, `organizationReferentId` must be present and `groupReferentId` absent.
- For `targetKind: "group"`, `groupReferentId` must be present and `organizationReferentId` absent.

- Organization membership targets must point to Organization Referents.
- Group membership targets must point to Group Referents.
- When `memberUserId` is present, it should match the User linked to `personReferentId`.
- A Membership created before its Person has a linked User should not grant account access until the User proves they are that Person through verified contact identity.
- A User may claim Pending Memberships associated with multiple verified Contact Identities, such as separate personal, school, and church email addresses.
- Claiming a Pending Membership for a different Contact Identity should attach the Membership to the User's single Person when the evidence is clear; ambiguous Person consolidation should route through identity review rather than silently merging Person Referents.
- The onboarding rule that a User belongs to at least one Organization is required later, but not enforced by schema alone.
- A Group can be validly created before its full membership is known. Group membership should be optional enrichment rather than a blocking requirement for accepting a Group Smart Storage Proposal.

Knowledge Slots

- A Knowledge Slot is a workflow request, not a Knowledge Type.
- Each Knowledge Slot requests exactly one Knowledge Entry type through `requestedKnowledgeType`.
- `requestedKnowledgeType` must be an authorable entry type, so it must never be `biblePassage`.
- Slot Tags are the frozen Knowledge Context for the requested entry.
- Slot fulfillment should point to at most one `fulfilledEntryId`.
- A fulfilled entry's `knowledgeType` must match the slot's `requestedKnowledgeType`.
- A fulfilled entry should include the frozen Slot Tags in its Knowledge Context unless a future workflow explicitly changes that rule.
- A Knowledge Slot target must identify exactly one target matching `targetKind`.
- For `targetKind: "user"`, `targetUserId` must be present and the other target fields absent.
- For `targetKind: "person"`, `targetPersonReferentId` must be present and the other target fields absent.
- For `targetKind: "organization"`, `targetOrganizationReferentId` must be present and the other target fields absent.
- For `targetKind: "group"`, `targetGroupReferentId` must be present and the other target fields absent.
- For `targetKind: "public"`, no target ID fields should be present.
- Slot status transitions should be coherent: only fulfilled slots should have `fulfilledEntryId`.
- `dueAt` is optional; overdue status should be derived or maintained consistently by workflow code.

Series and Deferred Relationships

- Series is a Knowledge Type; ordered membership is represented by `seriesItems`.
- A `seriesItems` row belongs to one Series Knowledge Entry.
- A `seriesItems` row must identify exactly one item matching `itemKind`.
- `position` should be unique within a Series unless the product later supports ties.
- Generic `entryRelations` are intentionally deferred.

- Cross-type person-role search through `entryPeople` is intentionally deferred.
- Thumbnail or image asset tables are intentionally deferred.

Open Questions

- How should Human Weight be calculated or assigned for ordinary user-created entries?
- What Type Behaviors belong to the first non-Scripture Knowledge Types?
- How should networks of organizations be represented in Visibility Scope?

MVP Frontend Core Loop

This document captures product/UI implementation decisions for the MVP Explore/Contribute loop. Domain vocabulary belongs in `CONTEXT.md`; frontend state, component boundaries, routing policy, and build sequencing belong here.

Decisions

Page-level loop state owns the active Knowledge Context

The Explore/Contribute surface should derive one shared loop state at the page level from the current route and active Tags. The Knowledge Navigator and KnowledgeRequestComposer can update that state, but neither component owns it directly.

This keeps the six MVP components coordinated around one current Knowledge Context:

- Knowledge Navigator edits active Tags.
- KnowledgeRequestComposer submits a Knowledge Request and may propose or apply Tags.
- Answer Feed reads the mapped Knowledge Context and shows Answers.
- Knowledge Slot Card reads the same Knowledge Context and shows missing future Answers.
- Contribution Editor contributes into the current or frozen Knowledge Context.
- Knowledge Entry Card renders Answer data supplied by the feed or another parent surface.

`CONTEXT.md` should continue to define Knowledge Context as a domain term. Any frontend type such as `KnowledgeLoopState` is an implementation contract and should stay in product/UI documentation or TypeScript modules.

Active Tags are URL state; Knowledge Requests are not automatic URL state

The URL should update automatically when active Tags change, because active Tags determine the user's current Dashboard, Referent Page, or Context Page location.

The URL should not automatically serialize the full submitted Knowledge Request, Answer results, Knowledge Slots, Contribution drafts, Smart Storage state, or editor content. A Knowledge Request can stay in page state unless the user explicitly saves it as a Question or uses a later share action.

Initial MVP route shape:

- `/` for the Dashboard with no active Tags.
- `/scripture/:passageString` for one Bible Passage Referent Page.
- `/goto/:tagId` for one non-Scripture Referent Page.
- `/explore?tagIds=a,b,c` for a multi-Tag Context Page.

This keeps the URL focused on "where am I?" rather than "everything I typed."

Active Tags are an unordered set for Knowledge Context identity

Multi-Tag Context Pages should treat active Tags as an unordered set. Selection order may matter later for analytics or interaction history, but it should not change Knowledge Context identity.

The MVP should canonicalize multi-Tag URLs and context keys from sorted Tag IDs:

- `/explore?tagIds=romans-8,atonement`
- `/explore?tagIds=atonement,romans-8`

Both examples should resolve to the same Knowledge Context and `contextKey`.

If a future UI needs one Tag to remain visually or behaviorally primary, represent that as explicit state such as `focusTagId` rather than relying on Tag array order.

Knowledge Requests propose mapped Tags before changing the URL

Submitting a Knowledge Request should not immediately mutate active Tags or navigate the user away from the current location. The KnowledgeRequestComposer should first produce a proposed mapped Knowledge Context that the user can accept or edit.

Initial MVP request state:

```
type KnowledgeRequestDraft = {
  text: string;
  mappedTags: ActiveTag[];
  mappingStatus: "idle" | "mapping" | "proposed" | "applied";
};
```

When the user applies the proposed Knowledge Context, active Tags update, the URL changes, and the Answer Feed refreshes. The user may also ignore the proposal and ask within the current Knowledge Context.

This preserves the Knowledge Navigator as the source of truth for active Tags while letting the KnowledgeRequestComposer influence them.

Knowledge Composer is the shared input family

Header search, Knowledge Navigator ask/add inputs, KnowledgeRequestComposer, and Contribution Editor should be treated as variants of a broader Knowledge Composer surface. They share the same conceptual job: accept user text from a current Knowledge Context and route it toward either a transient Knowledge Request or a durable Contribution.

The global header search is the exception to current-page context inheritance. It should search everything the current User can access, independent of the current Active Knowledge Context, and should not be confused with public-only search or search limited to the Global Knowledge Context. Context-aware asking belongs in the Knowledge Navigator or page composer where the active Tags are visible.

The UI must still make the submission behavior explicit. A compact composer may only make a transient Knowledge Request, a full Contribution Editor may create a Knowledge Entry or start Smart Storage, and a richer composer may offer both paths. Do not imply that every search or ask input has created a stored Question unless the user intentionally saves or contributes it as one.

When a user saves or contributes a Question from a Knowledge Composer, the app should create the Question entry without automatically changing active Tags or navigating away. The resulting Question Tag may be offered as a next action, such as "Open Question Context", which would apply the existing active Tags plus the Question Tag.

Saving a Question should not automatically create a Knowledge Slot. The UI may offer a separate "Request an answer" action from the saved Question or Question context when the user wants to direct a future Answer request to a person, group, organization, network, or open audience.

The Knowledge Navigator input should default to asking within the active Tags when the user submits question-shaped text or other free text. While the user types, it should offer Tag suggestions as selectable completions. Selecting a Tag changes active Tags; submitting free text creates a transient Knowledge Request inside the current Knowledge Context.

The Answer Feed can include Knowledge Slots as missing future Answers

The Answer Feed should own the ordered reading experience for the current Knowledge Context. It can render both existing Answers and Knowledge Slots, as long as the data contract preserves the domain distinction between a Knowledge Entry considered as an Answer and a Knowledge Slot requesting a future Answer.

Initial MVP feed item contract:

```
type AnswerFeedItem =
  | { kind: "answer"; entry: KnowledgeEntrySummary }
  | { kind: "slot"; slot: KnowledgeSlotSummary };
```

Responsibilities:

- Answer Feed queries, orders, and groups feed items for the current Knowledge Context.
- Knowledge Entry Card renders existing Answers.
- Knowledge Slot Card renders missing future Answers and contribution calls to action.
- A secondary rail may show compact contribution affordances, but it should not be the only place users discover Knowledge Slots.

This supports one Explore/Contribute loop while keeping efficient search concerns visible: entries and slots need compatible context-matching contracts, but they remain different domain objects.

Context fit means an item contains all active Tags

For the MVP, a Knowledge Entry or Knowledge Slot fits the current Knowledge Context when its Tags contain every active Tag. The item may have additional Tags.

```
function fitsKnowledgeContext(itemTagIds: string[], activeTagIds: string[]) {
  return activeTagIds.every((tagId) => itemTagIds.includes(tagId));
}
```

This means an entry tagged with `Romans 8`, `atonement`, and `lesson` still fits a current Knowledge Context of `Romans 8` plus `atonement`.

Initial ordering should be easy to explain before it becomes clever:

- Show entries and slots that contain all active Tags.
- Optionally boost exact-context matches.
- Order existing Answers by Feed Priority, with Human Weight as the primary signal.
- Reserve enough feed exposure for low-Evidence Maturity weight-bearing Answers to gather feedback that can inform later Human Weight evaluation.

Knowledge Entry surfaces may offer Human Weight Feedback through a subtle entry-level dialog. The interaction should feel like lightweight evaluation, not a prominent social-like system. Feedback should be always reachable from a small entry action and only occasionally prompted for low-Evidence Maturity entries after meaningful interaction, never as a loud first-view modal or public like counter.

- Place Knowledge Slots where missing future Answers are understandable and actionable.

Exact `contextKey` values may be useful for caching, denormalization, or exact-match boosts, but exact Tag-set equality should not be the baseline definition of context fit.

Phase 2 can add broader-result recommendations

When the current Knowledge Context has no matching Answers, a later phase can recommend Answers from broader result sets. For example, if the active Knowledge Context is `Romans 8:28` plus `Holy Spirit`, an entry tagged only with `Romans 8:28` does not fit that exact Knowledge Context because it does not contain every active Tag. It belongs to a broader result set produced by relaxing one or more active Tags.

Call this Phase 2 fallback **Broader Context Recommendations**. Be precise about the word "superset": the broader result set is a superset of the exact-context result set, but its Tag set is not a superset of the active Tags.

Related terms for product/UI docs:

- **Context Match**: contains all active Tags.
- **Exact Context Match**: contains exactly the active Tags.
- **More Specific Context Match**: contains all active Tags plus extra Tags.
- **Broader Context Recommendation**: relaxes one or more active Tags because no Context Matches exist.
- **Related Recommendation**: overlaps through other relationship logic, not merely relaxed active Tags.

Shared Frontend Contracts

ActiveTag

The minimum shared frontend contract for an active Tag should be display-ready without duplicating full database rows.

```
type ActiveTag = {
  id: string;
  label: string;
  knowledgeType: KnowledgeType;
  canonicalKey: string;
  href: string;
};
```

Do not include counts, aliases, recognition state, full Referent data, or feed-specific metrics on `ActiveTag`. Components that need those details should query them through focused contracts.

Feed Summary Objects

Knowledge Entry and Knowledge Slot cards should receive compact summary objects from their parent feed or page. These contracts are intentionally smaller than full database documents.

```
type KnowledgeEntrySummary = {
  id: string;
  title: string;
  knowledgeType: KnowledgeType;
  previewText: string;
  primaryTagLabel: string;
  contextPreviewTagLabels: string[];
  humanWeight: number;
  href: string;
  updatedAt: number;
};

type KnowledgeSlotSummary = {
  id: string;
  title: string;
  requestedKnowledgeType: KnowledgeType;
```

```
  promptText?: string;
  status: "open" | "fulfilled" | "cancelled" | "overdue";
  contextPreviewTagLabels: string[];
  targetLabel: string;
  dueAt?: number;
  href: string;
};
```

Keep full Entry Representations, Source details, long content, comment threads, fulfillment history, and Smart Storage progress out of these feed summary contracts.

Build Order

Build the MVP frontend in this order:

1. Shared contracts and route/context parsing.
2. Knowledge Entry Card and Knowledge Slot Card using fixtures.
3. Answer Feed using mixed fixture items.
4. Knowledge Navigator writes active Tags to the URL.
5. KnowledgeRequestComposer proposes mapped Tags.
6. Contribution Editor uses the current Knowledge Context.
7. Backend queries and mutations replace fixtures.
8. Integrated Explore/Contribute loop test.

This sequence keeps the shared Knowledge Context contract stable before component work fans out, while still letting card and feed work begin before backend queries are complete.

Parallel handoff candidates:

- Contracts and route/context parser: `docs/handoffs/mvp-frontend-contracts-and-routing.md`.
- Knowledge Entry Card and Knowledge Slot Card: `docs/handoffs/mvp-entry-and-slot-cards.md`.
- Answer Feed fixture behavior: `docs/handoffs/mvp-answer-feed.md`.
- Knowledge Navigator URL behavior: `docs/handoffs/mvp-knowledge-navigator.md`.
- Later KnowledgeRequestComposer and Contribution Editor behavior after context contracts stabilize.

Test Strategy

Component Tests

Use fixture props for component tests. Mock Convex only at page/container boundaries.

Recommended component coverage:

- Knowledge Entry Card: renders title, type, preview, Human Weight, Tag labels, and link from `KnowledgeEntrySummary`.
- Knowledge Slot Card: renders requested type, prompt, status, target, due date, Tag labels, and contribution call to action from `KnowledgeSlotSummary`.
- Answer Feed: renders mixed `AnswerFeedItem[]` fixtures, preserves the domain distinction between Answers and Slots, orders existing Answers by Human Weight, and handles empty states.

- Knowledge Navigator: parses and writes active Tags through the shared route/context contract.
- KnowledgeRequestComposer: captures a Knowledge Request, shows proposed mapped Tags, and supports apply/ignore behavior.
- Contribution Editor: shows a deterministic Contribution Preview, submits direct contributions with the current active Tags or a frozen Slot context, and distinguishes direct posting from Smart Storage when both are available.

Only page/container tests should mock Convex hooks or function references.

Integrated MVP Loop Test

The first integrated loop test should prove one happy-path Explore/Contribute flow without depending on nondeterministic AI behavior.

Scenario:

1. Start at `/scripture/romans-8-28`.
2. Page derives one active Tag for `Romans 8:28`.
3. User adds `Holy Spirit` through the Knowledge Navigator.
4. URL becomes `/explore?tagIds=holy-spirit,romans-8-28` using canonical sorted IDs.
5. Answer Feed shows entries that contain both active Tags, ordered by Human Weight.
6. If no matching entry exists, the feed shows a Knowledge Slot for that Knowledge Context.
7. User opens the Slot call to action.
8. Contribution Editor receives the frozen Slot Tags.
9. Submitting creates or simulates a Knowledge Entry with those Tags.
10. Feed shows the new Entry as an Answer.

Smart Storage should be deterministic in this test, either through a test mutation or fake adapter.

Contribution Editor Boundary

The Contribution Editor should not own Smart Storage. It should own contribution UI state, show what will be contributed if the user submits, and submit to a backend/container workflow that either creates a Knowledge Entry directly or starts Smart Storage.

The backend/container workflow should preserve the Bronze Layer Source immediately when the user chooses Smart Storage, before any LLM call. Smart Storage proposal generation should derive from that saved Source so failed enrichment can be retried without losing the user's raw contribution. Direct posting should create the Knowledge Entry currently displayed to the user without creating a Bronze Layer Source, Smart Storage Run, or Smart Storage Proposal by default.

The editor should also carry an active authorable Knowledge Type. The current Knowledge Context says where the contribution belongs; the active Knowledge Type says what kind of Knowledge Entry the user intends to contribute.

As the user types, changes selected context, or selects an upload, the editor should compute a deterministic Contribution Preview without calling an LLM. The preview should show the current best guess for:

- The Knowledge Type being contributed.
- The Knowledge Context where it will be contributed.
- The visible attributes or fields the resulting entry will have.

For the MVP, the Contribution Preview should not infer Knowledge Type from text patterns such as question marks, dates, URLs, or quotation marks. It should derive Knowledge Type from explicit workflow state only:

1. Knowledge Slot requested type, if fulfilling a Slot.
2. Explicit user-selected Knowledge Type, if present.
3. Default fallback to `words`.

The preview's visible attributes should also stay conservative: title, Knowledge Type, Knowledge Context Tags, and a bounded body or representation preview. Hidden inferred fields, factual enrichment, and type challenges belong in Smart Storage proposal review rather than in the direct Contribution Preview.

The primary submit action must match the visible Contribution Preview so the user knows exactly what pressing Enter or clicking submit will contribute. When both paths are available, the UI should distinguish direct posting, such as "Post Comment", from Smart Storage, such as "Store Smartly". Straightforward Comments should be eligible for direct posting without Smart Storage.

Smart Storage promotion should not be determined only by the active Knowledge Type. Because `words` is the conservative default for ordinary contribution surfaces, a Dashboard contribution may still promote Smart Storage even while the direct Contribution Preview says it would create a Words entry. Smart Storage should be promoted by contribution surface and input intent, such as Dashboard source/import contribution, uploads, URLs, long pasted material, or unsure/unknown type input.

When the user is in the Global Knowledge Context, meaning no Tags are active in the Knowledge Navigator, the default Contribution Editor mode should be Smart Storage. The primary action should be "Store Smartly" and should preserve a Bronze Layer Source before enrichment. Direct posting should remain available as a secondary action, such as "Post as Words", for users who intentionally want to create the displayed Words entry without Smart Storage.

When the user is on a Referent Page or multi-Tag Context Page, the default Contribution Editor mode should be direct posting because the active Tags already provide the Knowledge Context. A direct contribution from those pages should include the active Referent Tag or active Tags on the created Knowledge Entry by default. Smart Storage should remain available as a secondary or promoted option for source-like inputs such as uploads, URLs, long pasted material, or unknown-type contributions.

Active page Tags should become context Tags on direct contributions by default, not the represented Tag for the new Knowledge Entry. For non-Comment direct posts, the new entry should receive its own represented Referent and primary Tag from its submitted title and Knowledge Type, while the active Referent Page or Context Page Tags express what the entry references. For example, a Question contributed from a Bible Passage Referent Page references that Bible Passage; it does not represent the Bible Passage.

Non-Comment direct posts should require a title before submission so represented Referent identity stays deterministic. Comment contributions should not show a title field; they should behave like replies, with any stored title, display label, represented Referent key, and primary Tag generated by application logic rather than typed by the user.

Comment represented Referent identity should be generated from stable relational facts, not from body text. The MVP key shape can use the parent Knowledge Entry, the commenting Person Referent, and generated uniqueness such as creation time or an inserted ID. The stored title or display label can be generated as "Comment on {parent entry title}", while the body preview carries the user-visible substance.

Question direct posts should not ask for a separate title distinct from the question. The question text itself should become the stored title and represented Referent identity for the Question entry, with the active page Tags stored as context Tags. Question entries may also include optional details or body text, but those details should not change the represented Question identity.

Prayer Request direct posts should behave like normal titled entries in the MVP. The submitted title or subject becomes the Prayer Request entry's represented Referent identity, and any details remain supporting body or representation content.

RSVP should be hidden from the generic Contribution Editor Knowledge Type picker. RSVP entries should be created from Event-specific RSVP controls or an RSVP-focused Slot workflow because their identity depends on the Event and responding Person rather than a generic user-authored title.

Person, Organization, Group, and Place should remain creatable through the Contribution Editor because the application needs to model named things in the world, not only authored content. In the generic picker, these world-modeling types may be de-emphasized or placed after content-oriented types. Dedicated actions such as "Create Group" should route to the Contribution Editor with the relevant Knowledge Type preselected rather than opening a separate multi-field form.

For complex or world-modeling Knowledge Types, the preferred UI should be a guided contribution flow that asks for missing required information one question at a time. The Contribution Editor should avoid introducing separate multi-field forms for these types unless a later workflow proves that a compact form is meaningfully clearer.

For minimum required information, the MVP should use a name-first default for world-modeling types. Person, Group, and Place can generally become accept-ready from a name plus enough context to avoid obvious ambiguity. Organization also needs its MVP Organization kind when that kind is not already known or confidently inferred. Membership, address, roles, relationships, and other descriptive facts should be optional enrichment rather than blocking fields unless a Type Behavior explicitly requires them.

Guided contribution is not always Smart Storage. When a user explicitly chooses a Knowledge Type through an action such as "Create Group", the Contribution Editor can run a deterministic guided flow for required fields only, such as asking "What is the group called?" Smart Storage guided flows add interpretation, type recognition, enrichment, and ambiguity resolution on top of that prompt pattern.

Deterministic guided direct creation does not need a separate final summary confirmation before creating Gold. Once required answers are complete, the app may create the Knowledge Entry and then open or focus that new entry in an editable state. The Contribution Editor should hand off to the created entry rather than remain the primary correction surface after submission. This applies to explicit direct creation flows; Smart Storage still requires proposal review and user acceptance before creating Gold.

The same post-create focus rule should apply to the User Profile view. After profile-related guided creation or onboarding completes, the app should open or focus the Profile surface in an editable state so the user can immediately correct or enrich profile details there.

The Profile view should edit Person-facing identity and profile facts through the linked Person Knowledge Entry or related world-model records. Account-only fields, such as authentication, email, password, connected providers, or other access settings, should be edited in User Settings rather than on the Profile surface.

Natural-language recognition and follow-up questions belong to Smart Storage or a guided proposal flow, not to the direct Contribution Preview. For example, if a user enters "I started a basketball club at Arche" in the Global Knowledge Context, Smart Storage may recognize a proposed Group in the Knowledge Context of Arche Classical Academy, tag the subject Basketball, and ask who belongs to the group before the proposal is accepted as Gold.

Guided follow-up answers should update the current reviewed Smart Storage Proposal while it remains Silver Layer. They should not create partial Gold Layer records before the user accepts the completed proposal.

Guided follow-up questions should ask for required fields and blocking ambiguity resolution first. After the proposal is accept-ready, optional enrichment prompts may remain available, but they should be skippable and should not prevent acceptance.

The "no multi-field forms" preference applies to creation and editing prompts, not to final review. A Smart Storage proposal review should show a compact summary of the complete proposed Gold entry before acceptance. If the user edits a value from that summary, the edit can open a focused one-field or one-question interaction rather than turning the review into a large editable form.

Initial editor boundary:

```
type ContributionPreview = {
  attributes: { label: string; value: string }[];
  context: ActiveTag[];
  knowledgeType: KnowledgeType;
```

```
  mode: "direct" | "smartStorage";
  submitLabel: string;
};

type ContributionEditorProps = {
  context: ActiveTag[];
  activeKnowledgeType: KnowledgeType;
  contributionPreview: ContributionPreview;
  slot?: KnowledgeSlotSummary;
  onPostDirect(input: ContributionInput): Promise<ContributionResult>;
  onKnowledgeTypeChange(nextType: KnowledgeType): void;
  onStoreSmartly(input: ContributionInput): Promise<ContributionResult>;
};
```

The active Knowledge Type may come from a Knowledge Slot, an explicit user selection, or a later Smart Storage proposal. It must be an authorable Knowledge Entry type, so `biblePassage` is not valid for MVP contributions.

Knowledge Slot requested type should be locked or strongly fixed during Slot fulfillment so the resulting Knowledge Entry matches the Slot contract. Outside Slot fulfillment, Smart Storage may challenge the explicit user-selected type when the Source appears to match a more specific or more appropriate Knowledge Type. The review UI should make that challenge explicit and require the user to confirm the final type before creating a Knowledge Entry.

Active Knowledge Type precedence for Gold creation:

1. Knowledge Slot requested type.
2. User-confirmed Smart Storage Proposal type.
3. Explicit user-selected type.
4. Default fallback to `words`.

Naming

Use **Knowledge Composer** as the product/UI family name for text-entry surfaces where users ask, search, add context, or contribute from a Knowledge Context.

Use **KnowledgeRequestComposer** as the internal component name for the UI where a user makes a Knowledge Request.

Rationale:

- Knowledge Composer names the family of related input surfaces without making all of them durable contribution flows.
- A user submits a Knowledge Request during Explore.
- A Question is a Knowledge Type for a stored question mapped to a Knowledge Context.
- Keeping these distinct avoids confusing a transient request with a persisted Question Knowledge Entry.

The other current component names can remain:

- Knowledge Navigator
- Answer Feed
- Contribution Editor
- Knowledge Entry Card

- Knowledge Slot Card

Documentation Boundary

`CONTEXT.md` is a glossary and should stay free of implementation details. No new terms from this session need to be added to `CONTEXT.md` yet.

Terms already covered by `CONTEXT.md`:

- Knowledge Context
- Knowledge Request
- Question
- Answer
- Human Weight
- Knowledge Slot
- Smart Storage
- Explore
- Contribute
- Knowledge Navigator

Terms that should stay in product/UI docs or TypeScript contracts:

- `KnowledgeLoopState`
- `ActiveTag`
- `KnowledgeEntrySummary`
- `KnowledgeSlotSummary`
- `AnswerFeedItem`
- `ContributionPreview`
- Context Match
- Exact Context Match
- More Specific Context Match
- Broader Context Recommendation
- Knowledge Composer
- KnowledgeRequestComposer
- build order and handoff plan
- test strategy

Broader Context Recommendation can be reconsidered for `CONTEXT.md` later if it becomes a durable product concept rather than retrieval/UI behavior.

Separate Referents, Tags, and Knowledge Entries

Referents, Tags, and Knowledge Entries remain separate concepts in the data model. A Referent is the stable identity of a thing, a Tag is the canonical navigable pointer to that Referent, and a Knowledge Entry is content or work that represents one primary Referent and references other Referents through Tags. We keep them separate so users can navigate to and tag meaningful things before any Knowledge Entry represents them, while preserving stable identity across aliases, recognition, subscriptions, routes, and future naming changes.

Use Type Detail Tables for Knowledge Entries

Knowledge Entries store the Words-level fields needed for common search results and Knowledge Entry cards, including searchable content, display summary data, primary Tag navigation, denormalized card labels, visibility, and discoverability. Words has no separate one-to-one detail table because it defines the common entry shape, while every more specific Knowledge Type may have a one-to-one detail table for fields not shared by all entries; those detail tables should contain only actual type-specific fields and may start with only `entryId` when the MVP has no special attributes yet. Long or rich Words content belongs to Entry Representations instead. This keeps Explore and search queries fast in Convex by avoiding per-result hydration from type-specific tables, while allowing detail pages to load richer type-specific data only when a user navigates to an entry. Search result discoverability is separate from read visibility, so an entry may have a sanitized public preview without granting access to the full entry.

Model Knowledge Slots as Singular Workflow Requests

Knowledge Slots are workflow records, not Knowledge Types, and each slot requests exactly one Knowledge Entry of a specified Knowledge Type within a frozen Knowledge Context. The slot stores workflow state such as status and due date, captures its required Tags at creation, and may point to the single Knowledge Entry that fulfills it. This keeps slot fulfillment simple and stable while Series, Groups, and repeated slot creation handle workflows that need many requested entries.

Knowledge Entries Uniquely Represent Same-Typed Referents

A Knowledge Entry represents exactly one Referent with the same Knowledge Type, and a Referent may have at most one Knowledge Entry that represents it. Entry creation should create or reuse the Represented Referent and canonical Tag in the same persistence transaction, include that canonical Tag in the entry's `entryTags`, then refuse or redirect when another Knowledge Entry already represents that Referent; the user should be pointed to the existing Referent's Knowledge Page and guided toward tagging it, editing it, or proposing an authorized update. Smart Storage Proposal generation may suggest matches, but acceptance must perform this authoritative current identity check before creating Gold Layer knowledge. Other entries that are about that thing should reference its Tag or use a relationship rather than creating another entry for the same Represented Referent. Bible Passage is a Knowledge Type for Referents and Tags in the MVP, but Scripture text is modeled in Bible structure and verse text tables rather than as authorable Bible Passage Knowledge Entries.

Link Every User to a Person Entry

Every signed-up User must be linked to exactly one Person Knowledge Entry, while many Person entries may never be Users. This keeps User as authentication and access infrastructure, keeps Person as the taggable Knowledge Type, and lets other entries tag or relate to a user by referencing that user's Person Tag rather than treating User as a Knowledge Type.

Store Smart Storage Proposals as Durable Silver Records

Smart Storage Proposals are stored as durable Silver Layer records linked to Bronze Sources instead of existing only as temporary UI previews. This adds persistence complexity, but it preserves provenance, ambiguity, the original generated proposal, the current reviewed proposal, rejection, retry state, deferred review, partial acceptance, and future Reprocessing between the raw Source and any confirmed Gold Layer Knowledge Entries. Raw model output remains on the Smart Storage Run rather than becoming the reviewed proposal itself.

Store Smart Storage Proposals as Persistence-Agnostic Domain Contracts

Smart Storage Proposals store contract-shaped domain data instead of raw Convex write payloads. This keeps Silver Layer review records portable if the application migrates away from Convex or changes its internal schema, while acceptance remains responsible for validating the proposal and translating it into the current persistence model. Because proposals are durable, the Smart Storage Contract and Type Behavior versions that generated them are persisted as immutable content snapshots, not only version labels; request-specific Source, context, candidate, and evidence inputs are snapshotted separately. Acceptance validates against the original contract and behavior versions unless they have been marked incompatible or retired, in which case the proposal becomes stale and must be reprocessed.

Use Contribution Submissions as Smart Storage Parents

Multi-source Smart Storage contributions will be persisted under durable Contribution Submission parent records rather than grouped only by standalone Sources or transient composer state. This adds a parent workflow table before advanced extraction exists, but it preserves the user's single contribution intent, Primary Intended Entry, Review Scope, intended Visibility Scope, Contribution Note, Source inventory, runs, proposals, and later one-at-a-time acceptance in one reviewable spine. Simple direct posts can still create Gold Layer Knowledge Entries without durable Contribution Submission workflow state.

docs/agents/domain.md

Domain Docs

How the engineering skills should consume this repo's domain documentation when exploring the codebase.

Before exploring, read these

- `CONTEXT.md` at the repo root.
- `docs/adr/` for ADRs that touch the area about to be changed.

If any of these files do not exist, proceed silently. Do not flag their absence or suggest creating them upfront. Producer skills create them lazily when terms or decisions get resolved.

File structure

This is a single-context repo:

```
/
|-- CONTEXT.md
|-- docs/
|   `-- adr/
`-- src/
```

Use the glossary's vocabulary

When output names a domain concept in an issue title, refactor proposal, hypothesis, or test name, use the term as defined in `CONTEXT.md`. Do not drift to synonyms the glossary explicitly avoids.

If the concept is not in the glossary yet, note the gap for `grill-with-docs`.

Flag ADR conflicts

If output contradicts an existing ADR, surface it explicitly rather than silently overriding it.

Issue tracker: GitHub

Issues and PRDs for this repo live in GitHub Issues for `CordanoCorey/knowledgebase`. Use the `gh` CLI for issue tracker operations from inside this repository.

Conventions

- Create an issue: `gh issue create --title "..." --body "..."`
- Read an issue: `gh issue view <number> --comments`
- List issues: `gh issue list --state open --json number,title,body,labels,comments`
- Comment on an issue: `gh issue comment <number> --body "..."`
- Apply a label: `gh issue edit <number> --add-label "..."`
- Remove a label: `gh issue edit <number> --remove-label "..."`
- Close an issue: `gh issue close <number> --comment "..."`

Infer the repository from `git remote -v`. The `gh` CLI does this automatically when run inside this clone.

When a skill says "publish to the issue tracker"

Create a GitHub issue in `CordanoCorey/knowledgebase`.

When a skill says "fetch the relevant ticket"

Run `gh issue view <number> --comments`.

Triage Labels

The skills speak in terms of five canonical triage roles. This file maps those roles to the actual label strings used in this repo's issue tracker.

Label in mattpocock/skills	Label in our tracker	Meaning
`needs-triage`	`needs-triage`	Maintainer needs to evaluate this issue
`needs-info`	`needs-info`	Waiting on reporter for more information
`ready-for-agent`	`ready-for-agent`	Fully specified, ready for an AFK agent
`ready-for-human`	`ready-for-human`	Requires human implementation
`wontfix`	`wontfix`	Will not be actioned

When a skill mentions a role, use the corresponding label string from this table.

Edit the right-hand column to match the repository's issue tracker vocabulary if the labels change.

Code Handoff: Durable Bookmarked Knowledge Pages

Coding Agent Prompt

You are implementing the next vertical slice after durable sidebar pins.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/handoffs/header-sidebar-navigation.md`
- `docs/handoffs/durable-pinned-knowledge-pages.md`
- `src/App.tsx`
- `src/App.integrated.test.tsx`
- `src/index.css`
- `convex/schema.ts`
- `convex/lib/appAccess.ts`
- `convex/pinnedKnowledgePages.ts`
- `convex/pinnedKnowledgePages.test.ts`

Do not re-litigate the resolved navigation language unless code and docs directly contradict each other. Keep this slice focused on durable Bookmarks and the profile Bookmarks section; do not build subscriptions, full Global Search, drag ordering, or a full pin/bookmark management screen.

What To Do

Persist Bookmarks as a personal saved set of Knowledge Pages and surface them from the existing avatar/profile flow.

Implement durable user-specific bookmarks for Organization Knowledge Pages first:

- a Convex table and APIs for the current user's bookmarked Knowledge Pages,
- an Organization Knowledge Page Bookmark / Remove Bookmark control,
- the existing profile `Bookmarks` section backed by real bookmark data,
- the existing avatar menu `Bookmarks` shortcut pointing to that profile section,
- no sidebar placement and no notification/subscription behavior.

Target type: inferred next vertical slice.

Why This Target

The shell and durable pin slices made navigation usable and durable. The remaining resolved concept that already has a UI entry point is `Bookmark`: a User relationship to a Knowledge Page for later reference. `src/App.tsx` already has an avatar-menu link to `/profile?section=bookmarks` and a profile placeholder that says durable bookmarks are pending.

This slice turns that placeholder into real behavior while staying narrow. Organization Knowledge Pages are the first target because they already have durable page identity from the pin slice, visible page controls, active tags, and existing authenticated organization access data.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Previous shell handoff: `docs/handoffs/header-sidebar-navigation.md`
- Previous pin handoff: `docs/handoffs/durable-pinned-knowledge-pages.md`
- Convex guidance: `convex/_generated/ai/guidelines.md`
- Current app access helper: `convex/lib/appAccess.ts`
- Current pin implementation pattern: `convex/pinnedKnowledgePages.ts`, `convex/pinnedKnowledgePages.test.ts`
- Current schema: `convex/schema.ts`
- Current shell/profile/Organization pages: `src/App.tsx`, `src/index.css`
- Existing integrated tests: `src/App.integrated.test.tsx`
- Relevant ADRs: no bookmark-specific ADR found.
- Issue docs: no issue found for this slice; inline issue brief below.
- Prototype: no bookmark prototype found; current profile placeholder is the implementation reference.

Inline Issue Brief

What To Build

Add a persisted Bookmark model for Knowledge Pages and wire it through the visible product path:

1. A signed-in user can bookmark an Organization Knowledge Page.
2. The bookmarked page appears in the user's profile Bookmarks section.
3. The user can remove the bookmark from the Organization page or profile section.
4. The bookmark never appears in the primary sidebar and does not create notifications.

Acceptance Criteria

- [ ] A signed-in allowed user can query their bookmarked Knowledge Pages from Convex.
- [ ] A signed-in allowed user can bookmark an Organization Knowledge Page they can access.
- [ ] Bookmarking derives the current user server-side; no client function accepts `userId` for authorization.
- [ ] Bookmark uniqueness is per user and per Knowledge Page.
- [ ] Re-bookmarking an already bookmarked page is idempotent and updates `updatedAt` or `lastReferencedAt` rather than creating duplicates.
- [ ] Removing a bookmark deletes or hides only the current user's relationship to that Knowledge Page.
- [ ] Removing a bookmark does not unpin the page and does not unsubscribe the user from anything.
- [ ] Bookmarking does not add the page to the sidebar.

- [ ] The profile Bookmarks section renders bookmarked Knowledge Pages with label, kind/scope metadata, and a link back to each Knowledge Page.
- [ ] The avatar menu `Bookmarks` shortcut remains `/profile?section=bookmarks` and lands the user at the profile Bookmarks section.
- [ ] The Organization Knowledge Page shows a clear Bookmark / Remove Bookmark control that is visually distinct from Pin / Unpin.
- [ ] Bookmarking an Organization Knowledge Page records the action as meaningful for Recognized Context by upserting a user `tagRecognitions` row for that Organization's canonical Tag when one can be resolved.
- [ ] Existing durable pin behavior and sidebar overflow behavior still pass.

Out Of Scope

- Bookmarking every possible Knowledge Page type.
- Bookmarking User Views.
- Bookmark folders, tags, notes, bulk management, or search inside bookmarks.
- Recommendation UI or LLM prompting powered by bookmarks.
- Subscriptions or notification preferences.
- Pin management, drag ordering, or sidebar changes beyond proving bookmarks do not appear there.
- Full Global Search implementation.

Domain Language

- `Bookmark`: a User relationship to a Knowledge Page that saves it in the User's personal saved set for later reference and may inform user-specific tagging or Knowledge Context recommendations.
- `Pinned Knowledge Page`: a User relationship to a Knowledge Page that keeps it easy to return to from navigation, especially the sidebar.
- `Subscription`: a User relationship that affects notification behavior.
- `Knowledge Page`: a shared, world-facing location grounded in a Knowledge Context, Referent, Knowledge Entry, Organization, or other knowledge object.
- `User View`: a current-user scoped view such as Calendar, Notifications, Settings, or editing the user's own profile.
- `Recognized Context`: the historical union of Knowledge Contexts where a User or Organization has taken meaningful action.

Avoid `favorite` as product language. Do not use `Bookmark` to mean pin, subscription, or notification preference.

Decisions That Must Hold

- Bookmarks are separate from pins.
- Bookmarked Knowledge Pages do not appear directly in the primary sidebar by default.
- Bookmarks belong in user/account navigation, currently the user's profile section reached from the avatar menu.
- Bookmarking does not subscribe the user to notifications.
- Bookmarking is a meaningful action and may contribute to Recognized Context.
- Plain page visits alone should not add to Recognized Context.
- Dashboard stays the fixed first sidebar route.

- Pinned Knowledge Pages remain the sidebar middle group.
- Settings and the user's profile remain in the avatar account menu path, not duplicated in the header.

Relevant Code Map

- `convex/schema.ts:364`: existing `tagRecognitions` table; use this for Recognized Context when bookmarking resolves a canonical Tag.
- `convex/schema.ts:428`: existing `pinnedKnowledgePages` table; model bookmarks separately, do not overload this table.
- `convex/lib/appAccess.ts`: use `requireAppAccess(ctx)` for current `userId` and allowed Organizations.
- `convex/pinnedKnowledgePages.ts`: good pattern for page keys, auth, Organization membership checks, bounded list queries, and sidebar-ready summaries.
- `convex/pinnedKnowledgePages.test.ts`: good pattern for `convex-test`, seeded users/Organizations, and mutation authorization tests.
- `src/App.tsx:1134`: current durable pin query; add bookmark queries nearby only where needed.
- `src/App.tsx:1704`: avatar menu already links to `/profile?section=bookmarks`.
- `src/App.tsx:2385`: `OrganizationPage` already computes `profile`, active tags, `currentPageKey`, and Pin / Unpin state.
- `src/App.tsx:3324`: profile `Bookmarks` section currently contains a durable bookmarks placeholder.
- `src/App.integrated.test.tsx:50`: current app mock state includes durable pins; extend it for bookmarks.
- `src/App.integrated.test.tsx:716`: existing avatar menu test asserts the Bookmarks shortcut.
- `src/App.integrated.test.tsx:727`: existing durable pin toggle test; add a sibling bookmark toggle test without breaking pin expectations.
- `src/index.css:1985`: profile panel styles.
- `src/index.css:1514`: Organization Pin / Unpin button styles; add separate Bookmark control styling without making the hero feel crowded.

Suggested Data Model

Add a table such as `bookmarkedKnowledgePages` in `convex/schema.ts`.

Suggested fields:

- `userId: v.id("users")`
- `pageKey: v.string()`; stable unique key such as `organization:<referentId>`
- `pageKind: v.union(v.literal("organization"))` for this slice
- `organizationReferentId: v.optional(v.id("referents"))`
- `targetReferentId: v.optional(v.id("referents"))`
- `targetTagId: v.optional(v.id("tags"))`
- `labelSnapshot: v.string()`
- `hrefSnapshot: v.string()`
- `secondaryLabelSnapshot: v.optional(v.string())`
- `createdAt: v.number()`

- `updatedAt: v.number()`
- `lastReferencedAt: v.optional(v.number())`

Suggested indexes:

- `by_userId_and_pageKey`
- `by_userId_and_createdAt`
- `by_userId_and_updatedAt`
- optionally `by_userId_and_pageKind_and_updatedAt`

The exact shape can differ if a cleaner local pattern emerges, but preserve these capabilities:

- unique current-user/page relationship,
- bounded current-user listing,
- enough target metadata for profile display without expensive joins,
- enough target metadata for future recommendation logic,
- separation from pins and subscriptions.

Do not store an unbounded array of all future context tags on the bookmark row. For this slice, a single Organization target tag is enough. Broader Context Page bookmarks can add a separate child table later if needed.

Suggested Convex API

Create a focused module such as `convex/bookmarkedKnowledgePages.ts`.

Suggested public functions:

- `listForProfile`: query, no args or optional bounded `limit`. Requires app access. Returns bookmark summaries newest or recently referenced first.
- `getForPage`: query, args `{ pageKey: v.string() }`. Requires app access. Returns the current user's bookmark summary for that page or `null`.
- `bookmarkOrganizationPage`: mutation, args `{ organizationReferentId: v.id("referents") }`. Requires app access and active membership/access to that Organization. Upserts the bookmark relationship.
- `removeBookmark`: mutation, args `{ pageKey: v.string() }`. Requires app access. Removes only the current user's bookmark row for that page.

Implementation notes:

- Do not accept `userId` from the client.
- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Use indexes, not query filters.
- Keep query results bounded.
- Reuse page identity conventions from pins, e.g. `organization:${organizationReferentId}` and `/organizations/${organizationReferentId}`.
- To resolve Organization metadata, use the current user's `access.organizations` first. If a bookmark points at an old Organization the user no longer has access to, it is acceptable in this slice to omit it from `listForProfile` or return the snapshot only, but document the choice in code comments or final notes.
- For Recognized Context, when bookmarking an Organization page:
  - find a Tag for `organizationReferentId` via `tags.by_referentId`,

- upsert a user `tagRecognitions` row via `by_userId_and_tagId`,
- set `recognizerKind: "user"`, `userId`, `recognizedAt`, and `lastInteractedAt`,
- update `lastInteractedAt` on repeat bookmark,
- do not remove tag recognition when the bookmark is removed, because Recognized Context is historical.

Frontend Guidance

- Keep the profile route as the home for bookmarks. Do not create a new sidebar item or standalone Bookmarks User View in this slice.
- Keep the avatar menu `Bookmarks` link as `/profile?section=bookmarks`.
- If the app does not currently scroll/focus to the `#bookmarks` section for `?section=bookmarks`, add a small effect in `ProfilePage` or route handling so the shortcut feels intentional.
- Replace the profile Bookmarks placeholder with a real list backed by `api.bookmarkedKnowledgePages.listForProfile`.
- Show a calm empty state when no bookmarks exist.
- Each bookmark row/card should link to the Knowledge Page and include enough metadata to distinguish it, such as `Organization` or `School`.
- Add a remove action in the profile Bookmark row if it can be done without clutter. Use an icon button plus accessible label.
- Add a Bookmark / Remove Bookmark control on Organization Knowledge Pages, separate from Pin / Unpin. The two controls have different meanings:
  - Pin: quick return through sidebar.
  - Bookmark: save for later on the profile and for personal knowledge context.
- Keep both controls compact. Avoid adding explanatory in-app copy that teaches the distinction at length.
- Do not add notification language to bookmark UI.

Test Plan

Use TDD where practical.

Convex tests:

1. Add `convex/bookmarkedKnowledgePages.test.ts`.
2. Use the same seeded user/Organization helpers or patterns from `convex/pinnedKnowledgePages.test.ts`.
3. Assert `listForProfile` returns an empty list when the user has no bookmarks.
4. Assert `bookmarkOrganizationPage` creates one bookmark row for an accessible Organization Knowledge Page.
5. Assert calling `bookmarkOrganizationPage` twice does not create duplicates and updates the existing row.
6. Assert `getForPage` returns the current user's bookmark for that page.
7. Assert `removeBookmark` removes the bookmark for the current user only.
8. Assert unauthorized, inactive, and no-organization users cannot bookmark Organization pages.
9. Assert bookmarking an Organization page upserts a user `tagRecognitions` row when the Organization Tag exists.

Frontend tests:

1. Extend `src/App.integrated.test.tsx` mocks to support bookmark queries/mutations.
2. Assert the profile Bookmarks section renders durable bookmark data with links.
3. Assert the avatar menu Bookmarks shortcut still points to `/profile?section=bookmarks`.
4. Assert the Organization page Bookmark / Remove Bookmark control calls the correct mutation and toggles after mock state changes.
5. Assert bookmarked pages do not appear in `Knowledge Page destinations` unless they are also pinned.
6. Assert existing durable pin tests still pass.

Verification Commands

- `npx convex codegen`
- `npx vitest run convex/bookmarkedKnowledgePages.test.ts`
- `npx vitest run src/App.integrated.test.tsx`
- `npx vitest run convex/bookmarkedKnowledgePages.test.ts src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`

Manual browser check if a signed-in local session is available:

- Start Vite on an open port.
- Visit an Organization Knowledge Page.
- Bookmark it.
- Open the avatar menu and choose Bookmarks.
- Confirm the profile Bookmarks section lists the page.
- Remove it and confirm it disappears from the profile section and does not affect sidebar pins.

If the browser only reaches the unauthenticated sign-in screen, say so and rely on the automated tests for signed-in behavior.

Risks / Open Questions

- This slice supports Organization Knowledge Pages first. Broader Knowledge Page bookmarks, including Scripture, Explore contexts, and individual Knowledge Entries, should be separate follow-up slices.
- The app has not resolved a large saved-set management experience. Keep profile rendering simple and bounded.
- Bookmark-powered recommendations and LLM behavior are not part of this slice. Persisting/querying bookmarks and updating user tag recognition is enough to make later logic possible.
- If an Organization bookmark remains after the user loses access to that Organization, the product behavior is not fully specified. Prefer a conservative implementation: do not leak current Organization data beyond what `requireAppAccess` allows, and use snapshots only if the existing app already treats those snapshots as safe.
- `docs/*` may be ignored by `.gitignore`; if this handoff needs to be committed, force-add it intentionally.

Expected Final Response From Coding Agent

Summarize:

1. What durable bookmark backend and frontend behavior changed.

2. Whether tag recognition was updated for bookmark actions.
3. What tests/checks passed.
4. Any generated files or codegen steps.
5. What remains for broader Knowledge Page bookmark targets, recommendations, subscriptions, and saved-set management.

Code Handoff: Durable Direct Contributions To Gold Entries

Coding Agent Prompt

You are implementing the next vertical slice after durable pins, bookmarks, subscriptions, and notification inbox.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/handoffs/durable-notification-inbox.md`
- `src/App.tsx`
- `src/ContributionEditor.tsx`
- `src/ContributionEditor.test.tsx`
- `src/knowledgeContracts.ts`
- `src/App.integrated.test.tsx`
- `convex/schema.ts`
- `convex/answerFeed.ts`
- `convex/answerFeed.test.ts`
- `convex/smartStorage.ts`
- `convex/smartStorage.test.ts`

Do not re-litigate the resolved domain language unless code and docs directly contradict each other. Keep this slice focused on direct-post Gold Knowledge Entry creation and durable Answer Feed visibility; do not build notification generation, Smart Storage proposal acceptance, full type-specific editors, or delivery preferences.

What To Do

Persist direct Contribution Editor submissions as Gold Layer `KnowledgeEntry` rows and have the Answer Feed read them back from Convex.

Implement a narrow durable direct-post path:

- a Convex mutation for posting a direct Contribution as a Knowledge Entry,
- creation or reuse of the represented Referent and primary Tag,
- creation or reuse of context Tags from the active Knowledge Context,
- `entryTags` rows for the represented Tag and context Tags,
- Answer Feed rendering from durable Convex data for active Knowledge Contexts,
- removal of the runtime dependency on simulated local direct-post feed items for the normal direct-post path.

Target type: inferred prerequisite vertical slice.

Why This Target

The notification inbox is now durable, but notification generation needs a real domain event to react to. The app still treats direct posts as local deterministic feed items in `src/App.tsx`, while Smart Storage persists only Bronze/Silver state and does not create Gold entries yet.

This slice creates the missing Gold-layer event source: when a user chooses the direct post path, the app creates the Knowledge Entry currently shown in the Contribution Preview. Later slices can generate Subscription Notifications from these durable Knowledge Entry creations.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Durable notification handoff: `docs/handoffs/durable-notification-inbox.md`
- Convex guidance: `convex/_generated/ai/guidelines.md`
- Contribution UI: `src/ContributionEditor.tsx`, `src/ContributionEditor.test.tsx`
- Contribution contracts: `src/knowledgeContracts.ts`
- Current local direct-post behavior: `src/App.tsx`
- Current Answer Feed backend adapter: `convex/answerFeed.ts`, `convex/answerFeed.test.ts`
- Smart Storage backend, for contrast only: `convex/smartStorage.ts`, `convex/smartStorage.test.ts`
- Current schema: `convex/schema.ts`
- Relevant ADRs: `docs/adr/0001-separate-referents-tags-and-knowledge-entries.md`, `docs/adr/0002-use-type-detail-tables-for-knowledge-entries.md`, `docs/adr/0004-knowledge-entries-uniquely-represent-same-typed-referents.md`, `docs/adr/0006-store-smart-storage-proposals-as-durable-silver-records.md`, `docs/adr/0007-store-smart-storage-proposals-as-persistence-agnostic-domain-contracts.md`
- Issue docs: no issue found for this slice; inline issue brief below.
- Prototype: no separate durable direct-post prototype found; current local direct-post UI is the implementation reference.

Inline Issue Brief

What To Build

Make direct posting durable:

1. A signed-in user posts directly from a tagged Knowledge Context.
2. Convex creates a Gold Layer `KnowledgeEntry` with a represented Referent, primary Tag, and context Tags.
3. The created entry appears in the Answer Feed from durable Convex data.
4. The same entry can still be focused after submission in the current UI.
5. Smart Storage remains Bronze/Silver only and does not create Gold entries in this slice.

Acceptance Criteria

- [ ] A signed-in allowed user can create a direct Knowledge Entry through a Convex mutation.
- [ ] The mutation derives the current user server-side; no client function accepts `userId` for authorization.
- [ ] The mutation creates or reuses the represented Referent and primary Tag for the entry.
- [ ] The created `KnowledgeEntry` uses the selected authorable Knowledge Type, title, preview text, search text, primary Tag label, created user, and timestamps.
- [ ] The created `KnowledgeEntry` has an `entryTags` row with `tagPurpose: "represented"` for its primary Tag.
- [ ] The created `KnowledgeEntry` has `entryTags` rows with `tagPurpose: "context"` for every active context Tag.
- [ ] The mutation resolves existing Tags by stable lookup key and Knowledge Type before creating new context Referents/Tags.
- [ ] The mutation keeps direct posting distinct from Smart Storage: no Bronze `Source`, `smartStorageRun`, or `smartStorageProposal` is created for direct posts.
- [ ] The normal direct-post path in `src/App.tsx` calls the durable mutation instead of only creating a local simulated item.
- [ ] The Answer Feed can list durable entries for the active Knowledge Context.
- [ ] A newly posted direct entry appears in the Answer Feed after the mutation and is available after re-render/requery.
- [ ] Existing Smart Storage behavior still stores Bronze/Silver proposal state without creating Answer Feed items.
- [ ] Existing durable pin, bookmark, subscription, and notification inbox tests still pass.

Out Of Scope

- Smart Storage Proposal acceptance into Gold.
- Subscription notification generation from new Knowledge Entries.
- Delivery channels, notification preferences, or muting.
- Full type-specific detail-row persistence for every Knowledge Type.
- Rich identity resolution UI for ambiguous existing Referents.
- Full editor for entry representations beyond the current preview/body fields.
- General-purpose Knowledge Entry edit/delete flows.
- Exact large-scale counters or ranking changes for Answer Feed.

Domain Language

- `Contribute`: to add a future Answer to the knowledgebase, either by submitting a Source, creating a Knowledge Entry directly, or responding to an existing Knowledge Entry.
- `Knowledge Entry`: a typed, contextualized unit of knowledge that represents one same-typed Referent and whose Knowledge Context is constituted by its Tags.
- `Represented Referent`: the same-typed Referent a Knowledge Entry uniquely expresses or records.
- `Entry Tag`: the relationship between a Knowledge Entry and a Tag. Use `represented` for the entry's primary Tag and `context` for the surrounding Knowledge Context.
- `Smart Storage`: the AI-assisted Bronze/Silver path that may propose Gold entries but does not write Gold until user confirmation.

- `Gold Layer`: confirmed Knowledge Entries represented according to the current application model.

Avoid calling a direct post a `Source`, `Smart Storage Proposal`, or `Notification`.

Decisions That Must Hold

- Direct posting creates the Knowledge Entry currently displayed in the deterministic Contribution Preview.
- Direct posting should not create a Bronze Source by default.
- Smart Storage should preserve raw Source material and produce durable Silver Proposals, but it should not create Gold entries without user confirmation.
- A Knowledge Entry represents exactly one same-typed Referent.
- A Referent may have at most one Knowledge Entry representing it.
- A Knowledge Entry's canonical represented Tag should be included among its `entryTags`.
- Contributed Questions create Question Knowledge Entries and represented Question Tags, but they should not automatically move the user into that Question's Knowledge Context.
- Plain page visits should not add to Recognized Context. Contributing is a meaningful action.
- Notification generation is not part of this slice.

Relevant Code Map

- `src/App.tsx:2985`: `ComponentScaffold` currently keeps local `feedItems` state from `ANSWER_FEED_FIXTURE`.
- `src/App.tsx:3050`: `handleSubmitContribution` currently creates a deterministic local feed item.
- `src/App.tsx:3076`: `handleStoreSmartlyContribution` already calls durable Smart Storage mutations and should remain separate.
- `src/App.tsx:3185`: `ContributionEditorSurface` receives `onPostDirect` and `onStoreSmartly`.
- `src/App.tsx:3206`: `AnswerFeedSurface` currently receives local `feedItems`.
- `src/App.tsx:3831`: `createDeterministicContributionFeedItem` is the local direct-post simulation to replace for normal runtime.
- `src/ContributionEditor.tsx:140`: contribution submission builds a `ContributionInput` with title, body, Knowledge Type, context Tags, and optional slot.
- `src/knowledgeContracts.ts:139`: `ContributionInput` shape.
- `convex/schema.ts:321`: `knowledgeEntries` table.
- `convex/schema.ts:370`: `entryTags` table.
- `convex/answerFeed.ts:81`: `listForActiveTags` expects Convex Tag IDs.
- `convex/answerFeed.ts:117`: Answer Feed matching uses `entryTags`.
- `convex/answerFeed.test.ts`: good seed pattern for `knowledgeEntries`, `referents`, `tags`, and `entryTags`.
- `convex/smartStorage.ts:128`: Smart Storage contribution path, useful contrast for what direct posting should not do.

Suggested Convex API

Create a module such as `convex/directContributions.ts`.

Suggested public mutation:

- `postDirectContribution`

Suggested args:

- `knowledgeType`
- `title`
- `body`
- `contextTags`: array of active tag snapshots from `ContributionInput`
- optional `slotId`

Suggested return:

- `entry`: an Answer Feed-compatible `KnowledgeEntrySummary`
- `entryId`
- `primaryTagId`
- `representedReferentId`

Implementation notes:

- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Limit title/body/context fields similarly to `convex/smartStorage.ts`.
- Resolve context Tags by `knowledgeType` + canonical lookup key before creating missing context Tags.
- Resolve or create the represented Referent/primary Tag for the new entry. If a same-typed represented Referent already has a `KnowledgeEntry`, do not silently create a duplicate. For this MVP slice, returning a clear conflict error is acceptable.
- For user-authored direct posts where identity is inherently new, a stable generated canonical key based on Knowledge Type, normalized title, creator, and timestamp is acceptable if there is no better local identity pattern. Keep this choice explicit in code and tests.
- Insert `entryTags` for represented and context Tags.
- Set `visibilityKind` and `discoverabilityKind` conservatively from current app behavior. If no scoped visibility decision exists in code, use the existing public/default patterns already used by Answer Feed test fixtures and seed data.
- Use a modest default `humanWeight` consistent with the current local deterministic direct-post path unless a better existing constant/pattern exists.
- If a `slotId` is supplied and maps to a real `knowledgeSlots` row, fulfillment can be a later slice. Do not block direct posting on slot fulfillment unless the code already has the seam.

Answer Feed Guidance

- The frontend currently has active Tags as route snapshots, not necessarily Convex `Id<"tags">` values.
- Add an Answer Feed query seam that accepts active Tag snapshots or stable active tag keys and resolves them to Convex Tag IDs server-side, or add a small frontend/backend resolver before calling `api.answerFeed.listForActiveTags`.
- Prefer a new query such as `api.answerFeed.listForActiveTagKeys` or `api.answerFeed.listForActiveTagsByLookupKey` over forcing route state to carry Convex IDs everywhere.
- Keep existing `listForActiveTags` for tests and internal callers if it is still useful.

- If live query data is loading, use a stable loading state or combine with existing fixture data only where the app already intentionally uses fixtures. Do not show a direct-post local item as if it were durable after this slice.

Frontend Guidance

- In `ComponentScaffold`, replace normal `handleSubmitContribution` simulation with a call to the new durable direct-post mutation.
- After mutation success, focus the returned created entry in the existing `CreatedEntryFocusPanel`.
- Ensure the Answer Feed re-renders from durable data so the entry remains visible after re-render/requery.
- Keep `handleStoreSmartlyContribution` as the Bronze/Silver path and preserve its proposal review panel behavior.
- Keep the Contribution Editor mode rules already implemented:
  - Global Knowledge Context defaults to Smart Storage.
  - Tagged contexts default to direct post.
  - Comments are titleless.
  - Questions use question text as title with optional details.
  - RSVP remains hidden from generic picker.
  - Guided direct Group creation can remain as-is if full Group detail persistence is too large for this slice; do not break the existing UI.
- If some world-modeling types need detail rows that are not yet supported, prefer a clear narrow scope and tests over a partial broken generic implementation.

Test Plan

Use TDD where practical.

Convex tests:

1. Add `convex/directContributions.test.ts`.
2. Seed an allowed user and at least two context Tags.
3. Assert `postDirectContribution` creates a `knowledgeEntries` row for a direct Words or Question contribution.
4. Assert it creates/reuses the represented Referent and primary Tag.
5. Assert it creates represented and context `entryTags`.
6. Assert the created entry appears from `api.answerFeed.listForActiveTags` or the new lookup-key query for the same context.
7. Assert direct posting does not create `sources`, `smartStorageRuns`, or `smartStorageProposals`.
8. Assert unauthorized, inactive, and no-organization users cannot post direct contributions.
9. Assert duplicate represented Referent conflicts are handled intentionally.

Frontend tests:

1. Extend `src/App.integrated.test.tsx` mocks for the new direct-post mutation and live Answer Feed query.
2. Assert tagged-context direct posting calls the durable mutation.
3. Assert a returned entry appears in the Answer Feed and Created Entry focus panel.

4. Assert a re-render/requery still shows the created entry from durable mock data.
5. Assert Store Smartly still calls Smart Storage mutations and does not create an Answer Feed item.
6. Assert global context still defaults to Smart Storage and tagged contexts still default to direct posting.

Verification Commands

- `npx convex codegen`
- `npx vitest run convex/directContributions.test.ts`
- `npx vitest run convex/answerFeed.test.ts`
- `npx vitest run src/ContributionEditor.test.tsx`
- `npx vitest run src/App.integrated.test.tsx`
- `npx vitest run convex/directContributions.test.ts convex/answerFeed.test.ts src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`

Manual browser check if a signed-in local session is available:

- Start Vite on an open port.
- Visit a tagged Knowledge Context.
- Post directly.
- Confirm the created entry appears in the Answer Feed and focus panel.
- Refresh/revisit the same context and confirm the entry still appears.
- Confirm Store Smartly still produces the Smart Storage Proposal review path rather than a direct Answer Feed entry.

If the browser only reaches the unauthenticated sign-in screen, say so and rely on automated tests for signed-in behavior.

Risks / Open Questions

- The app has many Knowledge Types, but this slice should not become a full type-detail persistence project. Start with the generic fields needed by `knowledgeEntries` and only add type-detail rows where the existing schema and UI make the mapping obvious.
- Identity resolution is intentionally narrow. Rich candidate matching and user confirmation belong in a later slice.
- Direct post persistence enables future notification generation, but this slice should not create Notification rows yet.
- Smart Storage Proposal acceptance remains future work.
- `docs/*` may be ignored by `.gitignore`; if this handoff needs to be committed, force-add it intentionally.

Expected Final Response From Coding Agent

Summarize:

1. What durable direct-post backend and frontend behavior changed.
2. How represented Referents, primary Tags, and context Tags are created or reused.
3. What tests/checks passed.
4. Any generated files or codegen steps.

5. What remains for notification generation, Smart Storage Proposal acceptance, type-detail persistence, and identity-resolution UI.

Code Handoff: Durable Notification Inbox

Coding Agent Prompt

You are implementing the next vertical slice after durable pins, bookmarks, and subscriptions.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/handoffs/header-sidebar-navigation.md`
- `docs/handoffs/durable-pinned-knowledge-pages.md`
- `docs/handoffs/durable-bookmarked-knowledge-pages.md`
- `docs/handoffs/durable-subscriptions-notification-readiness.md`
- `src/App.tsx`
- `src/App.integrated.test.tsx`
- `src/index.css`
- `convex/schema.ts`
- `convex/lib/appAccess.ts`
- `convex/knowledgeSubscriptions.ts`
- `convex/knowledgeSubscriptions.test.ts`

Do not re-litigate the resolved navigation language unless code and docs directly contradict each other. Keep this slice focused on durable notification inbox rows and read/unread state; do not build notification generation, delivery channels, notification preferences, muting, or broader subscription targets.

What To Do

Replace the static Notifications feed source with a durable current-user Notification inbox.

Implement durable user-specific notifications with:

- a Convex table and APIs for listing current-user inbox notifications,
- durable read/unread status,
- notification summary/filter counts and sidebar badge derived from durable rows,
- a mark read/unread action from the Notifications page,
- current static notification fixtures moved into tests or seed helpers as needed,
- no automatic notification generation from subscriptions yet.

Target type: inferred next vertical slice.

Why This Target

The app now has durable `Subscription` sources, but the actual notification cards are still hard-coded in `src/App.tsx` as `USER_NOTIFICATIONS`. That means unread count, notification filters, and read state cannot persist.

This slice makes the Notifications User View real without taking on the larger system that decides when activity should create notifications. It preserves the distinction:

- `Subscription`: a standing relationship that may affect notification behavior.
- `Notification`: a concrete user-visible notice that something relevant occurred.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Previous shell handoff: `docs/handoffs/header-sidebar-navigation.md`
- Previous pin handoff: `docs/handoffs/durable-pinned-knowledge-pages.md`
- Previous bookmark handoff: `docs/handoffs/durable-bookmarked-knowledge-pages.md`
- Previous subscription handoff: `docs/handoffs/durable-subscriptions-notification-readiness.md`
- Convex guidance: `convex/_generated/ai/guidelines.md`
- Current app access helper: `convex/lib/appAccess.ts`
- Current subscription implementation pattern: `convex/knowledgeSubscriptions.ts`, `convex/knowledgeSubscriptions.test.ts`
- Current schema: `convex/schema.ts`
- Current Notifications page: `src/App.tsx`, `src/index.css`
- Existing integrated tests: `src/App.integrated.test.tsx`
- Relevant ADRs: no notification-inbox-specific ADR found.
- Issue docs: no issue found for this slice; inline issue brief below.
- Prototype: no notification inbox prototype found; current static Notifications page is the implementation reference.

Inline Issue Brief

What To Build

Add a persisted Notification inbox for the current user and wire the existing Notifications User View to it:

1. A signed-in user can see durable Notifications in the Notifications page.
2. The sidebar Notifications badge reflects durable unread notifications.
3. The Notifications page summary and filters use durable notification rows.
4. The user can mark a notification read and the state persists.
5. Durable subscription sources remain visible but do not themselves count as notification rows.

Acceptance Criteria

- [ ] A signed-in allowed user can query their recent inbox Notifications from Convex.

- [ ] Notification queries derive the current user server-side; no client function accepts `userId` for authorization.
- [ ] The query returns a bounded recent inbox and summary data needed by the UI.
- [ ] A Notification row includes title, body, kind, status, received time, context label, and context href.
- [ ] The Notifications page renders durable notification rows instead of `USER_NOTIFICATIONS`.
- [ ] The sidebar Notifications badge is derived from durable unread Notification rows, not static data and not active Subscription count.
- [ ] Filters for All, Unread, Knowledge Slots, and Events work against durable rows.
- [ ] Existing Subscription-kind notifications still show in All/Unread if present, even if there is no separate Subscription filter.
- [ ] The user can mark an unread Notification as read.
- [ ] Marking read updates the row only for the current user.
- [ ] After marking read, the unread badge/count and Unread filter update.
- [ ] Opening or reading a Notification does not unsubscribe from any Subscription.
- [ ] Active Subscription sources remain a separate Notifications-page panel and are not counted as notifications.
- [ ] Existing durable pin, bookmark, and subscription tests still pass.

Out Of Scope

- Generating notifications from subscription activity.
- Generating notifications from Knowledge Slot assignment or Event participation.
- Delivery channels such as email, push, SMS, or digests.
- Notification preferences, muting, quiet hours, or frequency controls.
- Exact unbounded unread counters if the implementation uses a bounded inbox query for MVP.
- Multi-user/admin notification broadcast tooling.
- Broader subscription targets.

Domain Language

- `Notification`: a user-visible notice that relevant activity occurred within a Subscription, assigned Knowledge Slot, or Event participation.
- `Subscription`: a user's standing interest in activity within a Knowledge Context, Organization, Knowledge Slot, or Event.
- `User View`: a current-user scoped view such as Calendar, Notifications, Settings, or editing the user's own profile.
- `Knowledge Page`: a shared, world-facing location grounded in a Knowledge Context, Referent, Knowledge Entry, Organization, or other knowledge object.

Avoid `message` and `feed item` as canonical domain terms. Do not call a Subscription row a Notification.

Decisions That Must Hold

- Notification is not a Knowledge Type.
- Notifications are reactions to existing Knowledge Entries, Knowledge Slots, Events, Subscriptions, or system opportunities.

- Subscription is separate from Notification.
- The Notifications route is a User View and should not show the Knowledge Navigator as its main navigator.
- Notifications remain a visible bottom sidebar User View icon because unread status needs a persistent badge.
- The badge means unread Notifications, not active Subscriptions.
- Subscription sources on the Notifications page remain separate from notification cards.

Relevant Code Map

- `src/App.tsx:207`: current `NotificationFilter`, `NotificationKind`, and `NotificationStatus` types.
- `src/App.tsx:555`: current static `USER_NOTIFICATIONS` fixtures; replace as runtime source.
- `src/App.tsx:1561`: sidebar currently gets unread count from `getUnreadNotificationCount()`.
- `src/App.tsx:2941`: `getUnreadNotificationCount()` currently counts static rows.
- `src/App.tsx:3892`: `NotificationsPage` currently queries subscription sources but filters static notification rows.
- `src/App.tsx:3907`: `filteredNotifications` currently comes from `USER_NOTIFICATIONS`.
- `src/App.tsx:3933`: notification summary cards.
- `src/App.tsx:3958`: Subscription Sources panel; keep it separate.
- `src/App.tsx:4012`: Notification Feed panel and filter controls.
- `src/App.integrated.test.tsx:1407`: current static notification-route behavior test.
- `src/App.integrated.test.tsx:1440`: current durable subscription sources test.
- `src/index.css:2885`: Notifications page styles.
- `convex/schema.ts:475`: existing `knowledgeSubscriptions`; do not overload this table for notification rows.
- `convex/knowledgeSubscriptions.ts`: current relationship API pattern and auth/access style.

Suggested Data Model

Add a table such as `userNotifications` in `convex/schema.ts`.

Suggested validators:

- `notificationKind`: `answer | event | knowledgeSlot | subscription`
- `notificationStatus`: `read | unread`
- optionally `notificationSourceKind`: `subscription | knowledgeSlot | event | system`

Suggested fields:

- `userId: v.id("users")`
- `notificationKind`
- `notificationStatus`
- `title: v.string()`
- `body: v.string()`
- `contextLabel: v.string()`
- `contextHref: v.string()`
- `receivedAt: v.number()`

- `readAt: v.optional(v.number())`
- `sourceKind: v.optional(notificationSourceKind)`
- `sourceSubscriptionKey: v.optional(v.string())`
- `sourceSubscriptionId: v.optional(v.id("knowledgeSubscriptions"))`
- `targetReferentId: v.optional(v.id("referents"))`
- `targetTagId: v.optional(v.id("tags"))`
- `createdAt: v.number()`
- `updatedAt: v.number()`

Suggested indexes:

- `by_userId_and_receivedAt`
- `by_userId_and_notificationStatus_and_receivedAt`
- `by_userId_and_notificationKind_and_receivedAt`
- `by_sourceSubscriptionKey_and_receivedAt`

The exact shape can differ if a cleaner local pattern emerges, but preserve these capabilities:

- current-user inbox listing,
- durable read/unread state,
- efficient bounded filtering by status and kind,
- future notification generation from Subscription targets,
- separation from Subscription rows themselves.

Suggested Convex API

Create a focused module such as `convex/userNotifications.ts`.

Suggested public functions:

- `listForInbox`: query, args `{ limit?: number }`. Requires app access. Returns recent notifications plus summary counts for the returned inbox window.
- `getUnreadSummary`: optional query if the sidebar needs a smaller payload than the full inbox. Requires app access. Returns unread count and latest received timestamp from a bounded recent window.
- `markRead`: mutation, args `{ notificationId: v.id("userNotifications") }`. Requires app access and verifies row ownership.
- `markUnread`: optional mutation, same ownership check.

Implementation notes:

- Do not accept `userId` from the client.
- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Use indexes, not query filters.
- Keep query results bounded.
- If exact counts beyond a bounded inbox become necessary later, add a denormalized counter table in a future slice rather than scanning every notification row.

- For this slice, tests may insert notification rows directly with `convex-test` or use an internal helper. Do not build a public notification-creation UI.
- If you add an internal creation helper for future generators, register it as `internalMutation`, not public `mutation`, unless there is a clear current UI need.

Frontend Guidance

- Replace runtime use of `USER_NOTIFICATIONS` with `api.userNotifications.listForInbox` data.
- Keep static notification fixture data only in tests or a seed helper if needed. It should not remain the live app source of truth.
- While the query is loading, show a stable loading state rather than falling back to static rows as if they are real.
- If the query returns an empty inbox, show the existing calm empty state.
- Derive filter tabs and summary counts from durable query results.
- Derive the sidebar unread badge from a durable unread summary or the inbox query data. It must not count active subscription sources.
- Add a compact action on each notification card to mark read when unread. If adding mark-unread for read notifications is small and fits the UI, it is acceptable but not required.
- Keep the existing `Open` action. Opening a Knowledge Page may also mark read if implemented carefully, but an explicit Mark read action is enough for this slice.
- Keep the Subscription Sources panel in `NotificationsPage`. Do not merge it into the Notification Feed.
- Do not add delivery preference controls.

Test Plan

Use TDD where practical.

Convex tests:

1. Add `convex/userNotifications.test.ts`.
2. Seed an allowed user and insert a few notification rows for that user.
3. Assert `listForInbox` returns only the current user's notifications, sorted newest first.
4. Assert `listForInbox` or `getUnreadSummary` reports unread count from durable rows.
5. Assert status and kind filters are backed by indexed queries or bounded query shaping.
6. Assert `markRead` changes an unread notification to read and sets `readAt`.
7. Assert one user cannot mark another user's notification read.
8. Assert unauthenticated, inactive, and no-organization users cannot read or mutate inbox notifications.

Frontend tests:

1. Extend `src/App.integrated.test.tsx` mocks to support durable notification queries/mutations.
2. Assert the sidebar Notifications badge reflects mocked durable unread rows.
3. Assert `/notifications` renders durable notification rows and summary counts.
4. Assert All, Unread, Knowledge Slots, and Events filters work with durable rows.
5. Assert marking a notification read calls the correct mutation and updates the badge/filter result after mock state changes.

6. Assert Subscription Sources still render separately and do not affect unread count.
7. Assert existing pin, bookmark, and subscription integrated tests still pass.

Verification Commands

- `npx convex codegen`
- `npx vitest run convex/userNotifications.test.ts`
- `npx vitest run src/App.integrated.test.tsx`
- `npx vitest run convex/pinnedKnowledgePages.test.ts convex/bookmarkedKnowledgePages.test.ts convex/knowledgeSubscriptions.test.ts convex/userNotifications.test.ts`
- `npx vitest run convex/userNotifications.test.ts src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`

Manual browser check if a signed-in local session is available:

- Start Vite on an open port.
- Visit `/notifications`.
- Confirm durable notification rows render.
- Mark one unread notification read.
- Confirm the unread summary and sidebar badge update.
- Confirm Subscription Sources remain visible and unchanged.

If the browser only reaches the unauthenticated sign-in screen, say so and rely on the automated tests for signed-in behavior.

Risks / Open Questions

- This slice does not decide when notifications are generated. That should be a separate workflow/generator slice.
- Exact unread counts across an unbounded inbox may require a denormalized counter later. A bounded MVP inbox summary is acceptable for this slice unless the implementation already has a cheap counter pattern.
- Existing static notification fixtures may disappear from runtime unless seeded as durable rows. Prefer tests or seed helpers over keeping static runtime source-of-truth arrays.
- Notification preferences, muting, and delivery channels are unresolved.
- `docs/*` may be ignored by `.gitignore`; if this handoff needs to be committed, force-add it intentionally.

Expected Final Response From Coding Agent

Summarize:

1. What durable notification inbox backend and frontend behavior changed.
2. How unread counts and read state are persisted.
3. What tests/checks passed.
4. Any generated files or codegen steps.
5. What remains for notification generation, delivery preferences, muting, and exact large-scale counters.

Code Handoff: Durable Pinned Knowledge Pages

Coding Agent Prompt

You are implementing the next vertical slice after the header/sidebar shell work.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/handoffs/header-sidebar-navigation.md`
- `src/App.tsx`
- `src/App.integrated.test.tsx`
- `convex/schema.ts`
- `convex/lib/appAccess.ts`

Do not re-litigate the resolved navigation language unless code and docs directly contradict each other. Keep this slice focused on durable Pinned Knowledge Pages; do not build bookmarks, subscriptions, full Global Search, or Active Role persistence.

What To Do

Persist the sidebar's Pinned Knowledge Pages instead of deriving them only from local frontend state.

Implement durable user-specific pins for Knowledge Pages with:

- default Organization Knowledge Page seeding from the current user's active Organization memberships,
- at most one default seeded Organization pin per Organization kind,
- persistent unpin suppression for default seeded pins,
- ability to manually pin/unpin Organization Knowledge Pages,
- sidebar rendering from the durable pin query while preserving the shell layout implemented in the previous slice.

Target type: inferred next vertical slice.

Why This Target

The completed shell slice made the sidebar grammar visible, but it still uses `getSeededPinnedKnowledgePages(appAccess.organizations)` in `src/App.tsx`. That means pins are deterministic UI state only: default pins cannot be truly unpinned, user pin order cannot persist, and future manual pins have nowhere durable to live.

This slice turns the visible shell into real user state while staying narrow. It should support Organization Knowledge Pages first because those are the default pins already visible in the sidebar and they are available through current `AllowedAppAccess.organizations`.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Previous shell handoff: `docs/handoffs/header-sidebar-navigation.md`
- Existing frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Convex guidance: `convex/_generated/ai/guidelines.md`
- App access helper: `convex/lib/appAccess.ts`
- Current schema: `convex/schema.ts`
- Current shell implementation: `src/App.tsx`, `src/index.css`
- Existing integrated tests: `src/App.integrated.test.tsx`
- Existing Convex access tests: `convex/appAccess.test.ts`
- Relevant ADRs: No pin/bookmark/subscription-specific ADR found.
- Issue docs: No issue found for this slice; inline issue brief below.
- Prototype: No durable pins prototype found; current shell behavior is the implementation reference.

Inline Issue Brief

What To Build

Add a persisted Pinned Knowledge Page model and wire the sidebar to it for signed-in users. Default Organization pins should appear automatically, but when a user unpins one, that suppression should survive refresh and future default recalculation until the user pins it again.

Acceptance Criteria

- [ ] A signed-in allowed user can query sidebar Pinned Knowledge Pages from Convex.
- [ ] With no pin records, the query returns default Organization Knowledge Page pins derived from active memberships, capped to at most one Organization per kind.
- [ ] Default pins use specific Organization names as primary labels and include Organization kind metadata.
- [ ] Default pin overflow behavior in the sidebar still works.
- [ ] The user can unpin a default Organization Knowledge Page from the sidebar or Organization page.
- [ ] Unpinning a default Organization Knowledge Page persists suppression and the pin does not return after refresh/requery.
- [ ] The user can pin the same Organization Knowledge Page again and it reappears.
- [ ] User-created/manual pins can persist sort order or insertion order.
- [ ] Mutations derive the current user server-side; no userId is accepted from the client for authorization.
- [ ] Only Knowledge Pages are pin targets. Do not pin raw Knowledge Entries as non-page objects.
- [ ] Bookmarks remain separate from pins and are not shown directly in the primary sidebar.
- [ ] Subscriptions remain separate from pins and are not created by pinning.
- [ ] Existing route rendering and shell navigation tests still pass.

Out Of Scope

- Full pin management screen.
- Drag-and-drop pin ordering.
- Bookmark persistence.
- Subscription persistence.
- Notification persistence.
- Full Global Search implementation.
- Durable Active Role persistence.
- Pinning every possible Knowledge Page type. This slice may support Organization Knowledge Pages first, with a schema shape that can grow to Referent, Context, Entry, and other Knowledge Pages later.

Domain Language

- `Pinned Knowledge Page`: a User relationship to a Knowledge Page that keeps it easy to return to, especially from sidebar navigation. It does not subscribe the User to notifications.
- `Default Pinned Knowledge Page`: a Pinned Knowledge Page automatically seeded from User affiliation with a School, Church, Family, or Community. Unpin suppression persists unless the User manually pins it again.
- `Knowledge Page`: a user-facing location grounded in a Knowledge Context, Referent, Knowledge Entry, Organization, user relationship, or knowledge-oriented view.
- `Bookmark`: a saved Knowledge Page for later reference. It does not place the page in sidebar navigation and does not subscribe the User.
- `Subscription`: a standing interest that affects notification behavior.
- `User View`: current-user scoped view such as Calendar or Notifications; not a pin target for this slice.

Avoid `favorite` as a domain term. Avoid treating `Bookmark` and `Pinning` as the same relationship.

Decisions That Must Hold

- Dashboard is fixed first sidebar navigation and is not a user-removable pin.
- Explore is not a separate primary sidebar icon for now.
- Pinned Knowledge Pages live in the sidebar middle.
- Default Organization pins are seeded from affiliations but can be unpinned.
- If a user unpins a default Organization pin, suppression must persist and the pin must not return just because defaults are recalculated.
- Default seed should be capped; do not pin every affiliation automatically.
- When the user has multiple Organizations of the same kind, seed at most the most relevant one per kind. For this slice, choosing the first active membership per kind is acceptable unless a better local signal already exists.
- Organization pins should show the specific Organization name as primary label, not only `My School` / `My Church`.
- Pinning does not create subscriptions or notifications.
- Bookmarking is not part of this slice.

Relevant Code Map

- `src/App.tsx:1497`: current sidebar calls `getSeededPinnedKnowledgePages(appAccess.organizations)`.
- `src/App.tsx:1537`: sidebar renders `Pinned Knowledge Pages`.
- `src/App.tsx:2679`: temporary `getSeededPinnedKnowledgePages` helper.
- `src/App.tsx:435`: `SIDEBAR_VISIBLE_PIN_LIMIT`.
- `src/App.tsx:1640`: avatar menu; bookmarks stay there/profile, not in pin nav.
- `src/App.tsx:2185` and nearby Organization page code: likely place for Pin/Unpin action.
- `src/App.integrated.test.tsx:459`: current shell test expects seeded Organization pins and `+1 more`.
- `convex/schema.ts`: add the durable pin table here.
- `convex/lib/appAccess.ts`: use `requireAppAccess(ctx)` to authorize pin queries/mutations and get current `userId`/Organizations.
- `convex/appAccess.test.ts`: pattern for `convex-test`, seed data, and authenticated access tests.

Suggested Data Model

Add a table such as `pinnedKnowledgePages` in `convex/schema.ts`.

Suggested fields:

- `userId: v.id("users")`
- `pageKey: v.string()`; stable unique key such as `organization:<referentId>`
- `pageKind: v.union(v.literal("organization"))` for this slice
- `pinState: v.union(v.literal("pinned"), v.literal("suppressed"))`
- `pinSource: v.union(v.literal("defaultSeed"), v.literal("manual"))`
- `sortOrder: v.number()`
- `organizationReferentId: v.optional(v.id("referents"))`
- `organizationKind: v.optional(organizationKind)`
- `labelSnapshot: v.string()`
- `hrefSnapshot: v.string()`
- `createdAt: v.number()`
- `updatedAt: v.number()`

Suggested indexes:

- `by_userId_and_pageKey`
- `by_userId_and_pinState_and_sortOrder`
- `by_userId_and_pinSource`

The exact shape can differ if a cleaner local pattern emerges, but preserve these capabilities:

- unique user/page relationship,
- suppression records for default pins,
- manual pin records,

- stable ordering,
- future expansion beyond Organization pages.

Suggested Convex API

Create a focused module such as `convex/pinnedKnowledgePages.ts`.

Suggested public functions:

- `listForSidebar`: query, no args. Requires app access. Returns sidebar-ready pin summaries.
- `pinOrganizationPage`: mutation, args `{ organizationReferentId: v.id("referents") }`. Requires app access and active membership or other read access to that Organization. Upserts `pinState: "pinned"`.
- `unpinKnowledgePage`: mutation, args `{ pageKey: v.string() }` or a more strongly typed union. Requires app access. If the pin is a default seed, persist `pinState: "suppressed"` rather than deleting. If it is manual-only, deleting the row is acceptable.

Implementation notes:

- Do not accept `userId` from the client.
- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Use indexes rather than query filters.
- Return bounded collections.
- For default seeds, derive candidates from `access.organizations`, one per `organizationKind` in the same order the current access helper returns them.
- Merge default candidates with persisted records:
  - no record for default candidate means visible default pin,
  - `suppressed` record means hidden,
  - `pinned` record means visible and may carry order/snapshot,
  - manual pinned records not in default candidates should also be visible.
- Keep hrefs compatible with current routes, e.g. `/organizations/${organizationReferentId}`.

Frontend Guidance

- Replace the direct `getSeededPinnedKnowledgePages(appAccess.organizations)` sidebar data source with `useQuery(api.pinnedKnowledgePages.listForSidebar, {})`.
- While the query is loading, either show the existing deterministic seeded pins as a fallback or a stable loading placeholder that does not jump awkwardly.
- Keep the visible shell behavior from the previous slice:
  - Dashboard first,
  - Pinned Knowledge Pages group in the middle,
  - `+N more` overflow,
  - Calendar/Notifications bottom User View icons,
  - avatar account menu.
- Add a small Pin/Unpin control where scope is clear, preferably on Organization Knowledge Pages or each visible Organization pin's overflow/action area.

- Do not add Bookmarks to the primary sidebar.
- Do not add subscriptions.

Test Plan

Use TDD where practical.

Convex tests:

1. Add `convex/pinnedKnowledgePages.test.ts`.
2. Seed default Organizations/users using existing seed helpers.
3. Assert `listForSidebar` returns one Organization pin per kind when no rows exist.
4. Add a second Organization of the same kind and assert default seeding still caps to one per kind.
5. Unpin a default Organization pin and assert it disappears from `listForSidebar`.
6. Pin it again and assert it reappears.
7. Assert unauthenticated/inactive/no-organization users cannot mutate pins.

Frontend tests:

1. Update `src/App.integrated.test.tsx` mocks to return durable pin query data.
2. Assert sidebar renders durable pins and overflow.
3. Assert unpin/pin UI calls the correct mutation and updates visible state when mock data changes.
4. Assert Bookmarks still live in the avatar/profile path and are not sidebar pins.

Verification Commands

- `npx vitest run convex/pinnedKnowledgePages.test.ts`
- `npx vitest run src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`
- If codegen/API references fail after adding Convex functions, run the repo's usual Convex codegen/dev command and note it in the final response.

Risks / Open Questions

- There is no resolved full pin-management screen yet. Use sidebar/Organization page controls only for this slice.
- There is no resolved ordering UI. Store `sortOrder` and use insertion/default order; drag/drop can come later.
- This slice supports Organization Knowledge Pages first. Broader Knowledge Page pin targets should be added in later slices.
- Current shell tests mock Convex `useQuery` broadly; adding another query may require making the mock distinguish app access, analytics, scripture, and pins more carefully.
- `docs/*` may be ignored by `.gitignore`; if this handoff needs to be committed, force-add it intentionally.

Expected Final Response From Coding Agent

Summarize:

1. What durable pin backend and frontend behavior changed.

2. What tests/checks passed.
3. Any generated files or codegen steps.
4. What remains for broader pins, bookmarks, subscriptions, and pin management.

Code Handoff: Durable Smart Storage Spine

Coding Agent Prompt

You are implementing one vertical slice of the upgraded Knowledge Composer that has already been clarified through a grilling session.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/adr/0004-knowledge-entries-uniquely-represent-same-typed-referents.md`
- `docs/adr/0006-store-smart-storage-proposals-as-durable-silver-records.md`
- `docs/adr/0007-store-smart-storage-proposals-as-persistence-agnostic-domain-contracts.md`
- `docs/adr/0008-use-contribution-submissions-as-smart-storage-parents.md`
- `docs/handoffs/durable-smart-storage-spine.md`
- `convex/schema.ts`
- `convex/smartStorage.ts`
- `convex/smartStorage.test.ts`
- `src/ContributionEditor.tsx`
- `src/ContributionEditor.test.tsx`
- `src/knowledgeContracts.ts`
- `src/App.tsx`
- `src/App.integrated.test.tsx`

Do not re-litigate documented domain decisions unless code and docs directly contradict each other. Keep this slice focused on the durable Smart Storage spine. Do not build advanced extraction, real LLM contract generation, update-existing-entry acceptance, delivery channels, merge/split review, save drafts, or rich multi-proposal child-entry generation.

What To Do

Implement the durable Smart Storage spine for the Knowledge Composer:

Create and persist a durable Contribution Submission with multiple Bronze Sources, queue a Smart Storage Run, create a conservative scaffold Smart Storage Proposal, show and review that proposal, and accept it into one new Gold Layer Knowledge Entry with selected Entry Representations and Source/output links.

Target type: vertical slice.

Why This Target

The current Smart Storage path preserves one standalone Source and one Run, then creates a deterministic draft Proposal. The clarified product model says Smart Storage needs a durable parent Contribution Submission that preserves one user intent, Primary Intended Entry metadata, Review Scope, intended Visibility Scope, Contribution Note, multiple Sources, Runs, Proposals, citation child rows, and later one-at-a-time acceptance.

This slice upgrades the existing Bronze/Silver path without trying to solve extraction intelligence or all future review workflows at once.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- ADRs: `docs/adr/0004-knowledge-entries-uniquely-represent-same-typed-referents.md`, `docs/adr/0006-store-smart-storage-proposals-as-durable-silver-records.md`, `docs/adr/0007-store-smart-storage-proposals-as-persistence-agnostic-domain-contracts.md`, `docs/adr/0008-use-contribution-submissions-as-smart-storage-parents.md`
- Current Smart Storage backend: `convex/smartStorage.ts`, `convex/smartStorage.test.ts`
- Current schema: `convex/schema.ts`
- Contribution UI/contracts: `src/ContributionEditor.tsx`, `src/ContributionEditor.test.tsx`, `src/knowledgeContracts.ts`
- App integration surface: `src/App.tsx`, `src/App.integrated.test.tsx`
- Issue docs: no issue found for this exact slice; inline issue brief below.
- Prototype: no separate prototype found; current Contribution Editor and Smart Storage review panel are the implementation reference.

Inline Issue Brief

What To Build

Upgrade Smart Storage from a standalone Source/Run path into a durable Contribution Submission path:

- Persist a `contributionSubmissions` parent row for Smart Storage.
- Persist multiple child `sources` rows linked to the submission.
- Support Authored Text Source, uploaded file storage IDs, and external URL Sources.
- Track temporary browser-to-Convex uploads before they attach to a submission.
- Store Link Preview latest snapshot fields on external URL Source rows.
- Link Smart Storage Runs and Proposals to the Contribution Submission.
- Store Proposal Source citations as child rows.
- Add required `representationRole` on `entryRepresentations`.
- Add a small TypeScript Type Behavior interface/registry.
- Generate a conservative scaffold Proposal.
- Accept one scaffold Proposal into one new Gold Knowledge Entry.
- Explicitly stop with a target-exists review state instead of updating an existing entry.

Acceptance Criteria

- [ ] A signed-in allowed user can submit through Smart Storage and create one `contributionSubmissions` row.
- [ ] The submission stores `submittedByUserId`, `submissionStatus`, Primary Intended Entry metadata, Contribution Note, intended Visibility Scope, Review Scope, and timestamps.
- [ ] The first Smart Storage path defaults Review Scope to the submitting user unless an Organization or Group review context is explicit.
- [ ] The submission creates child Source rows for authored text, uploaded file storage IDs, and external URLs.
- [ ] New Smart Storage mutation paths always provide `sources.contributionSubmissionId`, even if the schema field remains optional for migration compatibility.
- [ ] Browser-to-Convex uploaded files are tracked in `temporaryUploads` before attachment and marked `attached` when used in a submission.
- [ ] External URL Sources preserve the submitted URL even when Link Preview fails.
- [ ] Link Preview failure does not block Contribution Submission persistence or Smart Storage queueing.
- [ ] A Smart Storage Run links to `contributionSubmissionId` and may link to a `primarySourceId`.
- [ ] A Smart Storage Proposal links to both `contributionSubmissionId` and `smartStorageRunId`, and backend code enforces that they match.
- [ ] Proposal Source citations are stored as child rows, not as an unbounded proposal array.
- [ ] Scaffold Proposals can be reviewed in the existing Smart Storage review UI or a small extension of it.
- [ ] Accepting a scaffold Proposal creates one new Gold Knowledge Entry, selected Entry Representations, and Source/output links atomically.
- [ ] Accepted Entry Representations include required `representationRole`, using `unspecified` when the role is unknown.
- [ ] If acceptance discovers that the proposed represented Referent already has a Knowledge Entry, it stops with a reviewable target-exists state instead of patching the existing entry.
- [ ] Direct post remains distinct from Smart Storage and does not create Bronze/Silver workflow state by default.
- [ ] Existing Smart Storage tests are updated rather than bypassed.

Out Of Scope

- Advanced extraction.
- Real LLM contract generation.
- Update-existing-entry acceptance.
- Delivery channels.
- Merge/split review.
- Save drafts or autosave.
- Rich multi-proposal child-entry generation.
- Full Course generation or complex child-entry workflows.
- Dataset-wide Reprocessing.
- Bulk proposal acceptance.

Deferred Terms

- `advanced extraction`: parsing uploaded files, remote pages, media, transcripts, or rich documents into structured text, ranges, fields, or derived child knowledge. This slice may preserve files and URLs, but it should not pretend to understand content it has not extracted.
- `real LLM contract generation`: sending versioned Smart Storage Contracts and Type Behavior snapshots to an LLM to produce true model-generated Proposals. This slice can keep the deterministic scaffold generator.
- `update-existing-entry acceptance`: accepting a Proposal as a patch to an already existing Gold Knowledge Entry. This is deferred because it needs edit permissions, conflict handling, and audit behavior.
- `delivery channels`: notifying, assigning, messaging, emailing, SMS-ing, or DM-ing recipients about a contribution. Visibility Scope and Review Scope are not delivery.
- `merge/split review`: permissioned identity correction when Referents or Knowledge Entries need to be combined, separated, aliased, or reclassified. This is separate from ordinary proposal acceptance.
- `save drafts`: preserving incomplete composer sessions before explicit submission. This slice begins durable state at explicit Smart Storage submission, while temporary uploads are tracked only to avoid abandoned storage.
- `rich multi-proposal child-entry generation`: generating many related Gold entries from one submission, such as a Course with Lessons or a transcript with many Quotes. This slice accepts one proposed Knowledge Entry at a time and may create only a conservative scaffold Proposal.

Domain Language

- `Contribution Submission`: a single user intent to Contribute, collecting material and choices submitted from a composer before it is posted directly or processed through Smart Storage.
- `Primary Intended Entry`: the one Knowledge Entry a Contribution Submission is primarily meant to create or update.
- `Source`: Bronze Layer raw material, not a Knowledge Type and not a Knowledge Entry.
- `Authored Text Source`: user-authored or pasted substantive text submitted as raw material in a Contribution Submission.
- `Contribution Note`: non-substantive guidance that steers a Contribution Submission without becoming represented knowledge by default.
- `Review Scope`: who may see or manage Bronze Sources, Smart Storage Runs, Smart Storage Proposals, and other pending review material before Gold Layer knowledge exists.
- `Visibility Scope`: who may access the accepted Gold Layer Knowledge Entry afterward.
- `Entry Representation`: a content or media form through which a Knowledge Entry is expressed.
- `Primary Representation`: the selected default Entry Representation for display/open/preview/playback.
- `Representation Role`: how an Entry Representation functions for the entry, such as manuscript, slides, transcript, recording, thumbnail, primary content, or supporting material.
- `Type Behavior`: a per-Knowledge-Type implementation rule set for identity, scope, composer defaults, Smart Storage challenge behavior, representation roles, primary representation defaults, Human Weight, and provenance.

Avoid using `Source` to mean Knowledge Entry, Entry Representation, or file type.

Decisions That Must Hold

- Durable Contribution Submissions are the parent grouping model for first-slice Smart Storage.

- Simple text-only direct posts may still skip durable Contribution Submission workflow state.
- Smart Storage should not run automatically for every Contribution.
- Attachments and external URLs in the first composer slice route through the durable Bronze path rather than simple direct posting.
- Review Scope and Visibility Scope are separate. Matching enum values do not make them interchangeable.
- Tags do not grant access, and Delivery cannot silently widen Visibility.
- Bronze Sources and Silver Smart Storage Proposals stay out of the normal Answer Feed.
- Link Preview is not a Knowledge Entry, Factual Provenance, Human Weight Evidence, or Source.
- Source citations for Proposals are child rows.
- Accepted proposals decide explicitly which Sources become Entry Representations.
- Bronze Sources remain usable for later extraction or Reprocessing after acceptance.
- Smart Storage Proposal acceptance creates new entries only in this slice.
- `Representation Role` is required on Gold Entry Representations, using `unspecified` when unknown.
- `Representation Role` does not replace explicit Primary Representation selection.
- Type Behavior is a small TypeScript interface/registry now, not a broad inheritance framework.

Schema Guidance

Add or update validators in `convex/schema.ts` following the generated Convex guidelines.

New `contributionSubmissions`

Minimum fields:

- `submittedByUserId`
- `submissionStatus`
- `primaryIntendedKnowledgeType`
- `primaryIntendedTitle`
- `primaryIntendedBodyPreview`
- `contributionNote`
- `intendedVisibilityKind`
- `intendedVisibilityTargetKey`
- `reviewScopeKind`
- `reviewScopeTargetKey`
- `createdAt`
- `updatedAt`

Initial `submissionStatus` enum:

- `submitted`
- `processing`
- `reviewReady`

- partiallyAccepted
- accepted
- rejected
- cancelled

Index suggestions:

- by_submittedByUserId_and_createdAt
- by_submissionStatus_and_createdAt
- by_reviewScopeKind_and_reviewScopeTargetKey_and_createdAt

Review Scope

Use a separate enum, initially:

- private
- organization
- group
- public

Use reviewScopeTargetKey string storage in the first implementation, with backend validation that the key format matches reviewScopeKind.

Update sources

Add:

- contributionSubmissionId: optional in schema for migration compatibility, required in new Smart Storage mutation paths.
- Link Preview latest snapshot fields for external URL Sources:
  - linkPreviewStatus
  - linkPreviewTitle
  - linkPreviewDescription
  - linkPreviewImageUrl
  - linkPreviewSiteName
  - linkPreviewFetchedAt
  - linkPreviewError

External URL Sources must always preserve externalUrl.

Index suggestions:

- by_contributionSubmissionId_and_submittedAt
- keep existing source indexes if still useful.

New temporaryUploads

Fields:

- storageId
- uploadedByUserId

- `fileName`
- `contentType`
- `fileSizeBytes`
- `uploadStatus`
- `expiresAt`
- `attachedContributionSubmissionId`
- `createdAt`
- `updatedAt`

Initial `uploadStatus` enum:

- `uploaded`
- `attached`
- `expired`
- `deleted`

Index suggestions:

- `by_uploadedByUserId_and_createdAt`
- `by_uploadStatus_and_expiresAt`
- `by_storageId`

Update `smartStorageRuns`

Add:

- `contributionSubmissionId`
- optional `primarySourceId`

The existing `sourceId` may remain during migration if needed, but the new workflow should treat the run as belonging to the Contribution Submission. If both old and new source fields exist temporarily, keep their meaning explicit.

Index suggestions:

- `by_contributionSubmissionId_and_createdAt`
- keep `by_status_and_createdAt`.

Update `smartStorageProposals`

Add:

- `contributionSubmissionId`

Backend code must enforce that the Proposal and Run point to the same Contribution Submission.

Index suggestions:

- `by_contributionSubmissionId_and_status_and_createdAt`
- keep `by_smartStorageRunId`.

New `proposalSourceCitations`

Fields:

- proposalId
- sourceId
- citationKind
- excerptText
- locator
- externalUrl
- rationale
- createdAt

Initial citationKind enum:

- wholeSource
- textExcerpt
- fileLocator
- externalUrl

Keep excerptText bounded and optional.

Index suggestions:

- by_proposalId
- by_sourceId

Update entryRepresentations

Add required:

- representationRole

Initial enum:

- unspecified
- primaryContent
- manuscript
- slides
- transcript
- recording
- thumbnail
- supportingMaterial

Type Behavior determines which roles are allowed or default per Knowledge Type. Use unspecified when unknown.

Type Behavior Guidance

Create a small TypeScript interface and registry, likely outside React-specific code. The first interface should expose:

- knowledgeType
- version
- identity

- `referentIdentityScope`
- `composerDefaults`
- `smartStorageChallenge`
- `representationRoles`
- `primaryRepresentation`
- `humanWeight`
- `provenance`

Use plain config objects or small functions. Shared resolver services should perform actual Tag, Referent, alias, and represented-entry lookup rather than duplicating database querying per Knowledge Type.

Backend Guidance

The existing `convex/smartStorage.ts` has the right broad shape but currently starts from one standalone Source. Evolve it into the new parent path carefully.

Suggested public mutations/actions, names may vary if local patterns suggest better:

- `createTemporaryUploadRecord`
- `startFromContributionSubmission`
- `generateScaffoldProposalForRun`
- `acceptScaffoldProposal`

Implementation notes:

- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Keep validators on every Convex function.
- Do not pass file bytes through contribution mutations; use storage IDs and metadata.
- Create the Contribution Submission and all child Sources in one mutation where feasible.
- Mark temporary uploads `attached` when their storage IDs become Source rows.
- Queue or create the first Smart Storage Run automatically only for the Smart Storage path.
- Generate a deterministic scaffold Proposal before real LLM contract generation exists.
- Use child `proposalSourceCitations` rows for evidence/source references.
- Preserve original vs current proposal shape.
- Accept only one Proposal into one new Gold Entry.
- Acceptance should create Knowledge Entry, represented Tag relation, context Tag relations, selected Entry Representations, and `sourceOutputs` atomically.
- If the represented Referent already has a Knowledge Entry, return or set a reviewable target-exists state rather than updating the existing entry.
- Keep direct post separate.

Frontend Guidance

Current relevant seams:

- `src/App.tsx:3167`: `startSmartStorage` mutation hook.

- `src/App.tsx:3168`: `generateSmartStorageProposal` mutation hook.
- `src/App.tsx:3243`: `handleStoreSmartlyContribution`.
- `src/App.tsx:3370`: `ContributionEditor` receives `onStoreSmartly`.
- `src/App.tsx:3497`: `SmartStorageProposalReviewPanel`.
- `src/ContributionEditor.tsx`: builds `ContributionInput` and preserves direct vs Smart Storage mode.
- `src/knowledgeContracts.ts:139`: current `ContributionInput` shape.

Frontend implementation can be incremental:

- Extend contribution contracts to carry authored text, file storage metadata, external URLs, and Contribution Note if not already present.
- Keep text-only direct post behavior separate.
- When uploaded files or external URLs are present, route the composer through Smart Storage/Bronze path.
- Show Link Preview status for external URL Sources only if it can be done without turning this into a large UI redesign.
- Reuse or minimally extend the existing Smart Storage proposal review panel for scaffold Proposals.
- Proposal review must let the user confirm which Sources become Entry Representations and which representation is primary when more than one representation exists.
- If the UI cannot fully support source-to-representation selection yet, keep the first review state narrow and explicit rather than silently accepting every Source as a representation.

Relevant Code Map

- `convex/schema.ts:553`: current `sources` table.
- `convex/schema.ts:570`: current `sourceOutputs` table.
- `convex/schema.ts:579`: current `smartStorageRuns` table.
- `convex/schema.ts:605`: current `smartStorageProposals` table.
- `convex/schema.ts:650`: current `entryRepresentations` table.
- `convex/smartStorage.ts:128`: current `startFromContribution` standalone Source path.
- `convex/smartStorage.ts:187`: current `generateDraftProposalForRun`.
- `convex/smartStorage.ts:311`: deterministic draft Proposal builder.
- `convex/smartStorage.test.ts:15`: existing Bronze Source and Run test.
- `convex/smartStorage.test.ts:118`: existing draft Proposal generation test.
- `src/App.tsx:3243`: current Smart Storage submit flow.
- `src/App.tsx:3497`: current Smart Storage proposal review panel.
- `src/ContributionEditor.tsx:140`: current contribution input creation.
- `src/ContributionEditor.test.tsx`: current mode and payload tests.
- `src/knowledgeContracts.ts:139`: current contribution input contract.

Test Plan

Use TDD where practical.

Convex tests:

1. Update or add tests around `convex/smartStorage.test.ts`.
2. Assert Smart Storage submission creates a Contribution Submission parent.
3. Assert multiple Sources can attach to one Contribution Submission.
4. Assert new Smart Storage Sources carry `contributionSubmissionId`.
5. Assert temporary uploads are marked attached when used.
6. Assert external URL Link Preview failure does not block preserving the URL Source.
7. Assert Runs and Proposals link to the same `contributionSubmissionId`.
8. Assert Proposal Source citations are child rows.
9. Assert `representationRole` is required on accepted Entry Representations.
10. Assert acceptance creates one new Knowledge Entry, Entry Representations, and Source/output links.
11. Assert acceptance refuses target-existing cases rather than updating existing entries.
12. Assert unauthorized users cannot create submissions, runs, proposals, or acceptance writes.

Frontend tests:

1. Extend `src/ContributionEditor.test.tsx` for any new contribution payload fields.
2. Extend `src/App.integrated.test.tsx` mocks for the new Smart Storage mutation return shapes.
3. Assert Store Smartly calls the durable Contribution Submission path.
4. Assert the proposal review panel can render a scaffold Proposal with source inventory/citations.
5. Assert direct post mode remains distinct.

Verification Commands

```
- npx convex codegen
- npx vitest run convex/smartStorage.test.ts
- npx vitest run src/ContributionEditor.test.tsx
- npx vitest run src/App.integrated.test.tsx
- npx vitest run convex/smartStorage.test.ts src/ContributionEditor.test.tsx
  src/App.integrated.test.tsx
- npx tsc -b --pretty false
- npm run build
```

Manual browser check if a signed-in local session is available:

- Start Vite on an open port.
- Submit text through Store Smartly.
- Submit a file through Store Smartly if the UI supports file selection in this slice.
- Submit an external URL through Store Smartly if the UI supports URL entry in this slice.
- Confirm a scaffold Proposal appears and can be accepted into a new entry.
- Confirm Direct Post still bypasses Bronze/Silver workflow state.

If the browser only reaches the unauthenticated sign-in screen, say so and rely on automated tests for signed-in behavior.

Risks / Open Questions

- Existing historical Source rows do not have `contributionSubmissionId`; keep the field optional in schema for migration compatibility while requiring it in new Smart Storage paths.
- Existing tests may assume one Source per Run; update them to the parent submission model rather than preserving the old assumption.
- The UI may not yet have file/URL controls in the Contribution Editor. If building those controls makes the slice too large, implement backend support plus the narrowest usable UI for one file and one URL.
- Link Preview fetching may require a Convex action; keep it non-blocking.
- This slice intentionally creates conservative scaffold Proposals rather than true extraction output.
- `docs/handoffs/*` is ignored by `.gitignore`; force-add this handoff intentionally if it needs to be committed.

Expected Final Response From Coding Agent

Summarize:

1. What schema and backend Smart Storage behavior changed.
2. What UI or contract changes were made.
3. How the implementation preserves Contribution Submission, Source, Run, Proposal, citation, and Entry Representation invariants.
4. What tests/checks passed.
5. What remains for extraction, LLM contract generation, update-existing acceptance, delivery, merge/split, drafts, and rich multi-proposal generation.

Code Handoff: Durable Subscriptions And Notification Readiness

Coding Agent Prompt

You are implementing the next vertical slice after durable pins and durable bookmarks.

Before editing code, read:

- `AGENTS.md`
- `convex/_generated/ai/guidelines.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/handoffs/header-sidebar-navigation.md`
- `docs/handoffs/durable-pinned-knowledge-pages.md`
- `docs/handoffs/durable-bookmarked-knowledge-pages.md`
- `src/App.tsx`
- `src/App.integrated.test.tsx`
- `src/index.css`
- `convex/schema.ts`
- `convex/lib/appAccess.ts`
- `convex/pinnedKnowledgePages.ts`
- `convex/bookmarkedKnowledgePages.ts`
- `convex/bookmarkedKnowledgePages.test.ts`

Do not re-litigate the resolved navigation language unless code and docs directly contradict each other. Keep this slice focused on durable Subscriptions and making them visible from the Notifications User View; do not build a full notification delivery engine, notification preferences, or broad saved-set management.

What To Do

Persist Subscriptions as the user's standing interest in activity around a Knowledge target, beginning with Organization Knowledge Pages.

Implement durable user-specific subscriptions for Organization Knowledge Pages first:

- a Convex table and APIs for current-user subscriptions,
- an Organization Knowledge Page Subscribe / Unsubscribe control,
- a Notifications-page section that lists active subscription sources,
- an unsubscribe action from the Notifications page,
- no sidebar placement, no bookmark side effect, no pin side effect, and no generated notification events yet.

Target type: inferred next vertical slice.

Why This Target

Pins and Bookmarks are now durable and intentionally separate. The remaining relationship clarified in the grill is `Subscription`: a standing interest that affects notification behavior. The app already has a persistent bottom `Notifications` User View and static notification cards, including a `subscription` kind, but there is no durable subscription model yet.

This slice creates the durable relationship without pretending notification generation is solved. It should make subscriptions visible where users expect notification-related state to live, while preserving the distinction:

- Bookmark: save for later.
- Pin: keep easy to navigate.
- Subscription: receive notification-relevant activity.
- Notification: a concrete notice that something happened.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Previous shell handoff: `docs/handoffs/header-sidebar-navigation.md`
- Previous pin handoff: `docs/handoffs/durable-pinned-knowledge-pages.md`
- Previous bookmark handoff: `docs/handoffs/durable-bookmarked-knowledge-pages.md`
- Convex guidance: `convex/_generated/ai/guidelines.md`
- Current app access helper: `convex/lib/appAccess.ts`
- Current bookmark implementation pattern: `convex/bookmarkedKnowledgePages.ts`, `convex/bookmarkedKnowledgePages.test.ts`
- Current schema: `convex/schema.ts`
- Current Organization page and Notifications page: `src/App.tsx`, `src/index.css`
- Existing integrated tests: `src/App.integrated.test.tsx`
- Relevant ADRs: no subscription-specific ADR found.
- Issue docs: no issue found for this slice; inline issue brief below.
- Prototype: no subscription prototype found; current Notifications page is the implementation reference.

Inline Issue Brief

What To Build

Add a persisted Subscription model for Organization Knowledge Pages and wire it through the visible product path:

1. A signed-in user can subscribe to an Organization Knowledge Page.
2. The Organization page shows whether the user is subscribed.
3. The Notifications User View lists active subscription sources.
4. The user can unsubscribe from the Organization page or Notifications page.
5. Subscribing does not pin, bookmark, or create notification rows.

Acceptance Criteria

- [ ] A signed-in allowed user can query their active Subscriptions from Convex.
- [ ] A signed-in allowed user can subscribe to an Organization Knowledge Page they can access.
- [ ] Subscribing derives the current user server-side; no client function accepts `userId` for authorization.
- [ ] Subscription uniqueness is per user and per target.
- [ ] Re-subscribing to an already subscribed target is idempotent and updates `updatedAt` rather than creating duplicates.
- [ ] Unsubscribing removes only the current user's Subscription relationship.
- [ ] Unsubscribing does not remove a pin or bookmark for the same Knowledge Page.
- [ ] Subscribing does not create a pin or bookmark for the same Knowledge Page.
- [ ] Subscribing does not create a Notification row or change the unread notification count in this slice.
- [ ] The Organization Knowledge Page shows a Subscribe / Unsubscribe control distinct from Pin / Unpin and Bookmark / Remove Bookmark.
- [ ] The Notifications page renders a bounded list of active subscription sources with label, target metadata, link to the Knowledge Page, and an unsubscribe action.
- [ ] The bottom sidebar Notifications badge continues to mean unread Notifications, not subscription count.
- [ ] Subscribing an Organization Knowledge Page records the action as meaningful for Recognized Context by upserting a user `tagRecognitions` row for that Organization's canonical Tag when one can be resolved.
- [ ] Existing durable pin and durable bookmark tests still pass.

Out Of Scope

- Notification event generation.
- Notification read/unread persistence.
- Notification delivery, email, push, digest, or preference settings.
- Subscribing to every possible target type.
- Subscribing to User Views.
- Subscription groups, folders, muting, frequency controls, or recommendation UI.
- Sidebar placement for subscriptions.
- Full Global Search.

Domain Language

- `Subscription`: a user's standing interest in activity within a Knowledge Context, Organization, Knowledge Slot, or Event.
- `Notification`: a user-visible notice that relevant activity occurred within a Subscription, assigned Knowledge Slot, or Event participation.
- `Bookmark`: a User relationship to a Knowledge Page saved for later reference. It does not place the page in sidebar navigation and does not subscribe the User to notifications.
- `Pinned Knowledge Page`: a User relationship to a Knowledge Page that keeps it easy to return to from sidebar navigation. It does not subscribe the User to notifications.

- `Knowledge Page`: a shared, world-facing location grounded in a Knowledge Context, Referent, Knowledge Entry, Organization, or other knowledge object.
- `User View`: a current-user scoped view such as Calendar, Notifications, Settings, or editing the user's own profile.
- `Recognized Context`: the historical union of Knowledge Contexts where a User or Organization has taken meaningful action.

Avoid `follow`, `alert`, and `notification setting` as canonical domain terms for this slice. The UI verb can be `Subscribe`, but the stored relationship should be a `Subscription`.

Decisions That Must Hold

- Subscription is separate from Bookmark.
- Subscription is separate from Pinning.
- Subscription is separate from Notification.
- A Subscription may affect notification behavior, but it is not itself a Notification.
- Notifications remain a bottom User View icon because unread status needs a persistent badge.
- The Notifications badge should not count Subscriptions.
- Bookmarked Knowledge Pages should not appear directly in the primary sidebar.
- Pinned Knowledge Pages remain the sidebar middle group.
- Plain page visits alone should not add to Recognized Context.
- Subscribing is a meaningful action and may contribute to Recognized Context.
- Dashboard stays the fixed first sidebar route.

Relevant Code Map

- `convex/schema.ts:364`: existing `tagRecognitions` table; use this for Recognized Context when subscribing resolves a canonical Tag.
- `convex/schema.ts:428`: existing `pinnedKnowledgePages`; do not overload this table for subscriptions.
- `convex/schema.ts:451`: existing `bookmarkedKnowledgePages`; model subscriptions separately.
- `convex/lib/appAccess.ts`: use `requireAppAccess(ctx)` for current `userId` and allowed Organizations.
- `convex/bookmarkedKnowledgePages.ts`: good pattern for current-user relationships, page keys, Organization access checks, idempotent upsert, tag recognition, and bounded list queries.
- `convex/bookmarkedKnowledgePages.test.ts`: good pattern for Convex relationship tests.
- `src/App.tsx:209`: `NotificationKind` already includes `subscription` for static notices.
- `src/App.tsx:581`: static subscription-kind notification example.
- `src/App.tsx:2422`: Organization page already has Pin and Bookmark mutations/queries; add Subscription nearby.
- `src/App.tsx:2530`: Organization page controls currently contain Pin and Bookmark buttons; add Subscribe without making the hero crowded.
- `src/App.tsx:3831`: `NotificationsPage` currently renders static `USER_NOTIFICATIONS`; add active subscription sources here without replacing notification cards.
- `src/App.integrated.test.tsx:769`: recent bookmark tests are the closest frontend pattern.
- `src/index.css:1520`: Organization page control styles.

- `src/index.css:2873`: Notifications summary/panel styles.

Suggested Data Model

Add a table such as `knowledgeSubscriptions` or `userKnowledgeSubscriptions` in `convex/schema.ts`.

Suggested fields:

- `userId: v.id("users")`
- `subscriptionKey: v.string()`; stable unique key such as `organization:<referentId>`
- `targetKind: v.union(v.literal("organization"))` for this slice
- `organizationReferentId: v.optional(v.id("referents"))`
- `targetReferentId: v.optional(v.id("referents"))`
- `targetTagId: v.optional(v.id("tags"))`
- `labelSnapshot: v.string()`
- `hrefSnapshot: v.string()`
- `secondaryLabelSnapshot: v.optional(v.string())`
- `createdAt: v.number()`
- `updatedAt: v.number()`

Suggested indexes:

- `by_userId_and_subscriptionKey`
- `by_userId_and_updatedAt`
- `by_userId_and_targetKind_and_updatedAt`
- `by_targetKind_and_targetReferentId`
- optionally `by_targetTagId`

The exact shape can differ if a cleaner local pattern emerges, but preserve these capabilities:

- unique current-user/target relationship,
- bounded current-user listing,
- future lookup of subscribers by target for notification generation,
- enough target metadata for UI display without expensive joins,
- separation from pins, bookmarks, and notifications.

Do not add notification frequency or delivery preferences in this slice. That is a separate settings problem.

Suggested Convex API

Create a focused module such as `convex/knowledgeSubscriptions.ts`.

Suggested public functions:

- `listForNotifications`: query, no args or optional bounded `limit`. Requires app access. Returns active subscription summaries newest first.
- `getForTarget`: query, args `{ subscriptionKey: v.string() }`. Requires app access. Returns the current user's subscription summary for that target or `null`.

- `subscribeOrganizationPage`: mutation, args `{ organizationReferentId: v.id("referents") }`. Requires app access and active membership/access to that Organization. Upserts the Subscription relationship.
- `unsubscribe`: mutation, args `{ subscriptionKey: v.string() }`. Requires app access. Removes only the current user's Subscription row for that target.

Implementation notes:

- Do not accept `userId` from the client.
- Use `requireAppAccess(ctx)` and derive `userId` server-side.
- Use indexes, not query filters.
- Keep query results bounded.
- Reuse target identity conventions from pins/bookmarks, e.g. `organization:${organizationReferentId}` and `/organizations/${organizationReferentId}`.
- To resolve Organization metadata, use `access.organizations` first.
- If a Subscription points to an Organization the user no longer has access to, do not leak current Organization data. It is acceptable in this slice to omit that Subscription from the list or return only snapshot fields, but document the choice in the final response.
- For Recognized Context, when subscribing to an Organization page:
  - find a Tag for `organizationReferentId` via `tags.by_referentId`,
  - upsert a user `tagRecognitions` row via `by_userId_and_tagId`,
  - set `recognizerKind: "user"`, `userId`, `recognizedAt`, and `lastInteractedAt`,
  - update `lastInteractedAt` on repeat subscribe,
  - do not remove tag recognition on unsubscribe, because Recognized Context is historical.

Frontend Guidance

- Add a compact Subscribe / Unsubscribe control to Organization Knowledge Pages near the existing Pin and Bookmark controls.
- Keep the control visually distinct from Pin and Bookmark. Use a bell-style icon if available from `lucide-react`.
- Do not add explanatory copy that turns the hero into a tutorial. The button label is enough for this slice.
- Add a query to the Organization page to determine whether the current target is subscribed.
- Add a query to `NotificationsPage` for active Subscriptions.
- Add a section or panel on `NotificationsPage` for active subscription sources. Suggested heading: `Subscriptions` or `Subscription Sources`.
- Each subscription row should link to the Knowledge Page and include target metadata such as `School`, `Church`, `Family`, or `Community`.
- Include an unsubscribe action in the subscription source row if it can be done without clutter.
- Keep existing static notification cards and filters working. This slice should not require a durable Notification table.
- Do not put Subscriptions in the sidebar or avatar menu.
- Do not change the Notifications badge semantics.

Test Plan

Use TDD where practical.

Convex tests:

1. Add `convex/knowledgeSubscriptions.test.ts`.
2. Use the same seeded user/Organization helpers or patterns from `convex/bookmarkedKnowledgePages.test.ts`.
3. Assert `listForNotifications` returns an empty list when the user has no Subscriptions.
4. Assert `subscribeOrganizationPage` creates one Subscription row for an accessible Organization Knowledge Page.
5. Assert calling `subscribeOrganizationPage` twice does not create duplicates and updates the existing row.
6. Assert `getForTarget` returns the current user's Subscription for that target.
7. Assert `unsubscribe` removes the Subscription for the current user only.
8. Assert unauthorized, inactive, and no-organization users cannot subscribe to Organization pages.
9. Assert subscribing an Organization page upserts a user `tagRecognitions` row when the Organization Tag exists.

Frontend tests:

1. Extend `src/App.integrated.test.tsx` mocks to support Subscription queries/mutations.
2. Assert the Organization page Subscribe / Unsubscribe control calls the correct mutation and toggles after mock state changes.
3. Assert subscribing does not change `Knowledge Page destinations`.
4. Assert subscribing does not create a bookmark in the profile Bookmarks section.
5. Assert the Notifications page renders durable subscription sources with links and an unsubscribe action.
6. Assert the bottom sidebar Notifications badge remains based on unread notifications rather than subscription count.
7. Assert existing pin and bookmark integrated tests still pass.

Verification Commands

- `npx convex codegen`
- `npx vitest run convex/knowledgeSubscriptions.test.ts`
- `npx vitest run src/App.integrated.test.tsx`
- `npx vitest run convex/pinnedKnowledgePages.test.ts convex/bookmarkedKnowledgePages.test.ts convex/knowledgeSubscriptions.test.ts`
- `npx vitest run convex/knowledgeSubscriptions.test.ts src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`

Manual browser check if a signed-in local session is available:

- Start Vite on an open port.
- Visit an Organization Knowledge Page.
- Subscribe to it.

- Open Notifications.
- Confirm the subscription source appears.
- Unsubscribe from Notifications and confirm the Organization page control reflects the change.
- Confirm sidebar pins and profile bookmarks are unchanged.

If the browser only reaches the unauthenticated sign-in screen, say so and rely on the automated tests for signed-in behavior.

Risks / Open Questions

- This slice supports Organization Knowledge Page Subscriptions first. Broader targets such as Context Pages, Tags, Knowledge Entries, Events, Groups, and Knowledge Slots should be separate follow-up slices.
- The durable Notification model is not part of this slice. The existing static notification cards can remain until a later notification-generation slice.
- Notification frequency, muting, delivery channels, and digest settings are unresolved. Do not add them here.
- If a user loses Organization access, the product behavior is not fully specified. Prefer a conservative implementation that does not expose current Organization data beyond `requireAppAccess`.
- `docs/*` may be ignored by `.gitignore`; if this handoff needs to be committed, force-add it intentionally.

Expected Final Response From Coding Agent

Summarize:

1. What durable Subscription backend and frontend behavior changed.
2. Whether tag recognition was updated for subscribe actions.
3. What tests/checks passed.
4. Any generated files or codegen steps.
5. What remains for broader subscription targets, durable notifications, delivery preferences, and notification generation.

Code Handoff: Header And Sidebar Navigation

Coding Agent Prompt

You are implementing one frontend shell slice that was clarified through a grilling session.

Before editing code, read:

- `AGENTS.md`
- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `docs/agents/domain.md`
- `src/App.tsx`
- `src/index.css`

Do not re-litigate the documented navigation language unless code and docs directly contradict each other. If you touch Convex backend code, read `convex/_generated/ai/guidelines.md` first; this handoff is intended to avoid backend changes.

What To Do

Refine and implement the app shell header/sidebar layout so it matches the resolved model:

- Sidebar carries destination navigation.
- Header carries Active Role and Global Search only.
- Current Knowledge Page / Active Knowledge Context stays below the header in page content.
- Account controls live behind the sidebar avatar, not in both header and sidebar.

Target type: narrowed frontend vertical slice.

This narrows the broader request to the first independently reviewable implementation: update the existing React/Vite shell UI using current route data and current signed-in access data. Do not add durable backend pin/bookmark/subscription persistence in this slice.

Why This Target

The current shell is an icon-only rail plus a header with brand, theme, notifications, sign out, and search. The clarified design needs a clearer grammar:

- `Dashboard` is the fixed first global Knowledge Page.
- Pinned Knowledge Pages occupy the middle of the sidebar.
- Calendar and Notifications are visible bottom User View icons.
- Avatar is the account-menu entry point.
- Header is quiet: Active Role switcher plus Global Search.

This can be implemented as a frontend tracer bullet using existing `AllowedAppAccess.organizations` and existing route/page code, while leaving durable pin/bookmark/subscription tables for a later slice.

Source Artifacts

- Domain language: `CONTEXT.md`
- Product decisions: `docs/product-core.md`
- Existing frontend loop decisions: `docs/mvp-frontend-core-loop.md`
- Agent docs: `AGENTS.md`, `docs/agents/domain.md`, `docs/agents/issue-tracker.md`, `docs/agents/triage-labels.md`
- Existing prototype references: `src/prototypes/LayoutPrototype.tsx`, `src/prototypes/layoutPrototype.css`
- Existing handoff style: `docs/handoffs/mvp-frontend-contracts-and-routing.md`
- Relevant ADRs: No header/sidebar-specific ADR found.
- Issue docs: No issue found for this slice; inline issue brief below.

Inline Issue Brief

What To Build

Rework the app shell navigation so the sidebar/header embody the documented distinction between Knowledge Pages, User Views, Active Role, Global Search, bookmarks, pins, and subscriptions.

Acceptance Criteria

- [ ] The first sidebar destination is fixed `Dashboard`; there is no separate `Explore` primary sidebar icon.
- [ ] The sidebar middle shows seeded Pinned Knowledge Pages derived from the current user's Organizations, capped to at most one Organization per kind where possible.
- [ ] Seeded Organization pins use the specific Organization name as the primary label, with kind shown through icon, secondary text, or grouping.
- [ ] Pinned Knowledge Pages have a visible label affordance on desktop; they are not icon-only except in a compact/collapsed responsive state.
- [ ] If visible pins exceed the available/capped space, the sidebar shows a concise overflow control such as `+3 more`.
- [ ] Bottom User View icons include Notifications with a visible unread/count badge and Calendar.
- [ ] Settings is not a persistent bottom icon; it is reachable through the sidebar avatar account menu.
- [ ] Profile is represented by the avatar only; there is no separate profile icon.
- [ ] The avatar/account menu includes Profile, Bookmarks shortcut, Settings, Sign out, and any existing user-preference action that must move out of the header, such as theme toggle.
- [ ] Profile editing is not a separate `Edit Profile` menu item; the profile page itself should expose editable fields when/where implemented.
- [ ] Header shows an Active Role display that also acts as a role switcher.
- [ ] Switching Active Role changes frontend acting-capacity state without navigating away.
- [ ] Navigating to an Organization Knowledge Page does not silently switch Active Role.
- [ ] Header Global Search remains visually separate from page/context-scoped ask/search controls.
- [ ] Header does not duplicate account controls already available through the sidebar avatar.

- [ ] The Knowledge Navigator remains visible on Knowledge Pages and is not shown as the main navigator on User Views such as Calendar or Notifications.
- [ ] Existing route rendering for Dashboard, Organization pages, Calendar, Notifications, Profile, and Settings still works.
- [ ] Existing related tests pass or are updated to match the new shell behavior.

Out Of Scope

- Durable backend storage for user-created pins.
- Durable backend storage for bookmarks.
- Durable backend storage for subscriptions.
- Full global search backend implementation.
- Building a complete pin-management screen.
- Building a complete bookmarks workflow beyond a profile section/shortcut placeholder if needed.
- Reworking all page bodies beyond what is required to place the shell/header/sidebar correctly.

Domain Language

- `Knowledge Page`: shared, world-facing location grounded in a Knowledge Context, Referent, Knowledge Entry, Organization, or similar knowledge object.
- `User View`: user-scoped view assembled around the current User's activity, responsibilities, preferences, or account state.
- `Active Knowledge Context`: the Knowledge Context in effect for the current Knowledge Page.
- `Recognized Context`: historical union of Knowledge Contexts where the user or organization took meaningful action; not the same as Active Knowledge Context.
- `Global Search`: search across everything the current User can access, independent of Active Knowledge Context.
- `Active Role`: the single Role the User is currently acting in; header display and switcher.
- `Bookmark`: saved Knowledge Page for later reference; no sidebar placement and no notifications.
- `Pinned Knowledge Page`: Knowledge Page kept easy to return to, especially from sidebar navigation; no notifications by itself.
- `Subscription`: standing interest that affects notifications.

Avoid using `Place` for app location because `Place` is a Knowledge Type. Avoid using `folder`, `screen`, or `route` as product language for Knowledge Page.

Decisions That Must Hold

- Dashboard is a fixed first sidebar route and not removable.
- Explore is an action, not a separate primary sidebar icon for now.
- Pinned Knowledge Pages should not rely on icon-only recognition.
- Default Organization pins are seeded from affiliations but can later be unpinned, and unpin suppression should persist in a future backend slice.
- Default pin seeding should be capped; do not dump every affiliation into the sidebar.
- Bookmarked Knowledge Pages should not appear directly in the primary sidebar.
- Bookmarks belong on the profile as a section/tab, with an avatar-menu shortcut.

- Notifications stay visible as a bottom User View icon because unread status needs a badge.
- Calendar is a bottom User View icon; specific Events remain Knowledge Pages.
- Settings belongs in the avatar/account menu unless future usage proves it needs a visible icon.
- The Active Role switcher does not navigate and should not auto-switch on Organization page navigation.
- Global Search result selection should navigate to the result's Knowledge Page and synchronize Knowledge Navigator active Tags to that page's Active Knowledge Context when that behavior is implemented.
- Context-scoped Search/Ask/Explore belongs below the header near the page title or Active Knowledge Context, not in the header.

Relevant Code Map

- `src/App.tsx`: current route definitions, route matching, shell, sidebar, topbar, page scaffold, organization page config, profile/settings/notifications/calendar pages.
- `src/App.tsx:410`: current `PRIMARY_ROUTE_IDS`.
- `src/App.tsx:416`: current `USER_ROUTE_IDS`.
- `src/App.tsx:722`: `ORGANIZATION_PAGE_CONFIGS`, useful for Organization pin icons/kinds.
- `src/App.tsx:1428`: current `Sidebar`.
- `src/App.tsx:1481`: current `RouteNavLink`.
- `src/App.tsx:1506`: current `ProfileRouteLink`.
- `src/App.tsx:1530`: current `TopBar`.
- `src/App.tsx:1716`: current `PageScaffold`.
- `src/App.tsx:3055`: `formatOrganizationKind`.
- `src/App.tsx:3830`: current `KnowledgeNavigator`.
- `src/index.css`: current shell/sidebar/topbar/nav styling and responsive rules.
- `src/App.integrated.test.tsx`: existing integrated route and shell behavior tests.
- `src/prototypes/LayoutPrototype.tsx` and `src/prototypes/layoutPrototype.css`: earlier layout prototype references only; do not blindly copy.

Implementation Guidance

- Keep the first slice frontend-only and deterministic.
- Derive seeded pinned Organization pages from `appAccess.organizations`.
- Use at most one default seeded pin per `organizationKind` in this slice. If the same kind appears more than once, choose the first/currently most available item and leave true relevance ranking for later.
- Use `ORGANIZATION_PAGE_CONFIGS` or existing organization helpers for icons and display labels where practical.
- Keep existing route definitions so direct URLs continue to work, even if some routes are removed from fixed sidebar navigation.
- Prefer small view-model helpers for sidebar items and Active Role options rather than hard-coding JSX branches throughout `Sidebar`.
- For Active Role, use local React state in this slice. Default to the first available organization/role from `appAccess.organizations`; expose a dropdown/menu from the header display; do not persist it yet.

- Move `SignOutButton` out of the topbar and into the avatar/account menu.
- Move theme toggle out of the topbar if keeping the header to Active Role + Global Search requires it; avatar menu is a reasonable temporary home.
- Keep Notifications visibly reachable from the bottom sidebar with a badge. It can use existing static/client notification data for now if no backend count exists.
- Make labels responsive. Desktop should show labels; small screens may use compact rail/drawer behavior.
- Avoid broad refactors of page bodies. Touch page content only as needed to keep Knowledge Navigator behavior aligned.
- The worktree may already be dirty. Inspect diffs before editing and do not revert unrelated changes.

Test Plan

Use TDD where practical:

1. Add or update an integrated test proving the shell renders Dashboard first, Organization pins with labels, and bottom User View icons.
2. Add or update a test proving Notifications has a visible badge/count affordance.
3. Add or update a test proving Settings is available through the avatar menu and not as a persistent bottom icon.
4. Add or update a test proving Active Role switching does not navigate away.
5. Add or update a test proving User Views do not render the main Knowledge Navigator while Knowledge Pages still do.
6. Implement the smallest UI/style changes to pass.
7. Do a browser check at desktop and mobile widths.

Prefer public behavior tests with Testing Library/Vitest instead of private implementation tests.

Verification Commands

- `npx vitest run src/App.integrated.test.tsx`
- `npx tsc -b --pretty false`
- `npm run build`
- Manual browser check with Vite, for example `npm run dev -- --port 5178`, then inspect `/`, `/organizations/arche-classical-academy`, `/notifications`, `/calendar`, and `/profile` at desktop and mobile widths.

Risks / Open Questions

- Analytics and Smart Storage placement in the new nav was not resolved in the grill. Avoid making them prominent fixed sidebar items unless needed to preserve existing tests; keep routes reachable by URL.
- Durable pin, bookmark, and subscription persistence is intentionally out of scope.
- True overflow-by-available-height may be more work than this slice needs. A deterministic visible cap with `+N more` is acceptable for the first slice.
- Full Global Search across all accessible content is out of scope. Preserve or lightly adapt current search suggestions; do not build a search backend in this slice.
- Organization pin seeding uses existing access data; true relevance ranking among multiple Organizations of the same kind is future work.

Expected Final Response From Coding Agent

Summarize:

1. What changed in the header/sidebar.
2. What tests/checks passed.
3. Any docs that were updated or contradicted.
4. Any follow-up slices for durable pins/bookmarks/subscriptions or full search.

MVP Frontend Handoff: Answer Feed

Objective

Implement the Answer Feed using mixed fixture items so the MVP Explore/Contribute loop can show existing Answers and missing future Answers together.

Required Reading

- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `src/App.tsx`

Decisions To Preserve

- Feed items use a discriminated union:

```
type AnswerFeedItem =
  | { kind: "answer"; entry: KnowledgeEntrySummary }
  | { kind: "slot"; slot: KnowledgeSlotSummary };
```

- Existing Answers render through Knowledge Entry Card.
- Missing future Answers render through Knowledge Slot Card.
- A Context Match means an item contains every active Tag.
- Existing Answers should initially be ordered primarily by Human Weight.
- Broader Context Recommendations are Phase 2.

Suggested Scope

- Accept `activeTags` and a fixture `AnswerFeedItem[]`.
- Filter or display only items that fit the current Knowledge Context according to the shared helper.
- Render mixed feed items with the correct card component.
- Add clear empty states for no Answers and no Slots.
- Keep the feed independent from Convex until backend queries replace fixtures.

Out Of Scope

- Backend query performance work.
- Broader Context Recommendations.
- Smart Storage.
- Contribution Editor implementation.
- Full-text search ranking.

Acceptance Criteria

- Feed preserves the domain distinction between Answers and Knowledge Slots.
- Existing Answers are ordered by Human Weight for the MVP.
- Slots remain discoverable in the feed, not only in a side rail.
- Empty states lead naturally toward Contribute without inventing new domain terms.

Verification

- Component tests for mixed feed fixtures.
- Tests for Human Weight ordering.
- Tests for no-match and slot-only states.
- Run the smallest relevant test/typecheck command available in `package.json`.

MVP Frontend Handoff: Entry and Slot Cards

Objective

Implement Knowledge Entry Card and Knowledge Slot Card against fixture props, using the shared summary contracts from `docs/mvp-frontend-core-loop.md`.

Required Reading

- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `src/App.tsx`

Decisions To Preserve

- A Knowledge Entry can be considered an Answer, but Answer is not a Knowledge Type.
- A Knowledge Slot is a workflow request for a future Knowledge Entry, not an Answer or Knowledge Type.
- Cards receive compact summary objects, not full database documents.
- Card tests should use fixture props, not Convex mocks.

Suggested Scope

- Implement `KnowledgeEntryCard` from `KnowledgeEntrySummary`.
- Implement `KnowledgeSlotCard` from `KnowledgeSlotSummary`.
- Render Knowledge Type, title, preview/prompt, context Tag labels, status, target, due date, href, and Human Weight where applicable.
- Keep card components presentational and reusable by the Answer Feed and other page surfaces.

Out Of Scope

- Feed ordering.
- Convex data loading.
- Smart Storage.
- Fulfillment mutation behavior.
- Long Entry Representation rendering.

Acceptance Criteria

- Entry cards make Human Weight visible without implying it is an AI quality score.
- Slot cards make the requested Knowledge Type and contribution call to action clear.
- Slot cards remain visually and semantically distinct from Entry cards.
- Cards do not require Convex hooks or page state.

Verification

- Component tests using fixture `KnowledgeEntrySummary` and `KnowledgeSlotSummary` objects.
- Empty/optional field coverage for Slot prompt and due date.
- Run the smallest relevant test/typecheck command available in `package.json`.

MVP Frontend Handoff: Contracts and Routing

Objective

Create the shared frontend contracts and route/context parsing foundation for the MVP Explore/Contribute loop.

Required Reading

- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `src/App.tsx`

Decisions To Preserve

- Page-level loop state owns the active Knowledge Context.
- Active Tags update the URL automatically.
- Knowledge Requests do not automatically update the URL.
- Active Tags are an unordered set for Knowledge Context identity.
- Multi-Tag context keys and URLs should be canonicalized from sorted Tag IDs.
- `/` is Dashboard, `/scripture/:passageString` is a Bible Passage Referent Page, `/goto/:tagId` is a non-Scripture Referent Page, and `/explore?tagIds=a,b,c` is a multi-Tag Context Page.

Suggested Scope

- Add shared TypeScript contracts for `KnowledgeLoopState`, `ActiveTag`, `KnowledgeRequestDraft`, `KnowledgeEntrySummary`, `KnowledgeSlotSummary`, and `AnswerFeedItem`.
- Add route/context helpers that derive active Tags and location kind from the current URL.
- Add helpers that build canonical hrefs from active Tags.
- Keep helpers framework-light and testable outside React where practical.

Out Of Scope

- Backend queries.
- Smart Storage.
- Polished card/feed UI.
- KnowledgeRequestComposer behavior beyond contract shape.
- Contribution submission.

Acceptance Criteria

- Multi-Tag URLs canonicalize active Tags in a stable sorted order.
- Tag order does not affect context identity.
- Knowledge Request text is not serialized into the URL by default.

- Bible Passage route parsing stays compatible with the existing Scripture route behavior.
- Non-Scripture one-Tag routes use `/goto/:tagId`.

Verification

- Unit tests for route/context parsing and href construction.
- At least one test proving differently ordered `tagIds` resolve to the same context key.
- Run the smallest relevant test/typecheck command available in `package.json`.

MVP Frontend Handoff: Knowledge Navigator

Objective

Implement Knowledge Navigator behavior for active Tags and canonical URL updates.

Required Reading

- `CONTEXT.md`
- `docs/product-core.md`
- `docs/mvp-frontend-core-loop.md`
- `src/App.tsx`

Decisions To Preserve

- Knowledge Navigator is the user-facing control for selecting active Tags and determining the current Knowledge Context.
- Active Tags are URL state.
- Active Tags are an unordered set for Knowledge Context identity.
- Adding/removing active Tags should produce canonical routes.
- KnowledgeRequestComposer may propose Tags, but the Navigator/page-level state remains the source of truth for applied active Tags.

Suggested Scope

- Render active Tags as removable chips or controls.
- Support adding Tags from fixtures or a simple local search source until backend lookup exists.
- Update the URL when active Tags change.
- Use `/` for zero active Tags, `/scripture/:passageString` or `/goto/:tagId` for one active Tag, and `/explore?tagIds=a,b,c` for multiple active Tags.
- Record or expose enough events later for navigator usage analytics without implementing analytics in this slice.

Out Of Scope

- Backend Tag search.
- Tag recognition mutations.
- Page visit analytics.
- Question mapping.
- Contribution submission.

Acceptance Criteria

- Removing the last active Tag returns the user to `/`.
- One Bible Passage Tag routes to `/scripture/:passageString`.

- One non-Scripture Tag routes to `/goto/:tagId`.
- Multiple Tags route to `/explore?tagIds=` with stable sorted IDs.
- Selection order does not change the final context identity.

Verification

- Tests for add/remove active Tag behavior.
- Tests for route generation across zero, one, and multiple active Tags.
- Test proving selection order does not affect the canonical URL.
- Run the smallest relevant test/typecheck command available in `package.json`.